TUGAS BESAR IF3170 Inteligensi Artifisial

The Seekers

D

I

S

U

S

U

N

Oleh:

Audric Yusuf Maynard Simatupang - 13524010

Ishak Palentino Napitupulu - 13524022

Mahatma Brahmana - 13524015

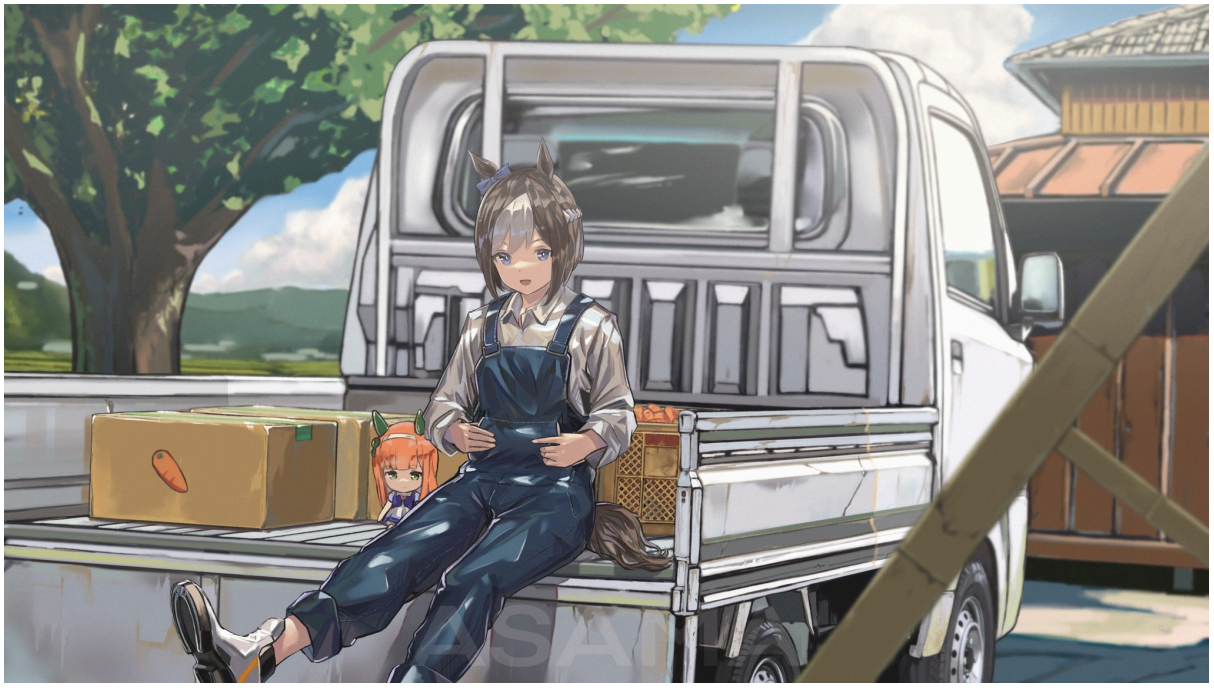
PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

2026

1. Deskripsi Persoalan....3
1.1. Latar Belakang Cerita....3
1.2. Gambaran Persoalan....3
1.3. Komponen Persoalan....4
1.4. Aturan Persoalan....4
1.5. Tujuan dan Ruang Lingkup Pengerjaan....5
2. Implementasi Algoritma....6
2.1. Gambaran Umum Program....6
2.1.1. Struktur Repository....6
2.1.2. Teknologi dan Arsitektur....6
2.1.3. Fitur Utama....7
2.2. Representasi Problem dan State....7
2.2.1. Problem....7
2.2.2. State....8
2.3. Fungsi Objektif....8
2.4. Hill-Climbing....9
2.4.1. Steepest Ascent Hill-Climbing....10
2.4.2. Stochastic Hill-Climbing....11
2.4.3. Hill-Climbing with Sideways Move....13
2.4.4. Random Restart Hill-Climbing....15
2.5. Simulated Annealing....17
2.6. Genetic Algorithm....22
2.7. Multi-Truk....26
3. Eksperimen dan Analisis....38
3.1. Eksperimen....38
3.2. Hill Climbing....38
3.3. Simulated Annealing....62
3.4. Genetic Algorithm....69
3.5. Analisis....107
3.5.1. Analisis Hill Climbing dan Simulated Annealing....107
3.5.2. Analisis Genetic Algorithm....108
3.5.3. Analisis Kedekatan terhadap Global Optimum....111
3.5.4. Analisis Konsistensi Hasil Akhir....113
4. Kesimpulan dan Saran....114
4.1. Kesimpulan....114
4.2. Saran....114
5. Pembagian Tugas....115
6. Lampiran....116
7. Daftar Pustaka....118

1. Deskripsi Persoalan

1.1. Latar Belakang Cerita



Gambar 1. Special Week, driver truk handal (Sumber: 真夏の銀堡垒🐴C108土曜日東地区 ホ-10a on X)

Keluarga Mejiro sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, Mejiro McQueen menawarkan posisi driver truk ujicoba kepada Special Week, karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama Mejiro Express dengan slogan “Lebih kilat dari Umamusume”.

Karena Special Week sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.

1.2. Gambaran Persoalan

Persoalan ini mirip dengan saat kita menyusun barang ke dalam koper atau bagasi mobil sebelum bepergian. Di persoalan ini, ada sekumpulan paket yang ukuran, nilai, dan berat yang berbeda-beda dan truk yang memiliki peti kemas dengan ukuran tetap sebagai tempat penyusunan paket. Tidak semua paket harus dimasukkan, sebagian boleh ditinggal di luar. Inilah tantangannya, untuk memilih paket mana yang layak masuk, dan bagaimana penataannya supaya muat di truk.

Jumlah kemungkinan susunannya sangat besar, karena setiap paket bisa dipilih atau tidak, diletakkan di banyak posisi, dan diputar ke beberapa orientasi. Mencoba semuanya satu

per satu tidak akan realistis. Karena itu, kami memakai pendekatan *local search*, yaitu mulai dari satu susunan awal acak, lalu memperbaikinya sedikit demi sedikit.

1.3. Komponen Persoalan

Permasalahan direpresentasikan menggunakan sekumpulan paket dan sebuah peti kemas truk. Setiap paket berbentuk kubus atau balok dan punya atribut berikut:

1. **Id** : Nama pengenal paket.
2. **Dimensi** : Lebar, panjang, dan tinggi paket.
3. **Orientasi** : Arah paket saat diletakkan. Ada enam kemungkinan orientasi, misalnya paket diputar 90° sehingga sisi lebar dan panjangnya bertukar.
4. **Value** : Nilai atau harga paket.
5. **Weight** : Berat paket.
6. **IsFragile** : Penanda apakah paket mudah pecah.
7. **ETA** : Perkiraan waktu kedatangan paket dalam sisa hari.

Peti kemas truk direpresentasikan menggunakan dimensi lebar, panjang, dan tinggi, serta kapasitas angkut maksimum. Posisi setiap paket dinyatakan menggunakan koordinat bilangan bulat. Representasi *state* harus dapat menunjukkan keadaan seluruh paket, baik yang berada di dalam maupun di luar peti kemas, konfigurasi peti kemas, serta nilai fungsi objektif pada konfigurasi tersebut.

Initial state dibangkitkan secara acak. Dalam setiap iterasi pencarian, konfigurasi dapat diubah melalui tiga jenis operasi *neighbor*, yaitu:

1. **Swap**: menukar posisi dua paket, baik paket yang berada di dalam maupun di luar peti kemas.
2. **Move**: memindahkan sebuah paket ke koordinat lain.
3. **Rotate**: memutar sebuah paket sebesar 90° terhadap salah satu sumbu.

Ketiga operasi tersebut digunakan untuk menjelajahi ruang solusi dan menemukan konfigurasi penempatan paket yang lebih baik.

1.4. Aturan Persoalan

Supaya susunan yang dihasilkan masuk akal di dunia nyata, ada beberapa aturan yang harus dipenuhi:

1. Paket tidak bisa melayang dan menembus dinding peti kemas. Paket harus berada sepenuhnya di dalam peti kemas, dan alasnya harus punya penopang, yaitu lantai truk atau bagian atas paket lain. Paket dianggap melayang kalau tidak ada bagian bawahnya yang bersisipan dengan paket lain atau lantai.
2. Barang pecah-belah tidak boleh di bawah paket lain. Paket fragile tidak boleh berada di bawah paket lain dan tidak boleh menjadi penopang.
3. Truk tidak boleh mengangkut melebihi kapasitasnya. Total berat paket di dalam truk tidak boleh melampaui kapasitas maksimum.

Seluruh aturan tersebut harus diperhitungkan dalam evaluasi konfigurasi agar proses pencarian menghasilkan solusi yang layak. Dengan demikian, keberhasilan optimasi tidak hanya ditentukan oleh besarnya total nilai paket, tetapi juga oleh kemampuan konfigurasi memenuhi ketentuan penempatan dan kapasitas yang berlaku.

1.5. Tujuan dan Ruang Lingkup Pengerjaan

Tujuan utama persoalan ini adalah untuk memaksimalkan total value paket yang berhasil dimuat ke dalam truk tanpa melanggar aturan yang ada. Dengan kata lain, kami mencari paket-paket yang paling berharga untuk dibawa, lalu menyusunnya agar semuanya tidak melanggar aturan yang ada. Paket bisa saja ditinggalkan di luar jika dianggap kurang menguntungkan.

Untuk mencapai hal tersebut, kami mengimplementasikan tiga algoritma local search, yaitu *Hill-Climbing*, *Simulated Annealing*, dan *Genetic Algorithm*. Untuk Hill-Climbing, kami mengerjakan semua variannya, yaitu *Steepest*, *Sideways*, *Stochastic*, dan *Random Restart*. Setiap algoritma akan dijalankan beberapa kali, lalu hasilnya akan dianalisis untuk melihat beberapa hal, yaitu seberapa dekat tiap algoritma bisa mendekati *global optima* dan apa penyebabnya, bagaimana perbandingan hasil dan durasi pencarian antar algoritma, serta seberapa konsisten hasil akhir yang didapat. Khusus untuk Genetic Algorithm, kami juga melihat pengaruh banyak iterasi dan jumlah populasi terhadap hasil pencarian.

Program yang kami buat berbentuk aplikasi berbasis web. Susunan yang dihasilkan akan ditampilkan di web dalam bentuk visualisasi, mencakup kondisi paket di dalam dan di luar truk, kondisi isi peti kemas, serta value dari susunan tersebut.

Selain spesifikasi wajib, kami juga mengerjakan dua bonus. Pertama fungsi objektif alternatif selain yang didefinisikan di spek, tanpa menambah variabel yang sudah ada. Kami membuat tiga alternatif, yaitu memaksimalkan jumlah paket yang dimuat (agar paket banyak terkirim), memaksimalkan total volume paket yang dimuat (agar ruang truk terpakai sebaik mungkin), dan memaksimalkan rasio total value terhadap total volume paket yang dimuat (nilai untuk value per volume paket). Kedua, multi-truk, sehingga program bisa menerima lebih dari satu truk dan memaksimalkan total value yang diangkut. Hasil dan analisisnya akan dibahas di bab berikutnya.

2. Implementasi Algoritma

2.1. Gambaran Umum Program

2.1.1. Struktur Repository

```
root/
├── README.md
├── docs/
│   └── .gitkeep
└── src/
    ├── backend/
    │   ├── algorithm/
    │   ├── core/
    └── frontend/
        └── src/
            └── visualization/
                ├── components/
                │   ├── 3d/
                │   └── ui/
                ├── style/
                └── utils/
```

- `README.md` : berisi petunjuk setup dan cara menjalankan program.
- `docs/` : folder untuk dokumentasi laporan.
- `src/` : folder utama kode program.
- backend/ : bagian server dan logika program Python.
- `algorithm/` : implementasi algoritma.
- `core/` : model paket, truk, dan state; validasi aturan; pembangkitan neighbor; fungsi objective; dan struktur hasil.
- `frontend/` : bagian antarmuka pengguna.
- `src/` : kode aplikasi React, termasuk dashboard.
- `visualization/` : komponen untuk menampilkan state penataan paket.
- `components/` : komponen visualisasi dan kontrol antarmuka.
- `3d/` : komponen untuk menampilkan susunan paket dan truk dalam tampilan 3D.
- `ui/` : komponen antarmuka.
- `style/` : CSS khusus tampilan visualisasi.
- `utils/` : fungsi bantu untuk mengolah data state dan menyusun objek visualisasi.

2.1.2. Teknologi dan Arsitektur

Backend menggunakan Python dan Flask, frontend menggunakan React, Vite menangani proses development dan build, Visualisasi 3D dibuat dengan Three.js, dan grafik objective menggunakan SVG langsung, tanpa library plotting khusus.

Frontend mengirim konfigurasi eksperimen melalui `POST /api/experiments`. Lalu backend membangkitkan problem dan initial

state, menjalankan algoritma, lalu frontend mengambil status dan hasil melalui `GET /api/experiments`. Hasil dapat dibuka sebagai detail, disimpan, dihapus, atau diekspor sebagai JSON.

2.1.3. Fitur Utama

Di website, pengguna dapat mengatur jumlah paket, truk, iterasi, seed, objective, serta parameter khusus algoritma seperti ukuran populasi GA dan temperatur SA.

Hasil di website akan menampilkan nilai awal, akhir, dan terbaik, grafik objective, iterasi, durasi, serta seed. Initial, final, dan best state dapat dilihat dalam bentuk tabel.

2.2. Representasi Problem dan State

2.2.1. Problem

```
class Package:
    def __init__(
        self,
        package_id,
        dimensions,
        value,
        weight,
        is_fragile,
        eta,
    ):
        self.id = package_id
        self.dimensions = dimensions
        self.value = value
        self.weight = weight
        self.is_fragile = is_fragile
        self.eta = eta

class Truck:
    def __init__(self, truck_id, dimensions, max_capacity):
        self.id = truck_id
        self.dimensions = dimensions
        self.max_capacity = max_capacity

class Problem:
    def __init__(self, trucks, packages):
        self.trucks = list(trucks)
        self.packages = list(packages)
        self.trucks_by_id = {truck.id: truck for truck in
self.trucks}
        self.packages_by_id = {package.id: package for package
in self.packages}
```

Problem menyimpan data tetap untuk masalah penyusunan paket, yang berisikan daftar truk (trucks) dan daftar paket (packages). Keduanya diindekskan berdasarkan ID yang unik agar truk atau paket bisa dicari dengan cepat.

Setiap paket memiliki ID, dimensi, nilai, berat, status mudah pecah, dan ETA. Setiap truk memiliki ID, dimensi, dan kapasitas berat. Orientasi paket menentukan bagaimana dimensi paket diputar saat ditempatkan.

2.2.2. State

```
class Placement:
    def __init__(
        self,
        package_id,
        position=None,
        orientation=Orientation.WLH,
        truck_id=None,
    ):
        self.package_id = package_id
        self.position = position
        self.orientation = orientation
        self.truck_id = truck_id


class State:
    def __init__(self, placements):
        self.placements = list(placements)
```

State adalah satu kandidat solusi. Isinya berupa list placements, dengan satu Placement untuk setiap paket. Setiap placement mencatat ID paket, orientasi, truk tujuan, dan koordinat posisi paket. State harus memuat setiap paket tepat satu kali, jadi state harus memuat setiap placement tepat satu kali juga. Pemeriksa validitas juga memastikan paket berada di dalam batas truk, tidak bertabrakan, memenuhi aturan support, dan tidak melebihi kapasitas truk.

Jika paket dimuat, Placement menyimpan ID truk dan posisi (x, y, z). Jika paket belum dimuat, `truck_id` dan position sama-sama bernilai `None`.

2.3. Fungsi Objektif

Fungsi objektif digunakan untuk memberi nilai pada suatu state berdasarkan paket yang berhasil dimuat ke truk. Program mengambil paket yang berada di dalam truk, lalu menghitung nilainya sesuai pilihan fungsi objektif.

```
#value paket dimuat
def evaluate_obj_func(problem, state):
    return sum(
        problem.get_package(placement.package_id).value
        for placement in state.get_inside_placements()
    )

#total paket dimuat
def evaluate_obj_func2(problem, state):
    return len(state.get_inside_placements())

#total volume paket yang dimuat
def evaluate_obj_func3(problem, state):
```

```
    load_volume = 0

    for placement in state.get_inside_placements():
        package = problem.get_package(placement.package_id)
        load_volume += (
            package.dimensions.width *
package.dimensions.length * package.dimensions.height)

    return load_volume

#total volume paket
def evaluate_obj_func4(problem, state):
    tot_value = 0
    load_volume = 0

    for placement in state.get_inside_placements():
        package = problem.get_package(placement.package_id)
        tot_value += package.value
        load_volume += (package.dimensions.width *
package.dimensions.length * package.dimensions.height)

    if load_volume == 0:
        return 0

    return tot_value / load_volume
```

- `evaluate_obj_func`
  Total nilai paket, menjumlahkan nilai semua paket yang dimuat. Ini menjadi pilihan bawaan.
- `evaluate_obj_func2`
  Total paket, menghitung banyaknya paket yang dimuat.
- `evaluate_obj_func3`
  Total volume paket, menjumlahkan volume paket yang dimuat, dihitung dari lebar × panjang × tinggi.
- `evaluate_obj_func4`
  Nilai per volume, membagi total nilai paket yang dimuat dengan total volumenya.

2.4. Hill-Climbing

Hill Climbing bertujuan untuk mencari susunan paket neighbor yang nilai fungsi objektifnya meningkat. Algoritma mulai dari sebuah state, membentuk neighbor, lalu memutuskan apakah akan berpindah ke salah satunya.

Secara umum, implementasi Hill Climbing menerima problem dan state awal, lalu menyalin state tersebut agar state input tetap utuh. Algoritma menghitung nilai objektif state awal, kemudian berulang kali membentuk neighbor. Neighbor di konteks ini adalah state lain yang diperoleh dengan mengubah penempatan paket dari state saat ini. Neighbor yang nantinya dihasilkan akan diperiksa terlebih dahulu validitasnya dan dihitung nilai objektifnya. Algoritma ini memiliki empat jenis varian, algoritma memilih neighbor dan menentukan apakah akan berpindah ke sana tergantung jenis variannya. Proses selesai ketika mencapai batas iterasi atau kondisi

berhenti di varian tersebut. Hasil pencarian mencatat state awal, state akhir, nilai objektif, riwayat nilai objektif, dan durasi eksekusi.

Pembentukan neighbor memakai operasi move, rotate, dan swap. `generate_all_neighbors` mencoba menghasilkan semua neighbor valid, sedangkan `generate_random_neighbor` mencoba memilih satu neighbor secara acak. Kedua fungsi ini juga memastikan state dari neighbor yang dihasilkan valid.

Berikut adalah empat jenis varian algoritmanya:

2.4.1. Steepest Ascent Hill-Climbing

Steepest Ascent memeriksa semua neighbor valid dari state saat ini, lalu berpindah ke neighbor dengan nilai objektif tertinggi. Jika tidak ada neighbor yang lebih baik, pencarian berhenti meskipun jumlah iterasi maksimum belum tercapai.

Potongan kode utama:

```
def hill_climbing_steepest_ascent(
    problem,
    state,
    max_iterations=1000,
    seed=None,
    objective_number=1,
):
    from time import perf_counter

    if not isinstance(max_iterations, int) or
max_iterations < 0:
        raise ValueError("max_iteration tidak valid")
    if not is_valid_state(problem, state):
        raise ValueError("initital state tidak valid")

    started_at = perf_counter()
    initial_state = state.copy()
    initial_value = evaluate_state(problem,
initial_state, objective_number)
    cur_state = initial_state.copy()
    cur_value = initial_value
    iterations = 0
    objective_history = [initial_value]
    termination_reason = "telah mencapai iterasi
maksimum"

    while iterations < max_iterations:
        best_neighbor = None
        best_neighbor_value = cur_value

        for neighbor in generate_all_neighbors(problem,
cur_state):
            neighbor_value = evaluate_state(problem,
neighbor, objective_number)
            if neighbor_value > best_neighbor_value:
```

```
                best_neighbor = neighbor
                best_neighbor_value = neighbor_value

        if best_neighbor is None:
            termination_reason = "tidak ada neighbor
yang valuenya lebih tinggi"
            break

        cur_state = best_neighbor
        cur_value = best_neighbor_value
        iterations += 1
        objective_history.append(cur_value)

    final_state = cur_state.copy()

    return Result(
        "Hill Climbing - Steepest Ascent",
        initial_state,
        final_state,
        best_state=final_state.copy(),
        initial_value=initial_value,
        final_value=cur_value,
        best_value=cur_value,
        iterations=iterations,
        objective_history=objective_history,
        execution_time=perf_counter() - started_at,
        termination_reason=termination_reason,
        problem=problem,
    )
```

Pada implementasi Steepest Ascent Hill-Climbing, fungsi membangkitkan seluruh neighbor yang valid dari state saat ini menggunakan generate_all_neighbors(problem, cur_state). Setiap neighbor kemudian dievaluasi menggunakan evaluate_state(problem, neighbor, objective_number) untuk memperoleh nilai fungsi objektifnya. Melalui kondisi if neighbor_value > best_neighbor_value, algoritma memilih neighbor dengan nilai objektif tertinggi yang lebih baik daripada state saat ini. Proses ini dilakukan secara berulang hingga mencapai batas maksimum iterasi yang ditentukan oleh while iterations < max_iterations, atau ketika tidak ditemukan lagi neighbor dengan nilai objektif yang lebih tinggi, yang ditandai dengan kondisi if best_neighbor is None. Pada kondisi tersebut, pencarian dihentikan menggunakan perintah break.

2.4.2. Stochastic Hill-Climbing

Stochastic Hill Climbing mengambil satu neighbor secara acak pada tiap iterasi, beda dengan Steepest Ascent yang mengevaluasi semua neighbor lain. Neighbor akan hanya diterima jika nilai objektifnya lebih baik. Jika tidak, algoritma akan tetap memakai state saat ini dan melanjutkan iterasi.

```
def hill_climbing_stochastic(
    problem,
    state,
```

```
    max_iterations=1000,
    seed=None,
    objective_number=1,
):
    from random import Random
    from time import perf_counter

    if not isinstance(max_iterations, int) or
max_iterations < 0:
        raise ValueError("max_iteration tidak valid")
    if not is_valid_state(problem, state):
        raise ValueError("initial state tidak valid")

    started_at = perf_counter()
    initial_state = state.copy()
    initial_value = evaluate_state(problem,
initial_state, objective_number)
    cur_state = initial_state.copy()
    cur_value = initial_value
    iterations = 0
    objective_history = [initial_value]
    termination_reason = "telah mencapai iterasi
maksimum"
    rng = Random(seed)

    while iterations < max_iterations:
        try:
            neighbor = generate_random_neighbor(problem,
cur_state, seed=seed, rng=rng)
        except RuntimeError:
            neighbor = None

        if neighbor is not None:
            neighbor_value = evaluate_state(problem,
neighbor, objective_number)
            if neighbor_value > cur_value:
                cur_state = neighbor
                cur_value = neighbor_value

        iterations += 1
        objective_history.append(cur_value)

    final_state = cur_state.copy()

    return Result(
        "Hill Climbing - Stochastic",
        initial_state,
        final_state,
        best_state=final_state.copy(),
        initial_value=initial_value,
        final_value=cur_value,
        best_value=cur_value,
        iterations=iterations,
        objective_history=objective_history,
        execution_time=perf_counter() - started_at,
        termination_reason=termination_reason,
        problem=problem,
    )
```

Pada implementasi Stochastic Hill-Climbing, fungsi membangkitkan satu neighbor secara acak dari state saat ini menggunakan generate_random_neighbor(problem, cur_state, seed=seed, rng=rng). Jika neighbor berhasil dibangkitkan, nilai fungsi objektifnya dievaluasi menggunakan evaluate_state(problem, neighbor, objective_number). Melalui kondisi if neighbor_value > cur_value, algoritma hanya berpindah ke neighbor apabila nilai objektifnya lebih tinggi daripada state saat ini. Jika nilai neighbor tidak lebih baik atau tidak ada neighbor yang berhasil dibangkitkan, algoritma mempertahankan state saat ini dan melanjutkan ke iterasi berikutnya. Proses ini dilakukan secara berulang hingga mencapai batas maksimum iterasi yang ditentukan oleh while iterations < max_iterations.

2.4.3. Hill-Climbing with Sideways Move

Sideways Move mirip dengan Steepest Ascent karena memeriksa. Bedanya, algoritma boleh berpindah ke neighbor dengan nilai objektif yang sama. Jumlah perpindahan ke neighbor dengan nilai objektif yang sama ini dibatasi oleh parameter `max_sideways`, maximum perpindahan ke neighbor dengan nilai objektif yang sama, agar algoritma tidak terus bergerak di area dengan nilai sama.

```
def hill_climbing_sideways_move(
    problem,
    state,
    max_iterations=1000,
    seed=None,
    objective_number=1,
    max_sideways=100,
):
    from time import perf_counter

    if not isinstance(max_iterations, int) or
max_iterations < 0:
        raise ValueError("max_iteration tidak valid")
    if not isinstance(max_sideways, int) or max_sideways
< 0:
        raise ValueError("max_sideways tidak valid")
    if not is_valid_state(problem, state):
        raise ValueError("initial state tidak valid")

    started_at = perf_counter()
    initial_state = state.copy()
    initial_value = evaluate_state(problem,
initial_state, objective_number)
    cur_state = initial_state.copy()
    cur_value = initial_value
    iterations = 0
    sideways_moves = 0
    objective_history = [initial_value]
    termination_reason = "telah mencapai iterasi
maksimum"

    while iterations < max_iterations:
```

```
        best_neighbor = None
        best_neighbor_value = cur_value

        for neighbor in generate_all_neighbors(problem,
cur_state):
            neighbor_value = evaluate_state(problem,
neighbor, objective_number)
            if neighbor_value >= best_neighbor_value:
                best_neighbor = neighbor
                best_neighbor_value = neighbor_value

        if best_neighbor is None:
            termination_reason = "tidak ada neighbor
yang valuenya lebih tinggi atau sama"
            break

        is_sideways = best_neighbor_value == cur_value
        if is_sideways:
            if sideways_moves >= max_sideways:
                termination_reason = "telah mencapai
sideways move maksimum"
                break
            sideways_moves += 1

        cur_state = best_neighbor
        cur_value = best_neighbor_value
        iterations += 1
        objective_history.append(cur_value)

        if is_sideways and sideways_moves >=
max_sideways:
            termination_reason = "telah mencapai
sideways move maksimum"
            break

    final_state = cur_state.copy()

    return Result(
        "Hill Climbing - Sideways Move",
        initial_state,
        final_state,
        best_state=final_state.copy(),
        initial_value=initial_value,
        final_value=cur_value,
        best_value=cur_value,
        iterations=iterations,
        objective_history=objective_history,
        execution_time=perf_counter() - started_at,
        termination_reason=termination_reason,
        metrics={
            "configured_max_sideways": max_sideways,
            "sideways_moves": sideways_moves,
        },
        problem=problem,
    )
```

Pada implementasi Hill-Climbing with Sideways Move, proses pembangkitan dan evaluasi neighbor dilakukan dengan cara yang sama seperti Steepest Ascent Hill-Climbing, yaitu menggunakan

generate_all_neighbors(problem, cur_state) dan evaluate_state(problem, neighbor, objective_number). Perbedaannya terletak pada kondisi if neighbor_value >= best_neighbor_value, yang memungkinkan algoritma memilih neighbor dengan nilai objektif lebih tinggi atau sama dengan state saat ini. Jika nilai objektif neighbor sama dengan state saat ini, kondisi if is_sideways akan terpenuhi dan jumlah sideways move ditambah melalui sideways_moves += 1. Jumlah perpindahan tersebut dibatasi oleh parameter max_sideways sehingga ketika kondisi sideways_moves >= max_sideways terpenuhi, pencarian dihentikan menggunakan perintah break. Selain itu, algoritma juga berhenti apabila mencapai batas maksimum iterasi atau tidak ditemukan lagi neighbor dengan nilai objektif yang lebih tinggi maupun sama.

2.4.4. Random Restart Hill-Climbing

Random Restart menjalankan Steepest Ascent beberapa kali, masing-masing dari state awal yang dibangkitkan secara acak. Tujuannya agar pencarian mencoba titik awal yang berbeda, karena ada kemungkinan dari satu state awal yang dibangkitkan, bisa stuck di local optima. Setelah semua pencarian selesai, algoritma akan mengembalikan hasil terbaik yang ditemukan.

```
def hill_climbing_random_restart(
    problem,
    state,
    max_iterations=1000,
    seed=None,
    objective_number=1,
    max_restarts=5,
):
    from random import Random
    from time import perf_counter

    if not isinstance(max_iterations, int) or
max_iterations < 0:
        raise ValueError("max_iteration tidak valid")
    if not isinstance(max_restarts, int) or max_restarts
< 0:
        raise ValueError("max_restarts tidak valid")

    started_at = perf_counter()
    rng = Random(seed)
    best_result = None
    initial_state = None
    objective_history = []
    iterations_per_restart = []

    for _ in range(max_restarts + 1):
        random_state = generate_initial_state(problem,
seed=seed, rng=rng)
        result = hill_climbing_steepest_ascent(
            problem,
            random_state,
            max_iterations=max_iterations,
```

```
            seed=seed,
            objective_number=objective_number,
        )

        if initial_state is None:
            initial_state = result.initial_state.copy()
        if best_result is None or result.best_value >
best_result.best_value:
            best_result = result

        iterations_per_restart.append(result.iterations)
        for value in result.objective_history:
            objective_history.append(
                max(objective_history[-1], value) if
objective_history else value
            )

    final_state = best_result.best_state.copy()

    return Result(
        "Hill Climbing - Random Restart",
        initial_state,
        final_state,
        best_state=final_state.copy(),
        initial_value=objective_history[0],
        final_value=best_result.best_value,
        best_value=best_result.best_value,
        iterations=len(objective_history) - 1,
        objective_history=objective_history,
        execution_time=perf_counter() - started_at,
        termination_reason="telah mencapai restart
maksimum",
        metrics={
            "configured_max_restarts": max_restarts,
            "completed_restarts":
len(iterations_per_restart) - 1,
            "iterations_per_restart":
iterations_per_restart,
            "total_search_iterations":
sum(iterations_per_restart),
        },
        problem=problem,
    )
```

Pada implementasi Random Restart Hill-Climbing, fungsi membangkitkan state awal secara acak menggunakan generate_initial_state(problem, seed=seed, rng=rng). State tersebut kemudian digunakan sebagai input untuk menjalankan algoritma Steepest Ascent Hill-Climbing melalui hill_climbing_steepest_ascent(problem, random_state, max_iterations=max_iterations, seed=seed, objective_number=objective_number). Setelah proses pencarian selesai, hasilnya dibandingkan dengan hasil terbaik sebelumnya melalui kondisi if best_result is None or result.best_value > best_result.best_value. Jika hasil yang diperoleh lebih baik, algoritma akan memperbarui best_result dengan hasil tersebut. Proses pembangkitan state acak dan pencarian menggunakan Steepest Ascent ini dilakukan secara berulang sesuai batas restart yang

ditentukan oleh for _ in range(max_restarts + 1). Setelah seluruh proses restart selesai, algoritma mengembalikan state terbaik yang ditemukan selama pencarian.

2.5. Simulated Annealing

Simulated Annealing (SA) mencari susunan paket dengan melakukan perpindahan dari satu state ke state tetangga secara bertahap. Neighbor dengan nilai objektif lebih tinggi atau sama akan langsung diterima. Sementara itu, neighbor yang lebih buruk masih memiliki peluang untuk diterima berdasarkan probabilitas $e^{\Delta E/T}$. Mekanisme tersebut memungkinkan SA keluar dari solusi lokal dan menjelajahi ruang pencarian secara lebih luas. Nilai temperatur diturunkan pada setiap iterasi menggunakan geometric cooling hingga mencapai temperatur minimum atau batas maksimum iterasi.

```
def simulated_annealing(
    problem,
    state,
    max_iterations=1000,
    seed=None,
    objective_number=1,
    initial_temperature=100.0,
    cooling_rate=0.99,
    minimum_temperature=0.01,
    cancel_event=None,
):
    # Validasi parameter
    if not isinstance(max_iterations, int) or
isinstance(max_iterations, bool):
        raise ValueError("max_iterations must be an
integer")
    if max_iterations < 0:
        raise ValueError("max_iterations must not be
negative")

    temperatures = {
        "initial_temperature": initial_temperature,
        "cooling_rate": cooling_rate,
        "minimum_temperature": minimum_temperature,
    }

    for name, value in temperatures.items():
        if isinstance(value, bool) or not
isinstance(value, (int, float)):
            raise ValueError(f"{name} must be a
number")
        if not isfinite(value):
            raise ValueError(f"{name} must be finite")

    if initial_temperature <= 0:
        raise ValueError("initial_temperature must be
```

```
greater than zero")
    if minimum_temperature <= 0:
        raise ValueError("minimum_temperature must be
greater than zero")
    if minimum_temperature >= initial_temperature:
        raise ValueError(
            "minimum_temperature must be lower than
initial_temperature"
        )
    if not 0 < cooling_rate < 1:
        raise ValueError("cooling_rate must be between
zero and one")
    if not is_valid_state(problem, state):
        raise ValueError("initial state is invalid")

    # Inisialisasi pencarian
    started_at = perf_counter()
    rng = Random(seed)

    initial_state = state.copy()
    initial_value = evaluate_state(
        problem,
        initial_state,
        objective_number,
    )

    current_state = initial_state.copy()
    current_value = initial_value

    best_state = initial_state.copy()
    best_value = initial_value

    temperature = float(initial_temperature)

    # Inisialisasi metrik
    iterations = 0
    accepted_moves = 0
    accepted_worse_moves = 0
    rejected_moves = 0
    improving_moves = 0
    stuck_iterations = 0

    stuck_history = []
    objective_history = [current_value]
    current_objective_history = [current_value]
    best_objective_history = [best_value]
    temperature_history = [temperature]
    acceptance_probability_history = []

    termination_reason = "maximum iterations reached"

    # Proses utama Simulated Annealing
    while (
        iterations < max_iterations
```

```
        and temperature > minimum_temperature
    ):
        _raise_if_cancelled(cancel_event)

        try:
            neighbor = generate_random_neighbor(
                problem,
                current_state,
                rng=rng,
            )
        except RuntimeError:
            termination_reason = (
                "failed to generate a valid neighbor"
            )
            break

        # Evaluasi neighbor
        neighbor_value = evaluate_state(
            problem,
            neighbor,
            objective_number,
        )
        delta_energy = neighbor_value - current_value

        # Neighbor lebih baik atau sama langsung
diterima.
        # Neighbor lebih buruk menggunakan
probabilitas e^(delta/T).
        acceptance_probability = (
            1.0
            if delta_energy >= 0
            else exp(delta_energy / temperature)
        )

        accepted = (
            delta_energy >= 0
            or rng.random() < acceptance_probability
        )

        if accepted:
            current_state = neighbor
            current_value = neighbor_value
            accepted_moves += 1

            if delta_energy > 0:
                improving_moves += 1
            elif delta_energy < 0:
                accepted_worse_moves += 1
        else:
            # Proposal ditolak sehingga state tidak
berubah.
            rejected_moves += 1
            stuck_iterations += 1
```

```
        # Memperbarui solusi terbaik sepanjang
pencarian
        if current_value > best_value:
            best_state = current_state.copy()
            best_value = current_value

        # Mencatat riwayat dan menurunkan temperatur
        iterations += 1
        stuck_history.append(not accepted)
        temperature *= cooling_rate

        acceptance_probability_history.append(
            acceptance_probability
        )

current_objective_history.append(current_value)
        objective_history.append(current_value)
        best_objective_history.append(best_value)
        temperature_history.append(temperature)

    # Menentukan alasan terminasi
    if iterations >= max_iterations:
        termination_reason = "maximum iterations
reached"
    elif temperature <= minimum_temperature:
        termination_reason = "minimum temperature
reached"

    # Menghitung frekuensi stuck
    final_state = current_state.copy()
    rejection_rate = (
        rejected_moves / iterations
        if iterations
        else 0.0
    )
    stuck_frequency = rejection_rate

    # Mengembalikan hasil dan metrik eksperimen
    return Result(
        "Simulated Annealing",
        initial_state,
        final_state,
        best_state=best_state.copy(),
        initial_value=initial_value,
        final_value=current_value,
        best_value=best_value,
        iterations=iterations,
        objective_history=objective_history,
        execution_time=perf_counter() - started_at,
        termination_reason=termination_reason,
        metrics={
            "acceptance_probability_history":
                acceptance_probability_history,
            "current_objective_history":
```

```
                current_objective_history,
            "best_objective_history":
                best_objective_history,
            "temperature_history":
                temperature_history,
            "stuck_frequency":
                stuck_frequency,
            "stuck_iterations":
                stuck_iterations,
            "stuck_history":
                stuck_history,
            "stuck_frequency_definition": (
                "Fraction of completed iterations
whose "
                "proposed neighbor was rejected,
leaving "
                "the current state unchanged."
            ),
            "rejection_rate":
                rejection_rate,
            "accepted_moves":
                accepted_moves,
            "accepted_worse_moves":
                accepted_worse_moves,
            "rejected_moves":
                rejected_moves,
            "improving_moves":
                improving_moves,
        },
        problem=problem,
    )
```

SA dimulai dengan menyalin state awal dan menghitung nilai objektifnya menggunakan evaluate_state. State tersebut digunakan sebagai current_state sekaligus best_state pertama. Pada setiap iterasi, satu state tetangga yang valid dibuat secara acak menggunakan generate_random_neighbor, kemudian nilai objektif neighbor dibandingkan dengan nilai current state.

Selisih nilai dihitung menggunakan ΔE = E(neighbor) - E(current). Jika ΔE >= 0, neighbor langsung diterima. Jika $\Delta E < 0$, neighbor diterima dengan probabilitas $P = \exp(\Delta E / T)$. Nilai acak antara 0 dan 1 digunakan untuk menentukan apakah perpindahan ke solusi yang lebih buruk tersebut diterima. Semakin rendah temperatur, semakin kecil peluang SA menerima solusi yang lebih buruk.

Setelah setiap iterasi, temperatur diturunkan menggunakan persamaan T(baru) = T(lama) x cooling_rate. Proses dihentikan ketika jumlah iterasi mencapai max_iterations, temperatur mencapai minimum_temperature, tidak ditemukan neighbor valid, atau eksekusi dibatalkan oleh pengguna.

Implementasi menyimpan current_state sebagai state yang sedang dikunjungi dan best_state sebagai state terbaik yang pernah ditemukan. Oleh karena SA dapat

menerima solusi yang lebih buruk, nilai current state dapat menurun selama pencarian. Namun, nilai best state tidak akan menurun karena hanya diperbarui ketika ditemukan solusi dengan nilai objektif yang lebih tinggi.

Pada implementasi ini, satu iterasi didefinisikan sebagai stuck ketika proposal neighbor ditolak sehingga current state tidak berubah. Oleh karena itu, stuck_iterations sama dengan jumlah proposal yang ditolak dan stuck_frequency dihitung menggunakan:

$$"stuck_frequency" = \frac{\text{rejected_moves}}{\text{completed_iterations}}$$

Definisi tersebut tidak mengaitkan kondisi stuck dengan local optimum karena SA tetap dapat berpindah ke state dengan kualitas sama maupun lebih buruk. Selain frekuensi stuck, hasil algoritma juga menyimpan jumlah proposal yang diterima, proposal yang ditolak, solusi lebih buruk yang diterima, riwayat temperatur, riwayat probabilitas penerimaan, riwayat nilai current state, dan riwayat nilai terbaik.

Hasil akhir SA mencakup initial state, final state, best state, nilai objektif masing-masing state, jumlah iterasi, durasi eksekusi, alasan terminasi, serta seluruh metrik yang diperlukan untuk grafik dan analisis eksperimen.

2.6. Genetic Algorithm

Genetic Algorithm (GA) bekerja dengan cara mencari susunan paket dengan mengembangkan sekumpulan kandidat state, yang disebut sebagai populasi. Setiap kandidatnya adalah sebuah state. Pada tiap iterasinya, GA akan memilih dua kandidat sebagai parent lalu menggabungkan susunannya untuk membuat anak. Anak yang dihasilkan tersebut memiliki probabilitas untuk bermutasi.

```
def genetic_algorithm(
    problem,
    state,
    max_iterations=1000,
    seed=None,
    objective_number=1,
    population_size=20,
):
    MAX_INVALID_CHILD = population_size * 10
    if (not isinstance(max_iterations, int) or max_iterations
< 0 ):
        raise ValueError("max_iteration tidak valid")
    if (not isinstance(population_size, int) or
population_size < 2 ):
        raise ValueError("population_size tidak valid")
    if not is_valid_state(problem, state):
        raise ValueError("initial state tidak valid")

    started_at = perf_counter()
```

```
    rng = random.Random(seed)
    population = [state.copy()]
    while len(population) < population_size:
        population.append(generate_initial_state(problem,
rng=rng))

    fitness = []
    for individual in population:
        temp = evaluate_state(problem, individual,
objective_number)
        fitness.append(temp)

    initial_value = fitness[0]
    best_index = 0
    max_fitness = fitness[0]
    tot_fitness = 0

    for i in range(population_size):
        tot_fitness += fitness[i]

        if fitness[i] > max_fitness:
            max_fitness = fitness[i]
            best_index = i

    max_fitness_history = [max_fitness]
    avr_fitness_history = [tot_fitness / population_size]
    best_state = population[best_index].copy()
    best_value = fitness[best_index]

    iterations = 0

    while iterations < max_iterations:
        next_population = []
        invalid_child = 0

        while len(next_population) < population_size:
            first_parent = _select_parent(population, fitness,
tot_fitness, rng)
            second_parent = _select_parent(population,
fitness, tot_fitness, rng)
            first_child, second_child = _crossover(problem,
first_parent, second_parent, rng)

            for child in (first_child, second_child):
                if not is_valid_state(problem, child):
                    invalid_child += 1

                    if invalid_child >= MAX_INVALID_CHILD:

next_population.append(first_parent.copy())
                        invalid_child = 0

                    if len(next_population) ==
population_size:
                        break
                    continue

                mutation_rate = rng.uniform(MIN_MUTATION_RATE,
MAX_MUTATION_RATE)
                if rng.random() < mutation_rate:
```

```
                    child = _mutate(problem, child, rng)

                next_population.append(child)

                if len(next_population) == population_size:
                    break

        population = next_population
        fitness = []
        for individual in population:
            temp = evaluate_state(problem, individual,
objective_number)
            fitness.append(temp)

        best_index = 0
        max_fitness = fitness[0]
        tot_fitness = 0

        for index in range(population_size):
            tot_fitness += fitness[index]

            if fitness[index] > max_fitness:
                max_fitness = fitness[index]
                best_index = index

        if fitness[best_index] > best_value:
            best_value = fitness[best_index]
            best_state = population[best_index].copy()

        max_fitness_history.append(max_fitness)
        avr_fitness_history.append(tot_fitness /
population_size)
        iterations += 1

    final_index = 0
    for index in range(population_size):
        if fitness[index] > fitness[final_index]:
            final_index = index

    final_state = population[final_index].copy()
    final_value = fitness[final_index]

    return Result(
        "Genetic Algorithm",
        state.copy(),
        final_state,
        best_state=best_state,
        initial_value=initial_value,
        final_value=final_value,
        best_value=best_value,
        iterations=iterations,
        objective_history=max_fitness_history.copy(),
        execution_time=perf_counter() - started_at,
        termination_reason="telah mencapai iterasi maksimum",
        metrics={
            "population_size": population_size,
            "maximum_fitness_history": max_fitness_history,
            "average_fitness_history": avr_fitness_history,
        },
        problem=problem,
```

```
    )
```

Di awal, GA memulai populasi dengan salinan state yang diterima sebagai input. Jika jumlah populasinya belum mencapai `population_size`, GA akan menambahkan state untuk dimasukkan ke populasi yang dibuat oleh `generate_initial_state`. Jadi state input digunakan sebagai salah satu anggota populasi awal.

Setelah populasinya terbentuk, GA menghitung fitness setiap kandidat di populasi menggunakan `evaluate_state`. Nilai fitness akan mengikuti fungsi objektif yang dipilih.

Setelah itu, lanjut dengan pemilihan parent menggunakan fungsi `_select_parent` dengan metode roulette wheel seperti di PPT, yang dimana kandidat dengan fitness lebih tinggi mendapat peluang terpilih lebih besar. Jika total fitness tidak positif, parent akan dipilih secara acak agar kode tetap berjalan.

Dilanjutkan dengan crossover melalui fungsi `_crossover` yang memilih satu titik potong acak (crosspoint). Fungsi `_make_child` lalu mengambil data penempatan paket sebelum titik potong dari satu parent dan penempatan sisanya dari parent lain. Kedua anak dibuat dengan urutan induk yang dibalik.

Karena penempatan dari dua induk digabungkan, anak yang terbentuk bisa melanggar aturan state. Jadi, GA memeriksa validitas anak terlebih dahulu dengan `is_valid_state`, dan anak yang tidak valid tidak dimasukkan ke populasi berikutnya. Jika jumlah percobaan anak tidak valid mencapai `population_size * 10` dalam satu iterasi, GA akan memasukkan salinan parent sebagai pengganti agar proses pembentukan populasi dapat berlanjut.

Setiap anak yang valid memiliki peluang mutasi yang diacak antara 1% dan 5%. Jika mutasi terjadi, fungsi `_mutate` akan berjalan untuk memilih satu paket dan mencoba salah satu perubahan berikut:

- Rotate: mengubah orientasi paket.
- Move: memindahkan paket, termasuk mengeluarkannya dari truk atau menempatkannya pada posisi baru.
- Swap: menukar penempatan dua paket, pilihan ini tersedia jika ada setidaknya dua paket.

Operasi perubahan memakai fungsi `rotate`, `move`, dan `swap` yang menghasilkan state baru jika perubahannya valid. Untuk pemindahan ke posisi baru, `_try_random_placement` mencoba truk dan posisi secara acak, termasuk kemungkinan posisi bertumpuk.

Setelah populasi baru terbentuk, GA akan menghitung kembali fitness seluruh anggotanya. Populasi lama akan diganti dengan populasi baru, dan GA mencatat fitness maksimum dan rata-rata populasi pada setiap generasi, serta

menyimpan `best_state` dan `best_value` dari populasi terbaik yang ditemukan selama pencarian.

Proses ini diulang terus menerus sampai jumlah iterasi mencapai `max_iterations`. Hasil akhir mencakup state terbaik pada populasi terakhir, state terbaik sepanjang pencarian, nilai objektif awal dan akhir, durasi, serta riwayat fitness maksimum dan rata-rata.

2.7. Multi-Truk

Fitur multi-truk memperluas permasalahan pengisian peti kemas sehingga program dapat mengelola lebih dari satu truk dalam satu state. Setiap truk memiliki dimensi dan kapasitas beratnya sendiri, sedangkan setiap penempatan paket menyimpan identitas truk tujuan melalui atribut truck_id. Tujuan pencarian adalah menentukan distribusi dan posisi seluruh paket pada armada sehingga nilai objektif total dari seluruh truk dapat dimaksimalkan tanpa melanggar constraint masing-masing peti kemas.

```
# 1. REPRESENTASI TRUCK DAN PROBLEM
# Problem menyimpan seluruh truk dan paket yang
digunakan.

class Truck:
    def __init__(self, truck_id, dimensions,
max_capacity):
        self.id = truck_id
        self.dimensions = dimensions
        self.max_capacity = max_capacity


class Problem:
    def __init__(self, trucks, packages):
        # Multi-truck direpresentasikan sebagai list
objek Truck.
        self.trucks = list(trucks)
        self.packages = list(packages)

        # Dictionary digunakan agar pencarian
berdasarkan ID lebih cepat.
        self.trucks_by_id = {
            truck.id: truck
            for truck in self.trucks
        }

        self.packages_by_id = {
            package.id: package
            for package in self.packages
        }

        if len(self.trucks_by_id) != len(self.trucks):
            raise ValueError("truck ids must be
```

```
unique")

        if len(self.packages_by_id) !=
len(self.packages):
            raise ValueError("package ids must be
unique")

    def get_truck(self, truck_id):
        # Mengambil objek truk berdasarkan ID.
        if truck_id not in self.trucks_by_id:
            raise ValueError(f"truck {truck_id!r} not
found")

        return self.trucks_by_id[truck_id]

    def get_package(self, package_id):
        # Mengambil objek paket berdasarkan ID.
        if package_id not in self.packages_by_id:
            raise ValueError(f"package {package_id!r}
not found")

        return self.packages_by_id[package_id]


# 2. REPRESENTASI PENEMPATAN PAKET DAN STATE ARMADA
# Setiap placement menyimpan truck_id tempat paket
berada.

class Placement:
    def __init__(
        self,
        package_id,
        position=None,
        orientation=Orientation.WLH,
        truck_id=None,
    ):
        self.package_id = package_id
        self.position = position
        self.orientation = orientation
        self.truck_id = truck_id


class State:
    def __init__(self, placements):
        # Satu state mencakup semua paket pada seluruh
truk.
        self.placements = list(placements)

    def get_inside_placements(self):
        # Mengambil seluruh paket yang berada di dalam
armada.
        return [
            placement
            for placement in self.placements
```

```
            if placement.truck_id is not None
            and placement.position is not None
        ]

    def get_outside_placements(self):
        # Paket di luar armada tidak memiliki truck_id
dan posisi.
        return [
            placement
            for placement in self.placements
            if placement.truck_id is None
            and placement.position is None
        ]

    def get_placements_in_truck(self, truck_id):
        # Mengambil paket yang berada pada satu truk
tertentu.
        return [
            placement
            for placement in self.placements
            if placement.truck_id == truck_id
            and placement.position is not None
        ]

    def get_placement(self, package_id):
        for placement in self.placements:
            if placement.package_id == package_id:
                return placement

        raise ValueError(
            f"package {package_id!r} not found in
state"
        )

    def copy(self):
        # Operasi neighbor menggunakan salinan state.
        return deepcopy(self)



# 3. MEMBUAT PROBLEM DENGAN JUMLAH TRUK YANG DITENTUKAN
# Fungsi ini dipanggil dari orchestration sebelum
algoritma.

def generate_problem(package_count=30, seed=None,
truck_count=1):
    if truck_count < 1:
        raise ValueError("truck_count must be at least
1")

    rng = random.Random(seed)

    # Membuat T001, T002, dan seterusnya.
    # Saat ini semua truk memiliki ukuran dan
```

```
kapasitas yang sama.
    trucks = [
        Truck(
            f"T{index + 1:03d}",
            Dimensions(10, 20, 5),
            500,
        )
        for index in range(truck_count)
    ]

    packages = []

    # Membuat data paket secara random berdasarkan
seed.
    for index in range(package_count):
        packages.append(
            Package(
                f"P{index + 1:03d}",
                Dimensions(
                    rng.randint(1, 4),
                    rng.randint(1, 4),
                    rng.randint(1, 3),
                ),
                rng.randint(10, 100),
                rng.randint(1, 20),
                rng.random() < 0.2,
                rng.randint(1, 7),
            )
        )

    return Problem(trucks, packages)


# 4. MEMBUAT INITIAL STATE UNTUK BANYAK TRUK
# Setiap paket dapat mencoba masuk ke salah satu truk
secara acak.

def generate_initial_state(
    problem,
    seed=None,
    placement_probability=0.7,
    max_attempts=100,
    *,
    rng=None,
):
    if not problem.trucks:
        raise ValueError("at least one truck is
required")

    if rng is None:
        rng = random.Random(seed)

    # Awalnya semua paket berada di luar armada.
    placements = [
```

```
        Placement(
            package.id,
            orientation=rng.choice(list(Orientation)),
        )
        for package in problem.packages
    ]

    state = State(placements)
    order = list(range(len(placements)))
    rng.shuffle(order)

    for index in order:
        placement = state.placements[index]

        if rng.random() > placement_probability:
            continue

        for _ in range(max_attempts):
            # Memilih salah satu truk dalam armada.
            truck = rng.choice(problem.trucks)

            placement.orientation = rng.choice(
                list(Orientation)
            )

            size = _oriented_dimensions(
                problem,
                placement,
            )

            if size.width > truck.dimensions.width:
                continue

            if size.length > truck.dimensions.length:
                continue

            if size.height > truck.dimensions.height:
                continue

            possible_z = [0]

            # Support hanya dicari pada truk yang
sama.
            for lower in
state.get_placements_in_truck(
                truck.id
            ):
                if lower.package_id ==
placement.package_id:
                    continue

                if problem.get_package(
                    lower.package_id
                ).is_fragile:
```

```
                    continue

                lower_size = _oriented_dimensions(
                    problem,
                    lower,
                )

                top = (
                    lower.position.z
                    + lower_size.height
                )

                if (
                    top + size.height
                    <= truck.dimensions.height
                ):
                    possible_z.append(top)

            # Menyimpan ID truk dan posisi paket.
            placement.truck_id = truck.id
            placement.position = Position(
                rng.randint(
                    0,
                    truck.dimensions.width -
size.width,
                ),
                rng.randint(
                    0,
                    truck.dimensions.length -
size.length,
                ),
                rng.choice(possible_z),
            )

            if is_valid_state(problem, state):
                break

            # Jika tidak valid, paket dikembalikan ke
luar.
            placement.truck_id = None
            placement.position = None

    return state



# 5. VALIDASI CONSTRAINT UNTUK SETIAP TRUK
# Berat, dimensi, overlap, support, dan fragile
diperiksa secara terpisah pada masing-masing truk.

def is_valid_state(problem, state):
    placement_ids = [
        placement.package_id
        for placement in state.placements
```

```
    ]

    # Memastikan tidak ada paket duplikat atau hilang.
    if len(placement_ids) != len(set(placement_ids)):
        return False

    if set(placement_ids) !=
set(problem.packages_by_id):
        return False

    for placement in state.placements:
        position = placement.position

        outside = (
            placement.truck_id is None
            and position is None
        )

        inside = (
            placement.truck_id is not None
            and position is not None
        )

        if not outside and not inside:
            return False

        if not isinstance(
            placement.orientation,
            Orientation,
        ):
            return False

        if inside:
            if placement.truck_id not in
problem.trucks_by_id:
                return False

            if not isinstance(position, Position):
                return False

            if not all(
                isinstance(value, int)
                for value in position.as_tuple()
            ):
                return False

    # Constraint diperiksa untuk setiap truk.
    for truck in problem.trucks:
        placements = state.get_placements_in_truck(
            truck.id
        )

        # Memeriksa kapasitas berat truk.
        loaded_weight = sum(
```

```
            problem.get_package(
                placement.package_id
            ).weight
            for placement in placements
        )

        if loaded_weight > truck.max_capacity:
            return False

        # Memeriksa batas dimensi peti kemas.
        for placement in placements:
            position = placement.position
            size = _oriented_dimensions(
                problem,
                placement,
            )

            if (
                position.x < 0
                or position.y < 0
                or position.z < 0
            ):
                return False

            if (
                position.x + size.width
                > truck.dimensions.width
            ):
                return False

            if (
                position.y + size.length
                > truck.dimensions.length
            ):
                return False

            if (
                position.z + size.height
                > truck.dimensions.height
            ):
                return False

        # Memeriksa overlap di dalam truk yang sama.
        for index, placement_a in
enumerate(placements):
            for placement_b in placements[index + 1:]:
                if _placements_overlap(
                    problem,
                    placement_a,
                    placement_b,
                ):
                    return False

        # Memeriksa support dan aturan fragile.
```

```
        for upper in placements:
            upper_position = upper.position

            # Paket pada z = 0 ditopang lantai truk.
            if upper_position.z == 0:
                continue

            has_support = False

            for lower in placements:
                if lower is upper:
                    continue

                lower_size = _oriented_dimensions(
                    problem,
                    lower,
                )

                lower_top = (
                    lower.position.z
                    + lower_size.height
                )

                if lower_top != upper_position.z:
                    continue

                if not _footprint_overlap(
                    problem,
                    lower,
                    upper,
                ):
                    continue

                # Paket fragile tidak boleh menjadi
support.
                if problem.get_package(
                    lower.package_id
                ).is_fragile:
                    return False

                has_support = True

            if not has_support:
                return False

    return True


# 6. OPERASI MOVE UNTUK MEMINDAHKAN PAKET ANTARTRUK
# truck_id tujuan dapat berbeda dari truck_id
sebelumnya.

def move(problem, state, package_id, truck_id,
position):
```

```
    if (truck_id is None) != (position is None):
        raise ValueError(
            "truck_id and position must both be set "
            "or both be None"
        )

    if truck_id is not None:
        # Memastikan truk tujuan tersedia.
        problem.get_truck(truck_id)

    neighbor = state.copy()
    placement = neighbor.get_placement(package_id)

    # Mengganti truk dan posisi paket.
    placement.truck_id = truck_id
    placement.position = position

    if state_signature(neighbor) ==
state_signature(state):
        return None

    if not is_valid_state(problem, neighbor):
        return None

    return neighbor


def _move_neighbor(problem, state, rng):
    placement = rng.choice(state.placements)

    # Sebagian move mengeluarkan paket dari armada.
    if placement.position is not None and rng.random()
< 0.2:
        return move(
            problem,
            state,
            placement.package_id,
            None,
            None,
        )

    # Memilih salah satu truk sebagai tujuan.
    # Truk tujuan dapat sama atau berbeda dari truk
asal.
    truck = rng.choice(problem.trucks)

    size = _oriented_dimensions(
        problem,
        placement,
    )

    if size.width > truck.dimensions.width:
        return None
```

```
    if size.length > truck.dimensions.length:
        return None

    if size.height > truck.dimensions.height:
        return None

    possible_z = [0]

    # Mencari posisi support di truk tujuan.
    for lower in state.get_placements_in_truck(
        truck.id
    ):
        if lower.package_id == placement.package_id:
            continue

        if problem.get_package(
            lower.package_id
        ).is_fragile:
            continue

        lower_size = _oriented_dimensions(
            problem,
            lower,
        )

        top = lower.position.z + lower_size.height

        if (
            top + size.height
            <= truck.dimensions.height
        ):
            possible_z.append(top)

    return move(
        problem,
        state,
        placement.package_id,
        truck.id,
        Position(
            rng.randint(
                0,
                truck.dimensions.width - size.width,
            ),
            rng.randint(
                0,
                truck.dimensions.length - size.length,
            ),
            rng.choice(possible_z),
        ),
    )
```

Truk disimpan dalam sebuah list pada objek Problem. Jumlah truk ditentukan saat eksperimen dibuat. Setiap truk memiliki ID unik, seperti T001 dan T002, agar penempatan paket dapat dikaitkan dengan truk yang tepat.

Validasi state dilakukan untuk setiap truk secara terpisah. Program mengelompokkan penempatan paket berdasarkan truck_id, lalu memeriksa apakah paket berada dalam batas dimensi, tidak saling bertumpuk, tidak melayang, mengikuti aturan fragile, dan tidak melebihi kapasitas berat truk.

Pembuatan initial state mempertimbangkan seluruh truk yang tersedia. Untuk setiap paket, program dapat mencoba menempatkannya pada salah satu truk yang masih memiliki ruang dan kapasitas. Paket yang tidak berhasil ditempatkan tetap direpresentasikan sebagai paket di luar armada sehingga tidak ada paket yang hilang dari state.

Operasi neighbor juga mendukung perpindahan antartruk yang tersedia. Operasi move dapat memindahkan paket ke posisi lain pada truk yang sama, ke truk berbeda, atau ke luar armada. Operasi swap dapat menukar penempatan dua paket meskipun keduanya berada pada truk yang berbeda atau salah satunya berada di luar. Operasi rotate mengubah orientasi paket tanpa menghilangkan informasi truk tempat paket tersebut berada. Setiap operasi hanya menghasilkan neighbor apabila state hasilnya tetap valid.

Fungsi objektif menghitung kontribusi seluruh paket yang berada di dalam semua truk. Dengan demikian, algoritma tidak mengoptimalkan satu truk secara terpisah, melainkan mencari konfigurasi terbaik untuk keseluruhan armada dalam satu proses pencarian.

Hasil pencarian menyimpan satu state yang mencakup seluruh truk. Pada visualisasi 3D, pengguna dapat memilih truk melalui dropdown container untuk melihat isi setiap peti kemas secara terpisah. Meskipun visualisasi hanya menampilkan satu truk pada satu waktu, nilai objektif dan state hasil tetap merepresentasikan keseluruhan armada nantinya.

3. Eksperimen dan Analisis

3.1. Eksperimen

3.1.1. Hill Climbing

3.1.2. Hill Climbing - Steepest Ascent

3.1.2.1. Parameter Input

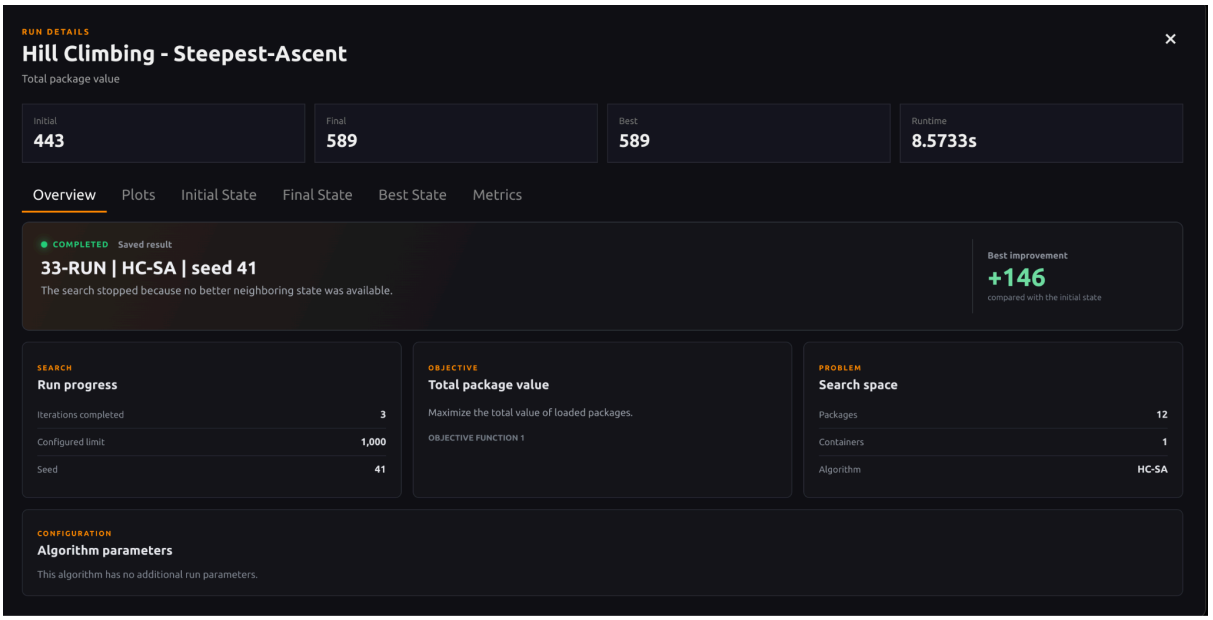
Berikut adalah parameter input yang digunakan saat menjalankan eksperimen :

Test case	Objective function	Package count	Truck count	Maximum iterations	Seed
1	Total package value	12	1	1000	41
2	Total package value	12	1	1000	42
3	Total package value	12	1	1000	43

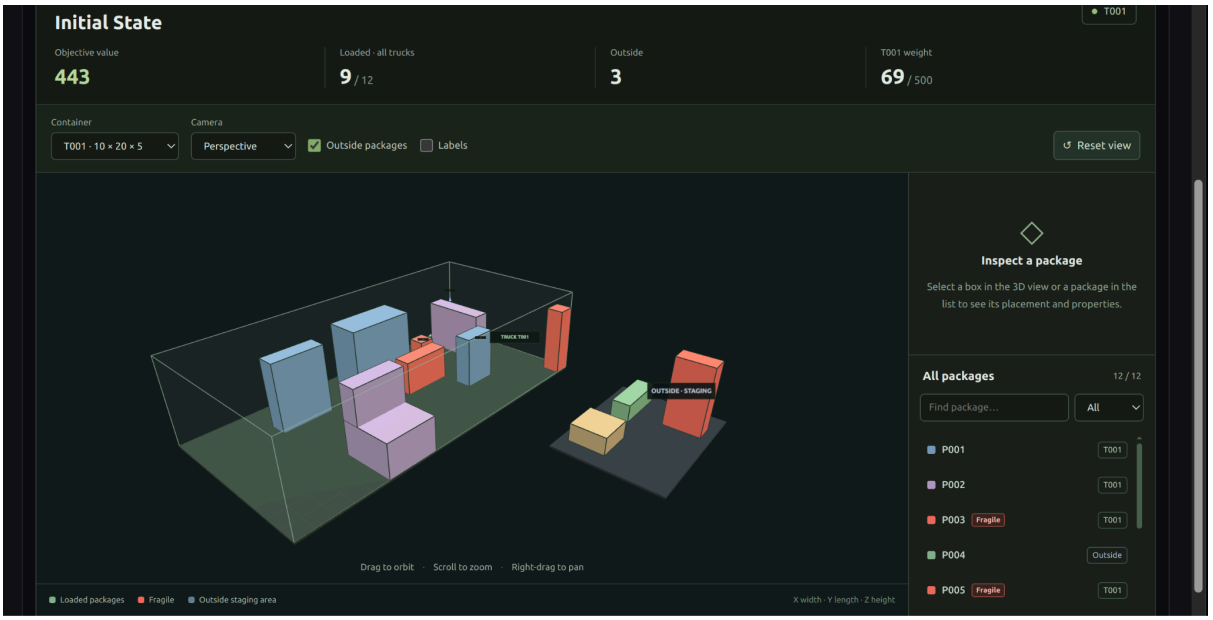
3.1.2.2. Hasil Eksperimen

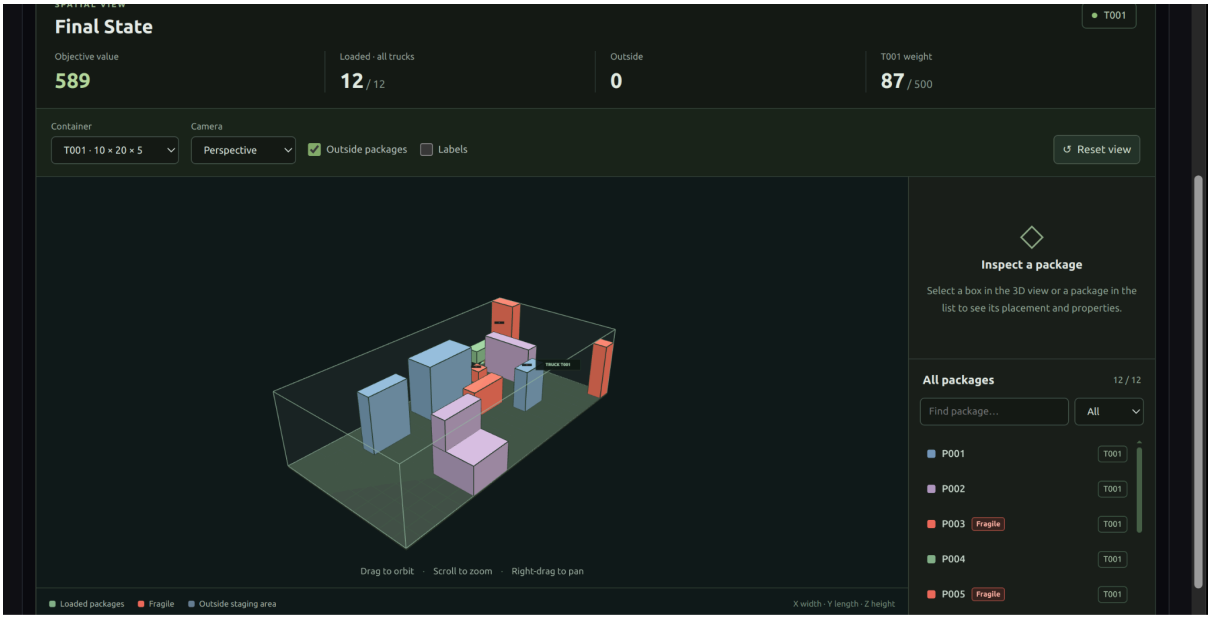
Berikut adalah hasil eksperimen untuk setiap test case :

Test Case 1 :

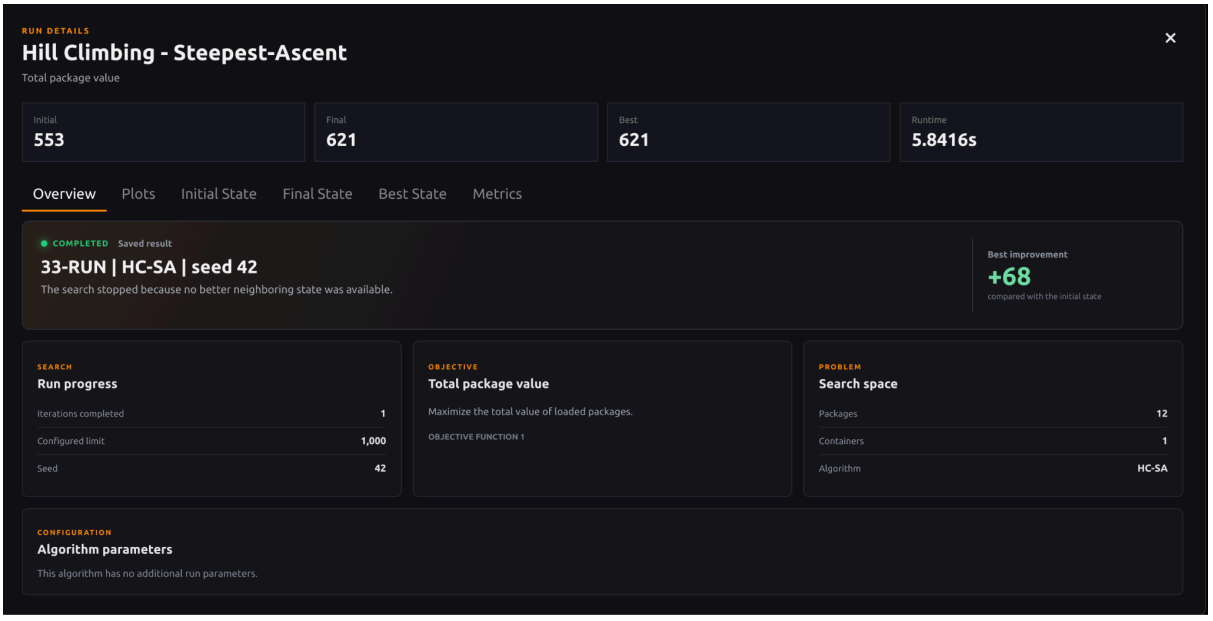


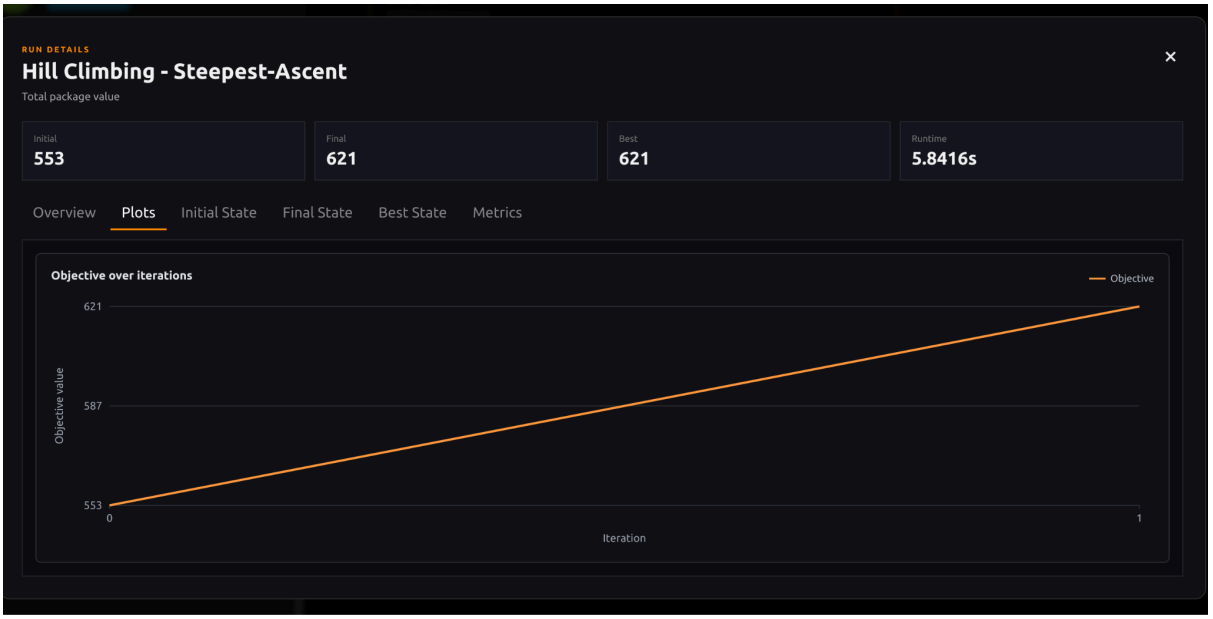
RUN DETAILS
Hill Climbing - Steepest-Ascent
Total package value
Initial
443
Final
589
Best
589
Runtime
8.5733s
Overview
Plots
Initial State
Final State
Best State
Metrics
Objective over iterations
Objective
589
516
443
Objective value
0
1
3
Iteration

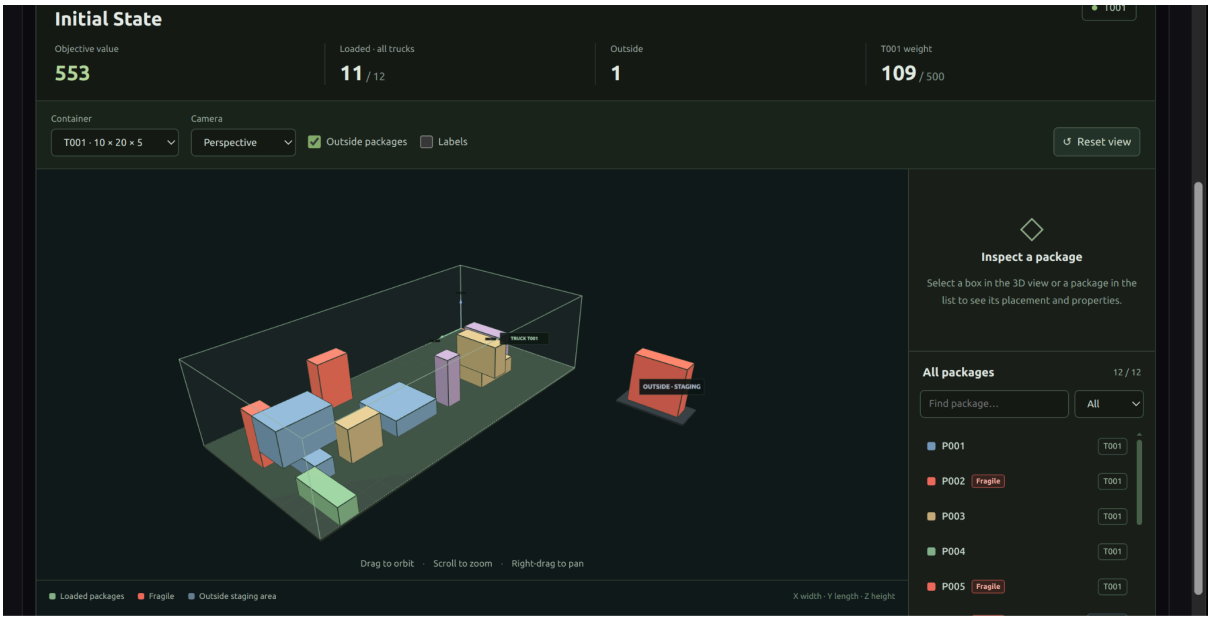

Initial State
T001
Objective value
443
Loaded · all trucks
9 / 12
Outside
3
T001 weight
69 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
T001
P003
Fragile
T001
P004
Outside
P005
Fragile
T001

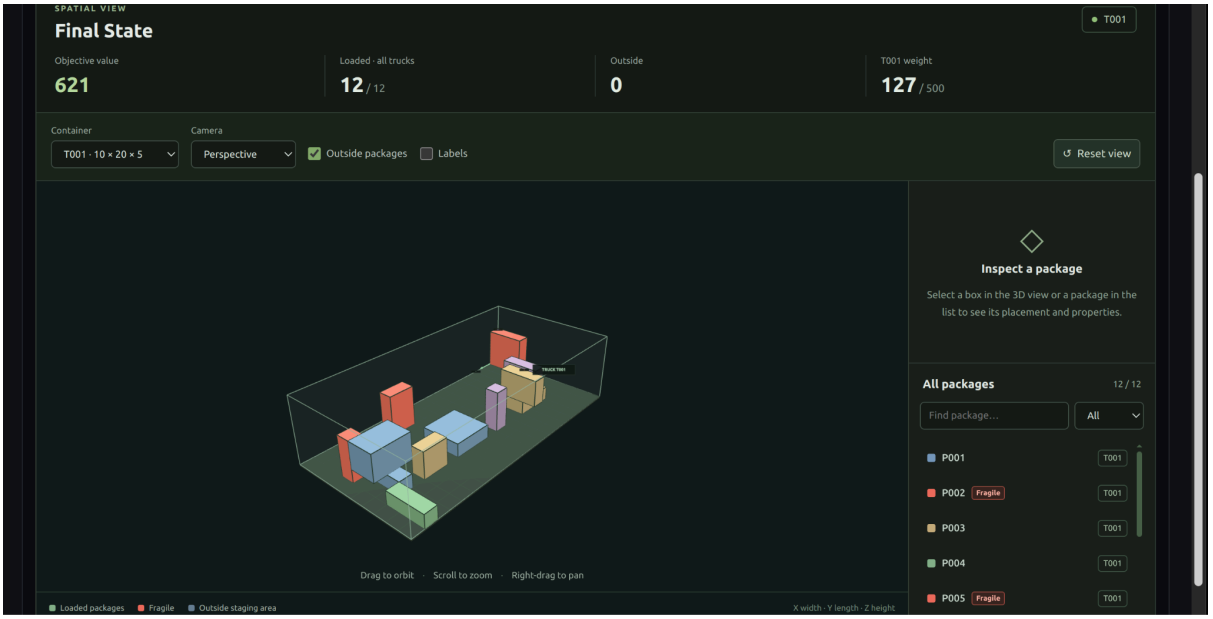

Final State
T001
Objective value
589
Loaded · all trucks
12 / 12
Outside
0
T001 weight
87 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
T001
P003
Fragile
T001
P004
T001
P005
Fragile
T001
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

Test Case 2 :

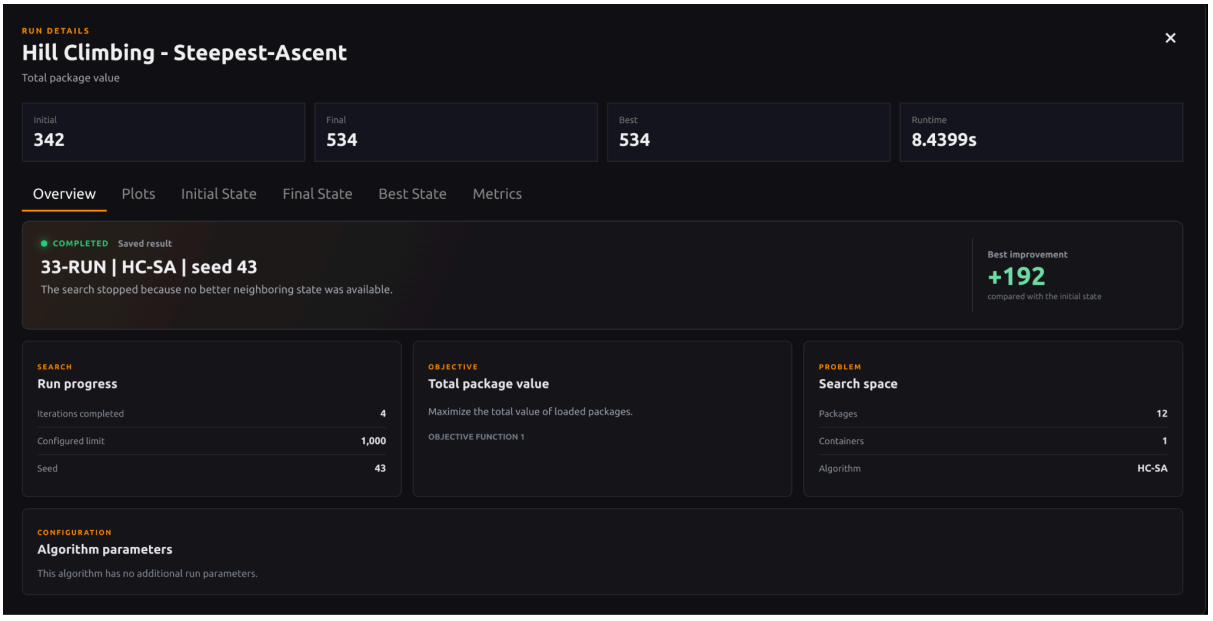

RUN DETAILS
Hill Climbing - Steepest-Ascent
Total package value
Initial
553
Final
621
Best
621
Runtime
5.8416s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | HC-SA | seed 42
The search stopped because no better neighboring state was available.
Best improvement
+68
compared with the initial state
SEARCH
Run progress
Iterations completed
1
Configured limit
1,000
Seed
42
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
HC-SA
CONFIGURATION
Algorithm parameters
This algorithm has no additional run parameters.

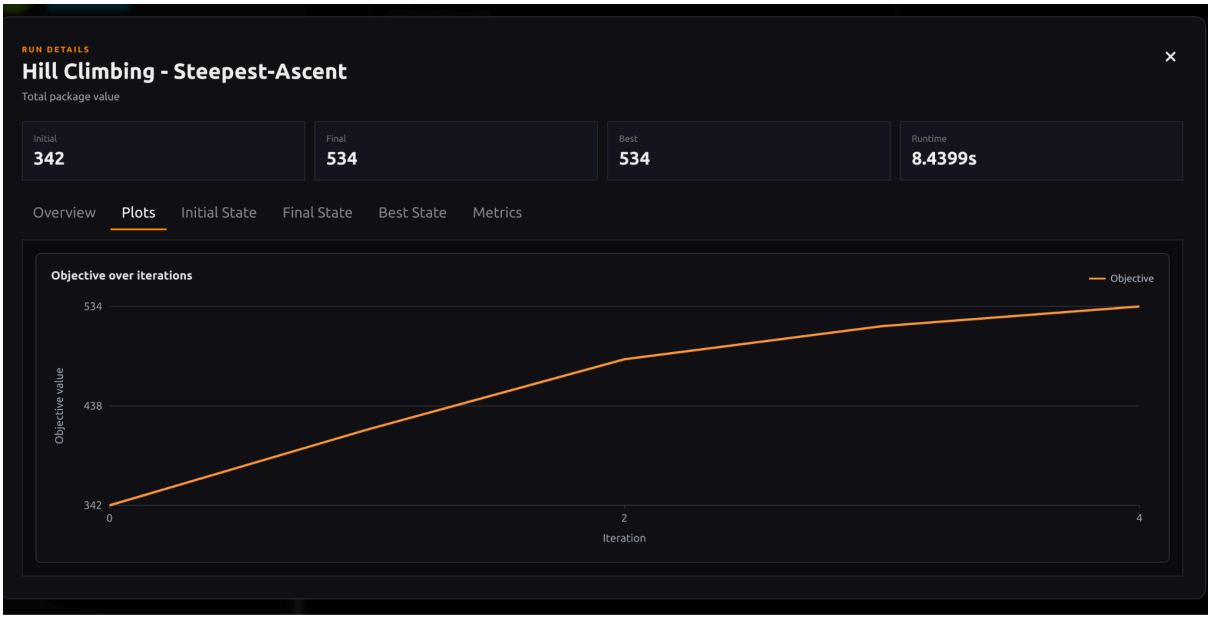

RUN DETAILS
Hill Climbing - Steepest-Ascent
Total package value
Initial
553
Final
621
Best
621
Runtime
5.8416s
Overview
Plots
Initial State
Final State
Best State
Metrics
Objective over iterations
Objective
621
587
553
Objective value
0
1
Iteration

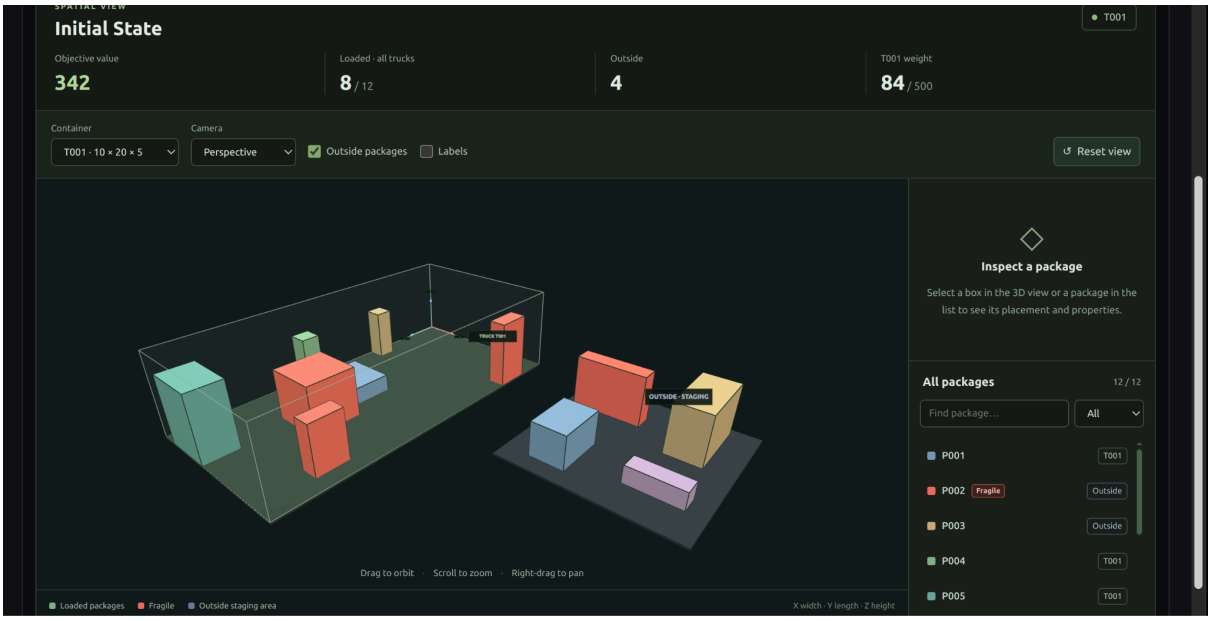

Initial State
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK TIRE
OUTSIDE - STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package...
All
P001
T001
P002
Fragile
T001
P003
T001
P004
T001
P005
Fragile
T001

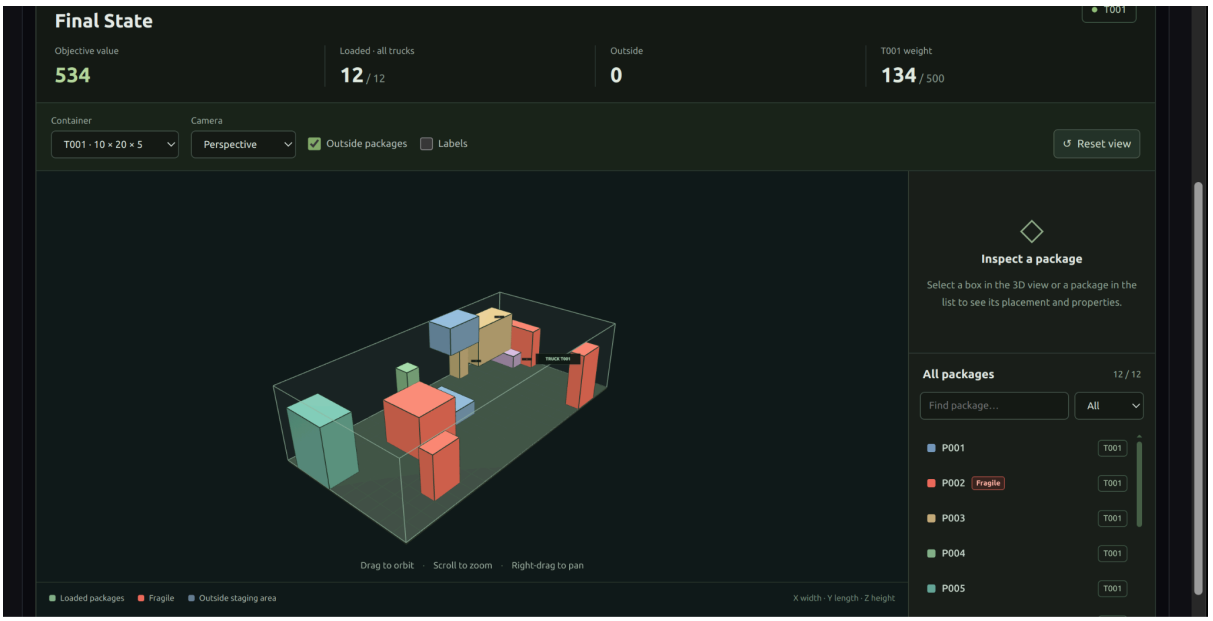

SPATIAL VIEW
Final State
T001
Objective value
621
Loaded · all trucks
12 / 12
Outside
0
T001 weight
127 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
T001
P003
T001
P004
T001
P005
Fragile
T001
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

Test Case 3 :


RUN DETAILS
Hill Climbing - Steepest-Ascent
Total package value
Initial
342
Final
534
Best
534
Runtime
8.4399s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | HC-SA | seed 43
The search stopped because no better neighboring state was available.
Best improvement
+192
compared with the initial state
SEARCH
Run progress
Iterations completed
4
Configured limit
1,000
Seed
43
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
HC-SA
CONFIGURATION
Algorithm parameters
This algorithm has no additional run parameters.


RUN DETAILS
Hill Climbing - Steepest-Ascent
Total package value
Initial
342
Final
534
Best
534
Runtime
8.4399s
Overview
Plots
Initial State
Final State
Best State
Metrics
Objective over iterations
Objective
534
438
342
Objective value
0
2
4
Iteration


Initial State
T001
Objective value
342
Loaded · all trucks
8 / 12
Outside
4
T001 weight
84 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
Outside
P003
Outside
P004
T001
P005
T001



3.1.3. Hill Climbing - Stochastic

3.1.3.1. Parameter Input

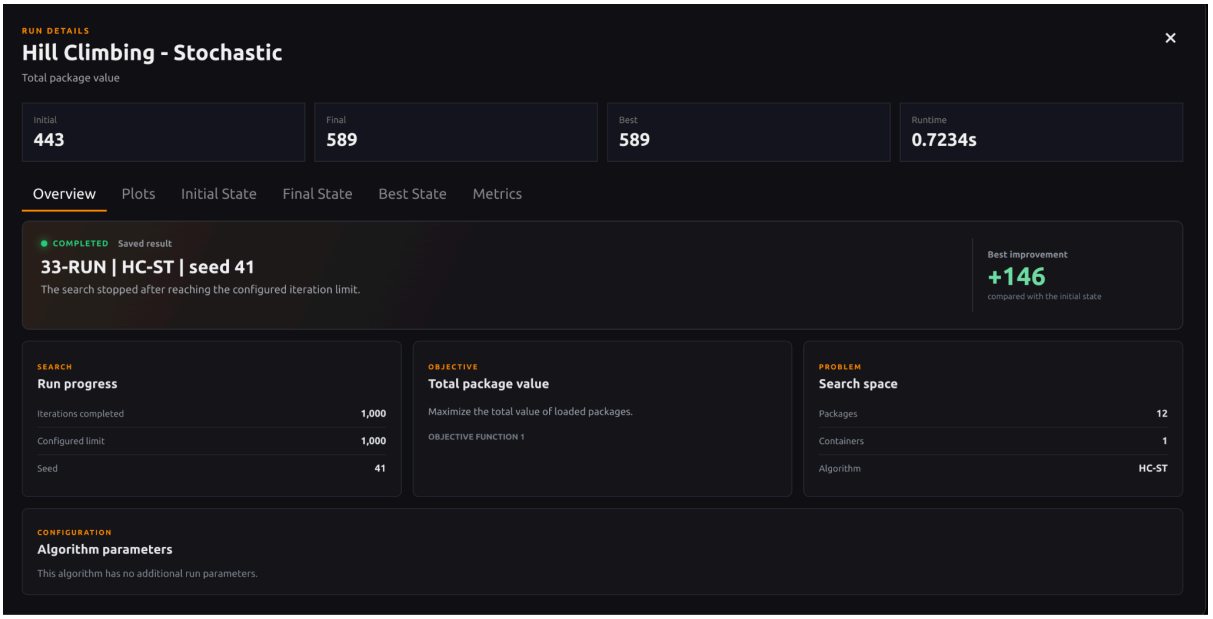
Berikut adalah parameter input yang digunakan saat menjalankan eksperimen :

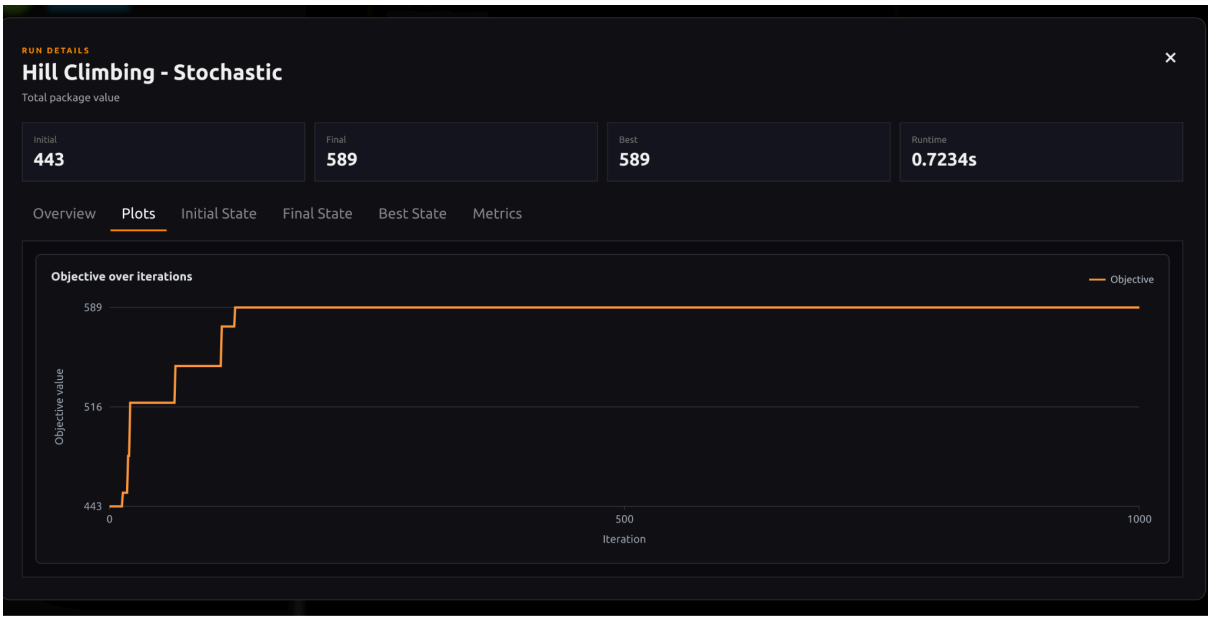
Test case	Objective function	Package count	Truck count	Maximum iterations	Seed
1	Total package value	12	1	1000	41
2	Total package value	12	1	1000	42
3	Total package value	12	1	1000	43

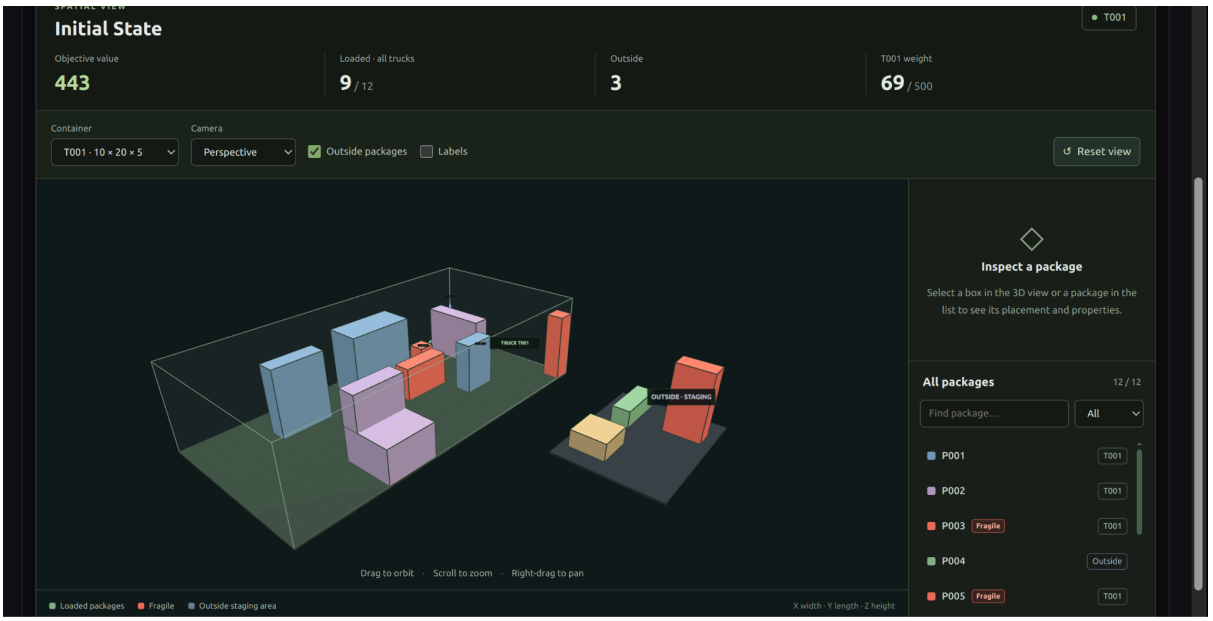
3.1.3.2. Hasil Eksperimen

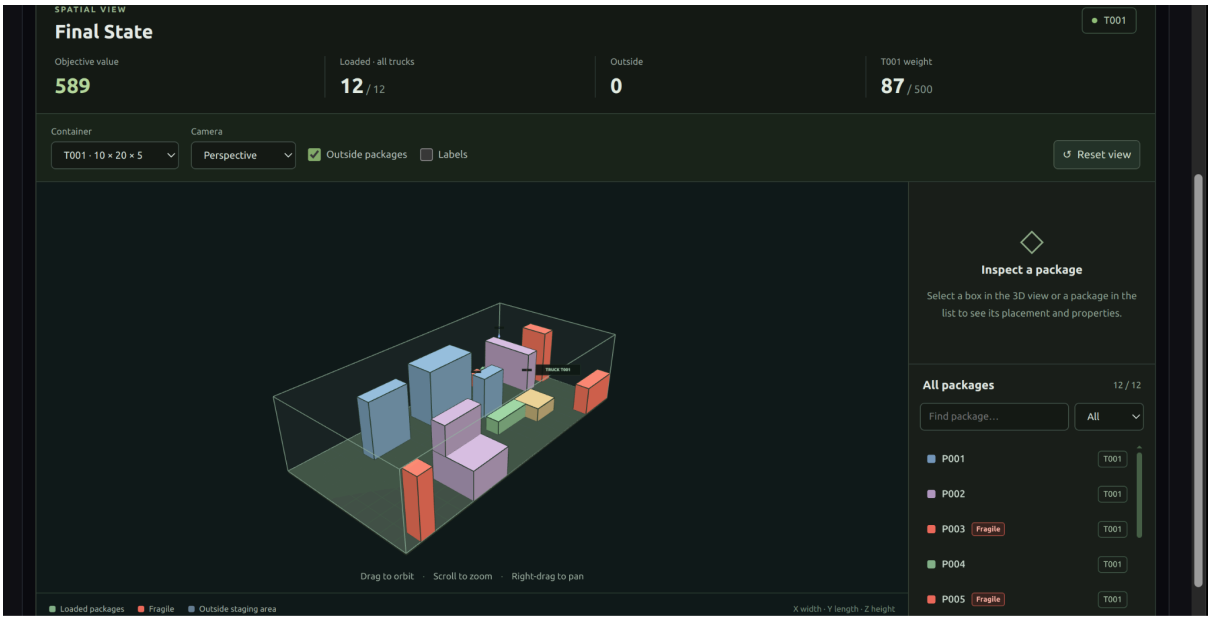
Berikut adalah hasil eksperimen untuk setiap test case :

Test Case 1 :

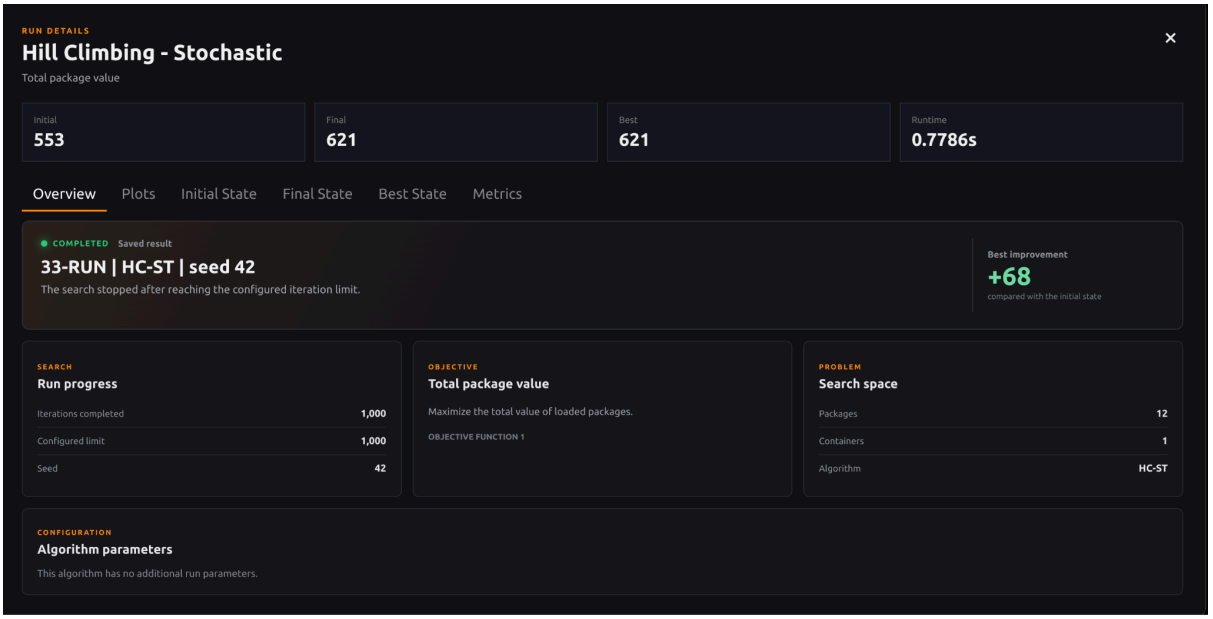

RUN DETAILS
Hill Climbing - Stochastic
Total package value
Initial
443
Final
589
Best
589
Runtime
0.7234s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | HC-ST | seed 41
The search stopped after reaching the configured iteration limit.
Best improvement
+146
compared with the initial state
SEARCH
Run progress
Iterations completed
1,000
Configured limit
1,000
Seed
41
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
HC-ST
CONFIGURATION
Algorithm parameters
This algorithm has no additional run parameters.

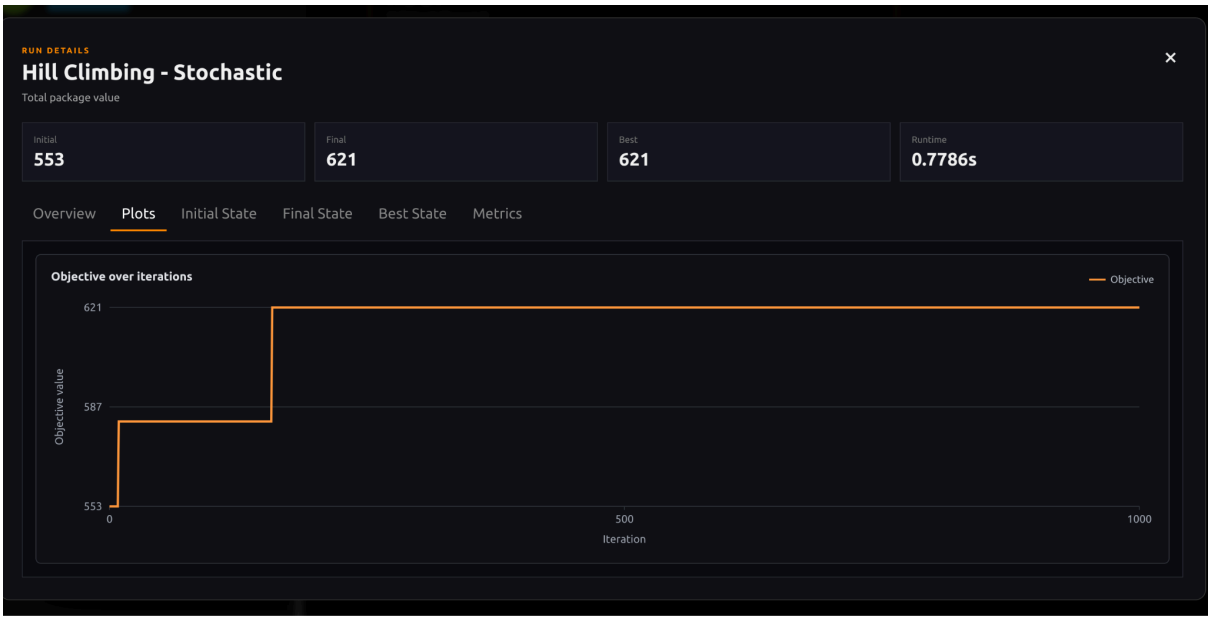

RUN DETAILS
Hill Climbing - Stochastic
Total package value
Initial
443
Final
589
Best
589
Runtime
0.7234s
Overview
Plots
Initial State
Final State
Best State
Metrics
Objective over iterations
Objective
589
516
443
Objective value
0
500
1000
Iteration

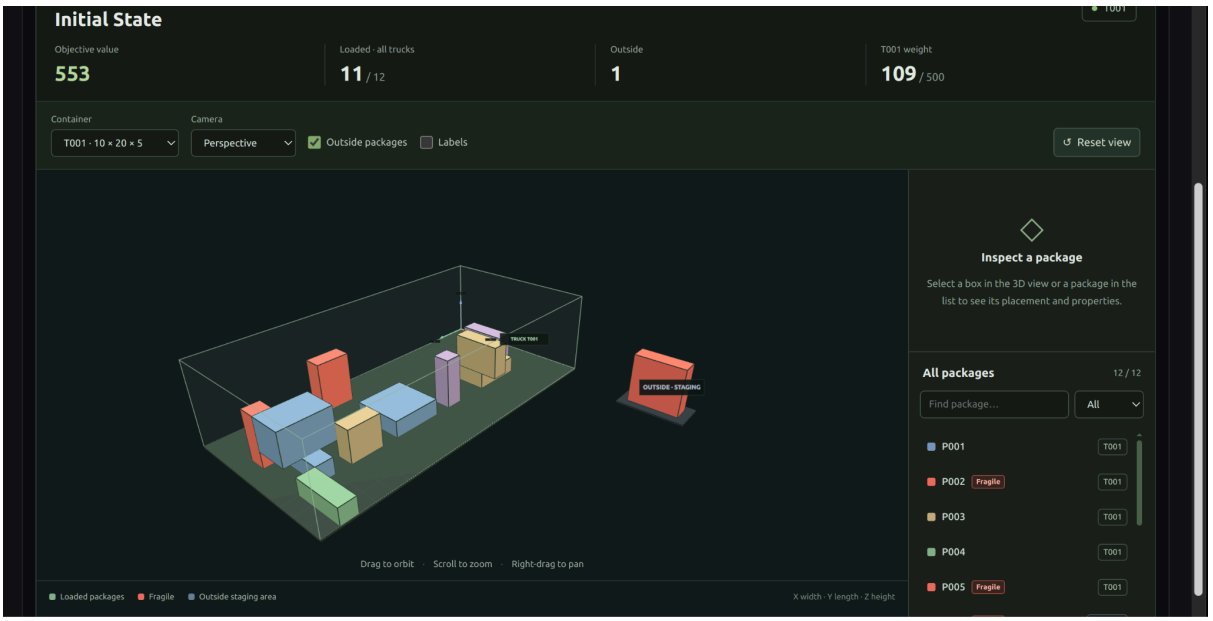

Initial State
T001
Objective value
443
Loaded · all trucks
9 / 12
Outside
3
T001 weight
69 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 T001
P003 Fragile T001
P004 Outside
P005 Fragile T001

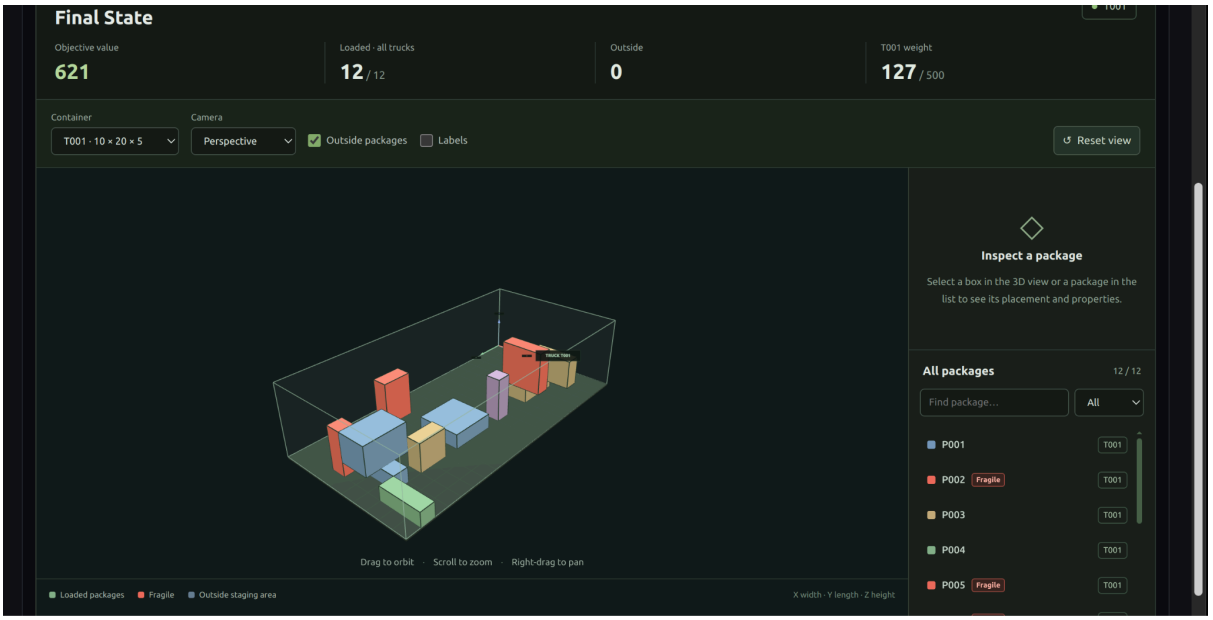

SPATIAL VIEW
Final State
T001
Objective value
589
Loaded · all trucks
12 / 12
Outside
0
T001 weight
87 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 T001
P003 Fragile T001
P004 T001
P005 Fragile T001

Test Case 2 :

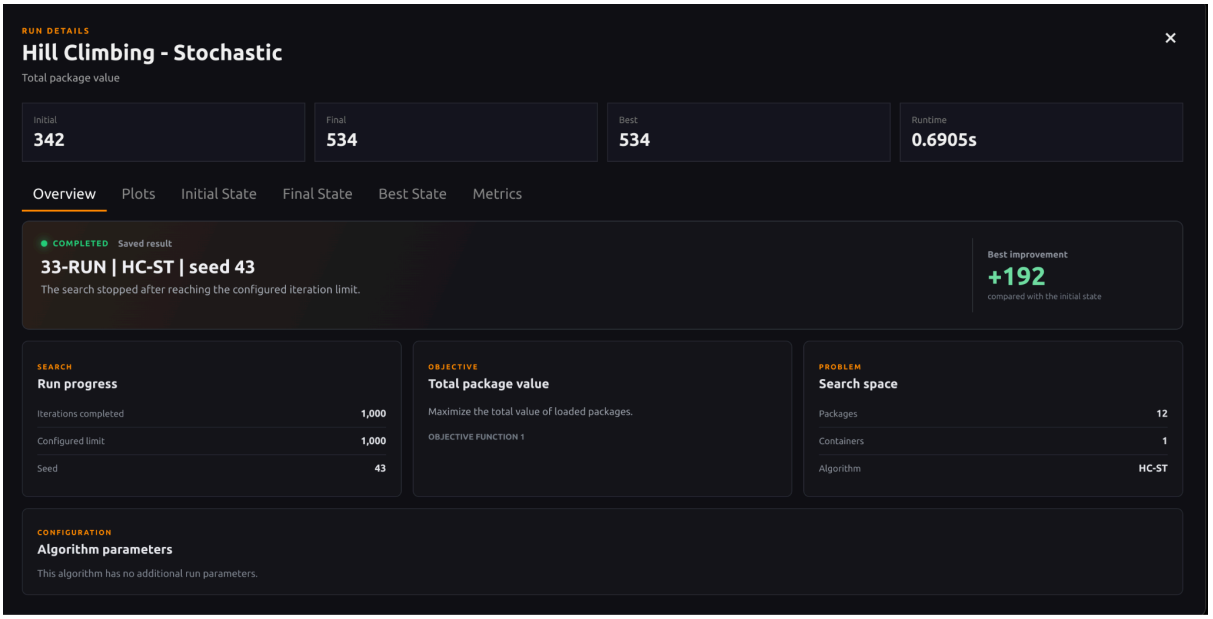

RUN DETAILS
Hill Climbing - Stochastic
Total package value
Initial
553
Final
621
Best
621
Runtime
0.7786s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | HC-ST | seed 42
The search stopped after reaching the configured iteration limit.
Best improvement
+68
compared with the initial state
SEARCH
Run progress
Iterations completed
1,000
Configured limit
1,000
Seed
42
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
HC-ST
CONFIGURATION
Algorithm parameters
This algorithm has no additional run parameters.


RUN DETAILS
Hill Climbing - Stochastic
Total package value
Initial
553
Final
621
Best
621
Runtime
0.7786s
Overview
Plots
Initial State
Final State
Best State
Metrics
Objective over iterations
Objective
621
587
553
Objective value
0
500
1000
Iteration

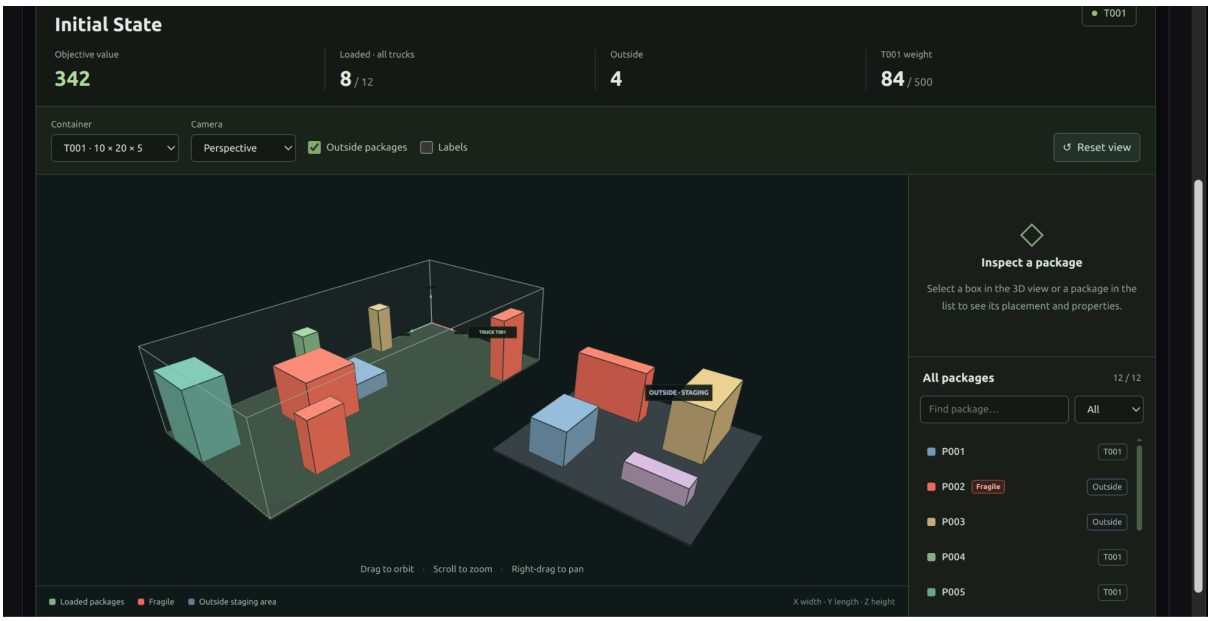

Initial State
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
OUTSIDE · STAGING
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Drag to orbit
Scroll to zoom
Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

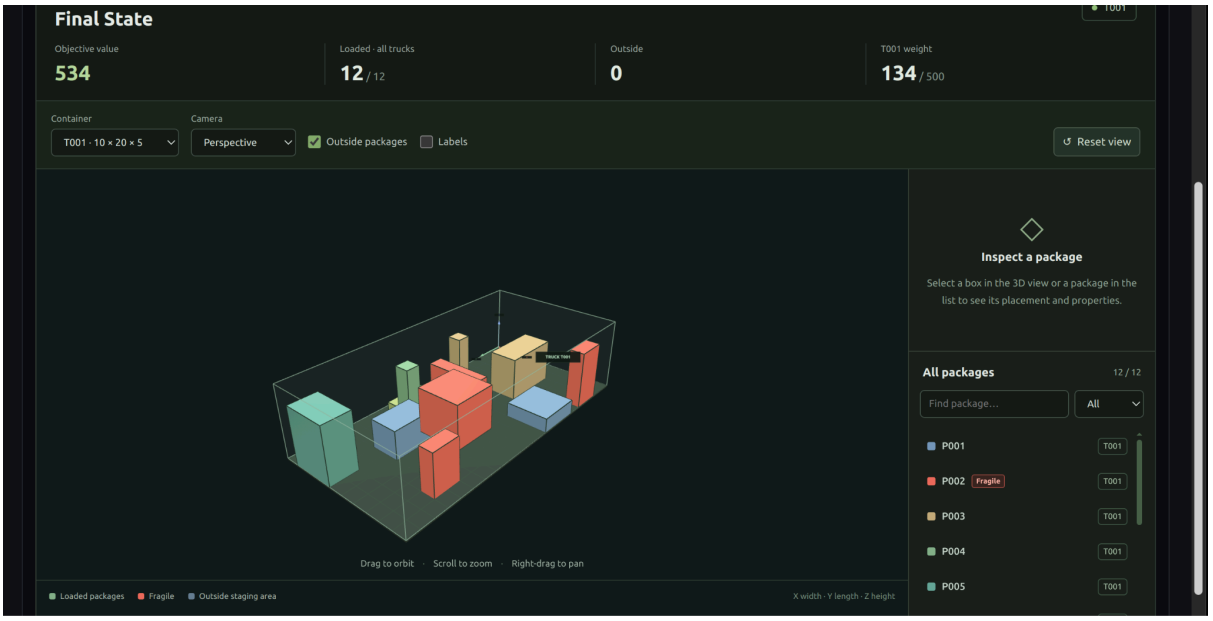

Final State
Objective value
621
Loaded · all trucks
12 / 12
Outside
0
T001 weight
127 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Drag to orbit
Scroll to zoom
Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

Test Case 3 :


RUN DETAILS
Hill Climbing - Stochastic
Total package value
Initial
342
Final
534
Best
534
Runtime
0.6905s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | HC-ST | seed 43
The search stopped after reaching the configured iteration limit.
Best improvement
+192
compared with the initial state
SEARCH
Run progress
Iterations completed
1,000
Configured limit
1,000
Seed
43
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
HC-ST
CONFIGURATION
Algorithm parameters
This algorithm has no additional run parameters.

RUN DETAILS
Hill Climbing - Stochastic
Total package value
Initial
342
Final
534
Best
534
Runtime
0.6905s
Overview
Plots
Initial State
Final State
Best State
Metrics
Objective over iterations
Objective
534
438
342
Objective value
0
500
1000
Iteration





3.1.4. Hill Climbing - Sideways Move

3.1.4.1. Parameter Input

Berikut adalah parameter input yang digunakan saat menjalankan eksperimen :

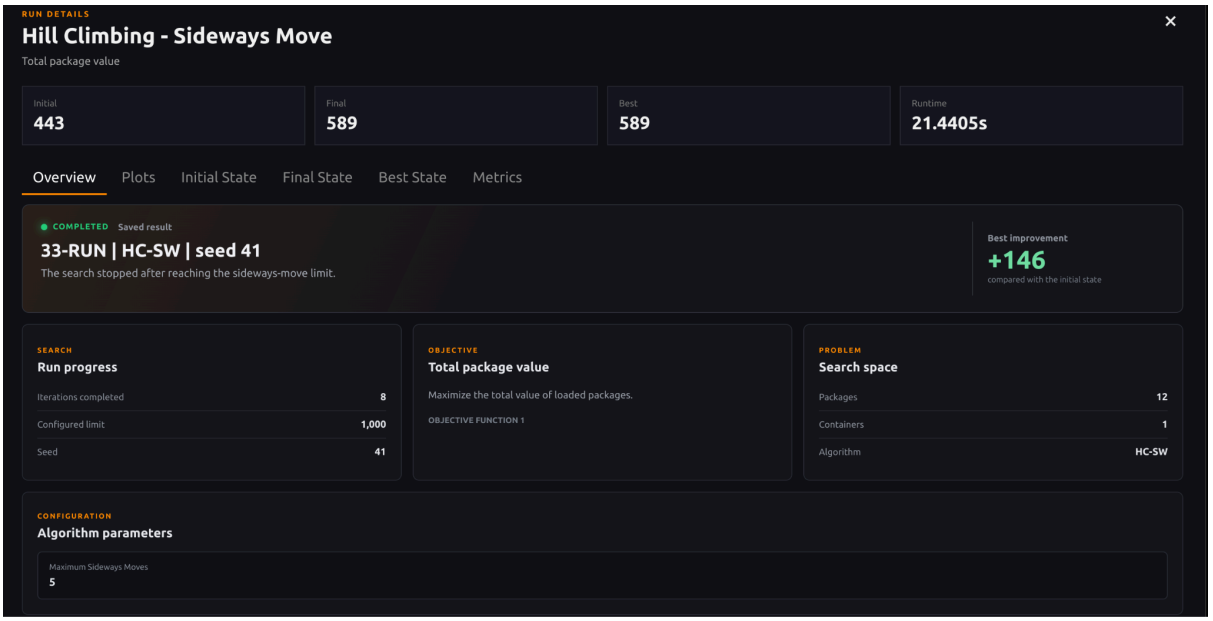
Test case	Objective function	Package count	Truck count	Maximum iterations	Seed	Maximum sideways
1	Total package value	12	1	1000	41	5

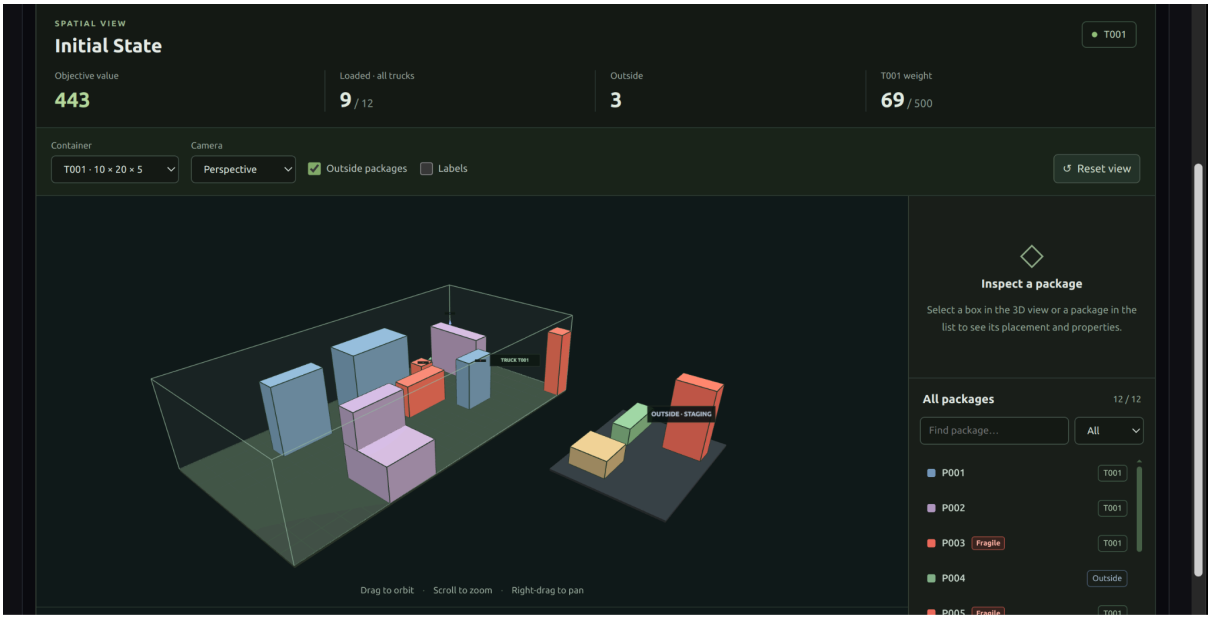
2	Total package value	12	1	1000	42	5
3	Total package value	12	1	1000	43	5

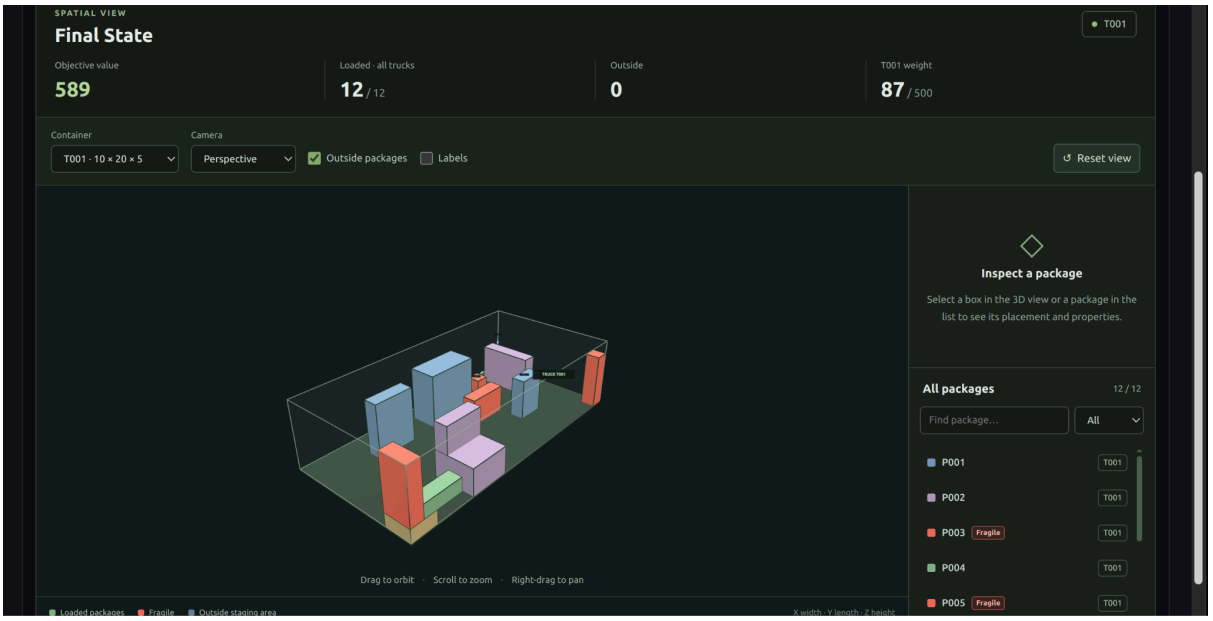
3.1.4.2. Hasil Eksperimen

Berikut adalah hasil eksperimen untuk setiap test case :

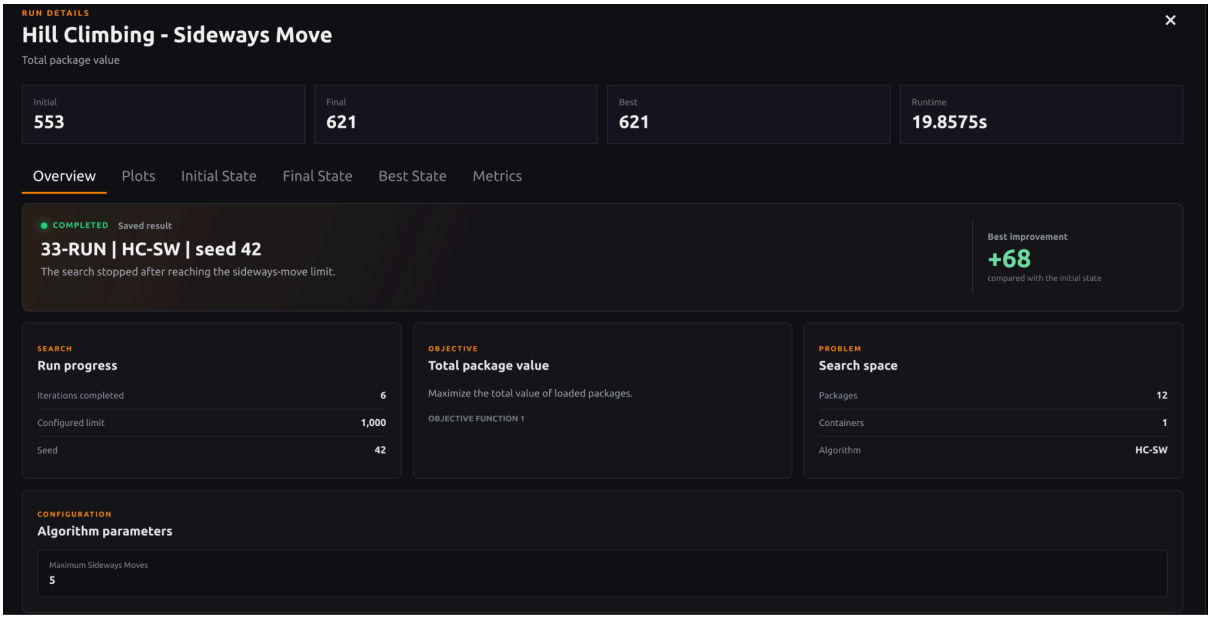
Test Case 1 :

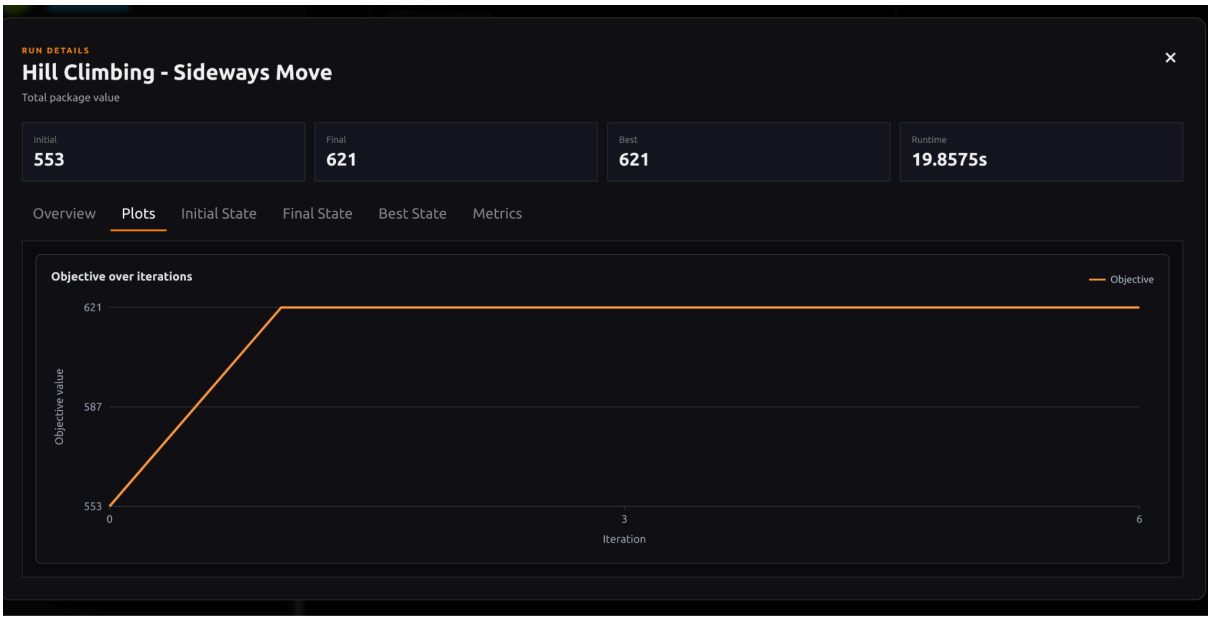


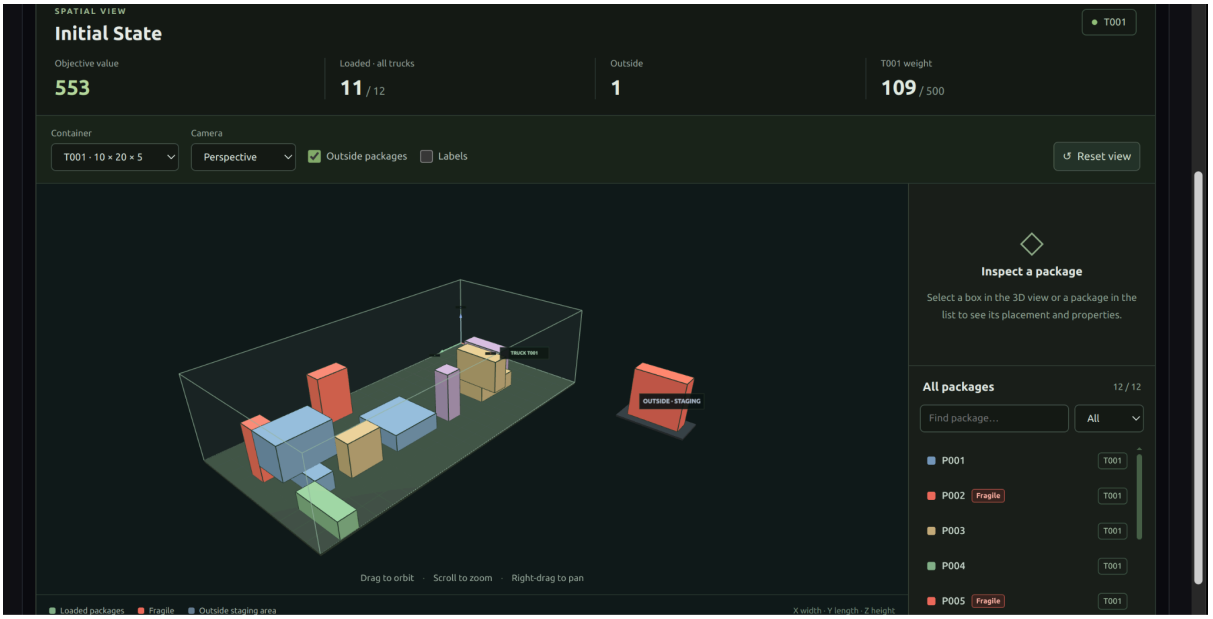

SPATIAL VIEW
Initial State
T001
Objective value
443
Loaded · all trucks
9 / 12
Outside
3
T001 weight
69 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 T001
P003 Fragile T001
P004 Outside

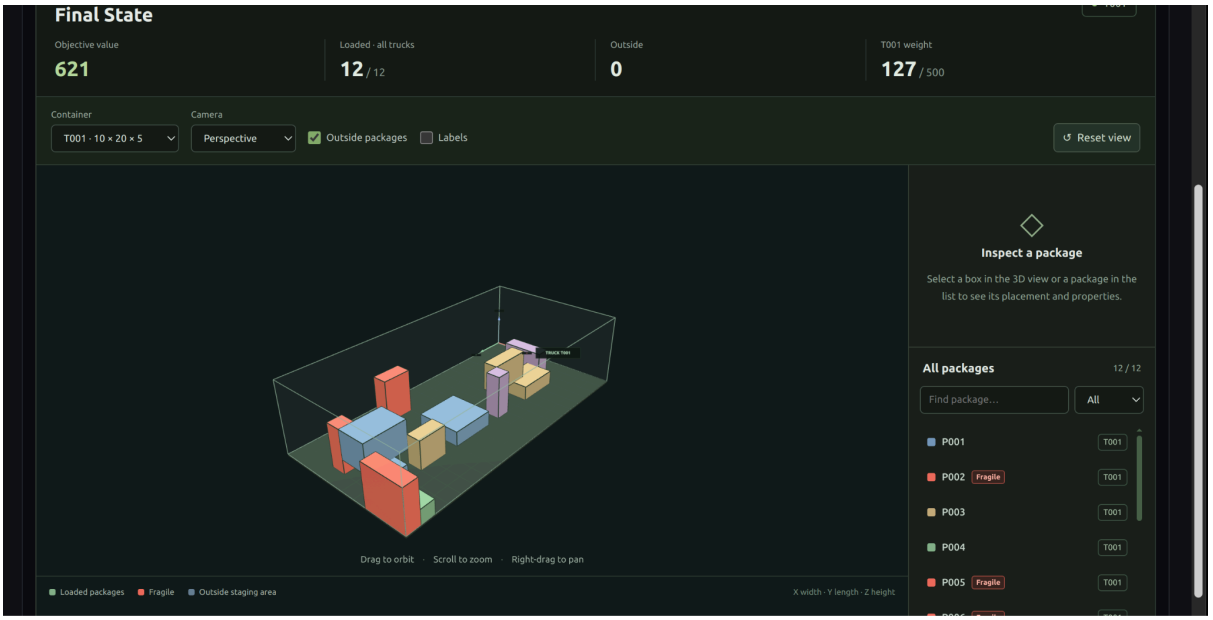

SPATIAL VIEW
Final State
T001
Objective value
589
Loaded · all trucks
12 / 12
Outside
0
T001 weight
87 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
Drag to orbit · Scroll to zoom · Right-drag to pan
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 T001
P003 Fragile T001
P004 T001
P005 Fragile T001

Test Case 2 :

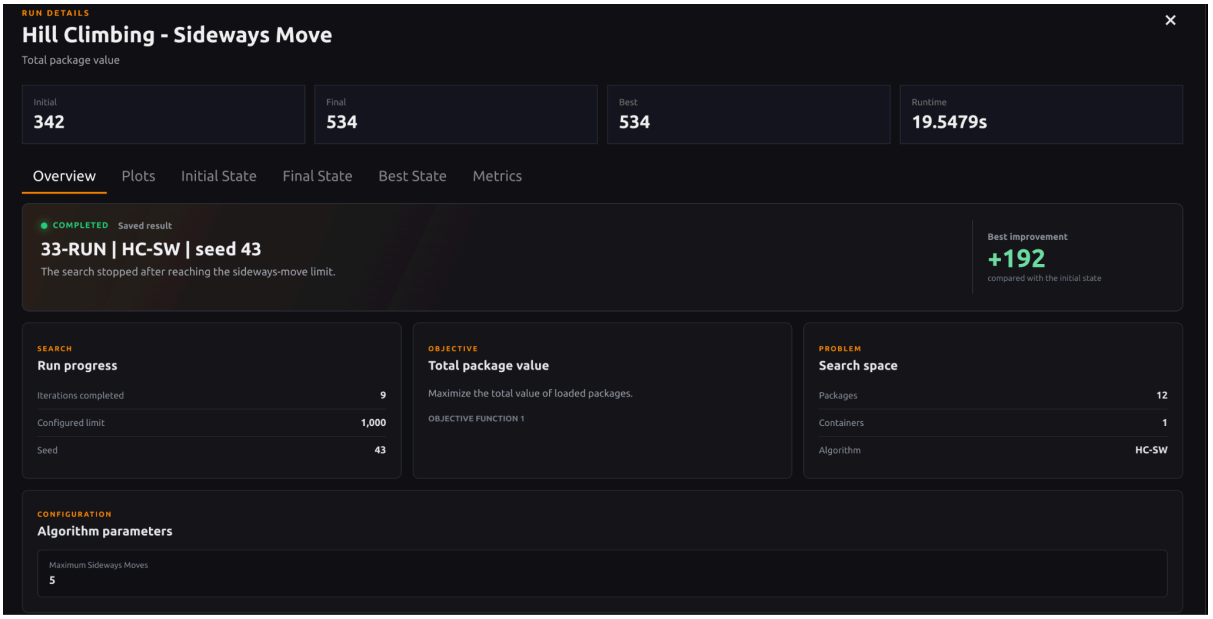

RUN DETAILS
Hill Climbing - Sideways Move
Total package value
Initial
553
Final
621
Best
621
Runtime
19.8575s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | HC-SW | seed 42
The search stopped after reaching the sideways-move limit.
Best improvement
+68
compared with the initial state
SEARCH
Run progress
Iterations completed
6
Configured limit
1,000
Seed
42
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
HC-SW
CONFIGURATION
Algorithm parameters
Maximum Sideways Moves
5


RUN DETAILS
Hill Climbing - Sideways Move
Total package value
Initial
553
Final
621
Best
621
Runtime
19.8575s
Overview
Plots
Initial State
Final State
Best State
Metrics
Objective over iterations
Objective
621
587
553
Objective value
0
3
6
Iteration

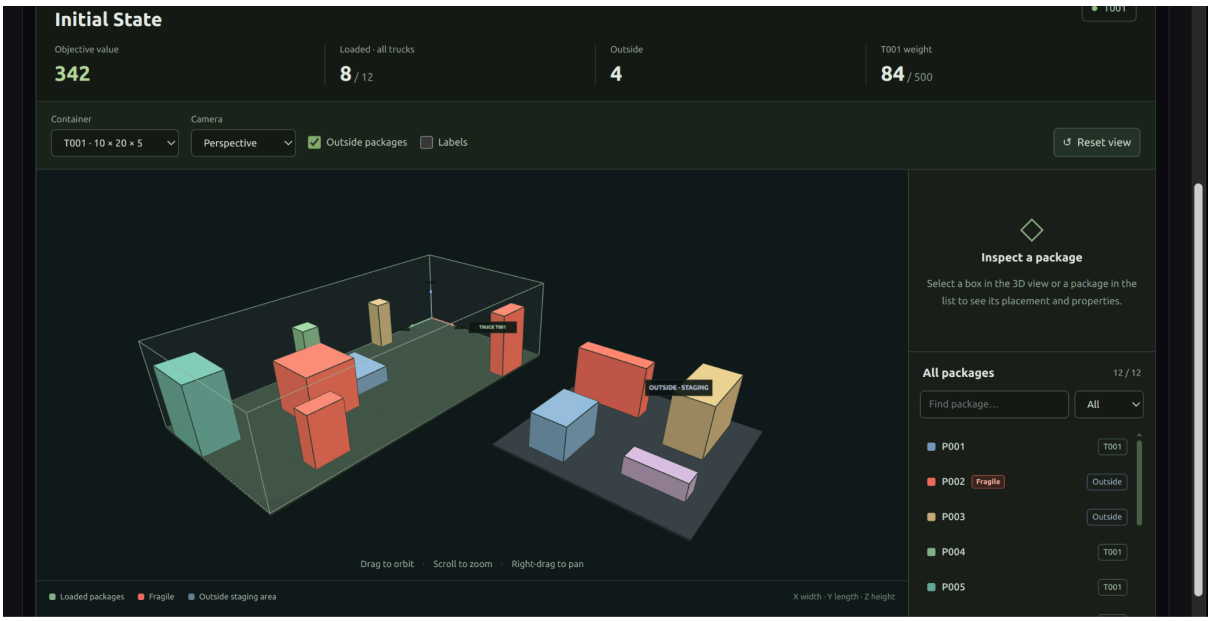

SPATIAL VIEW
Initial State
T001
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
T001
P003
T001
P004
T001
P005
Fragile
T001

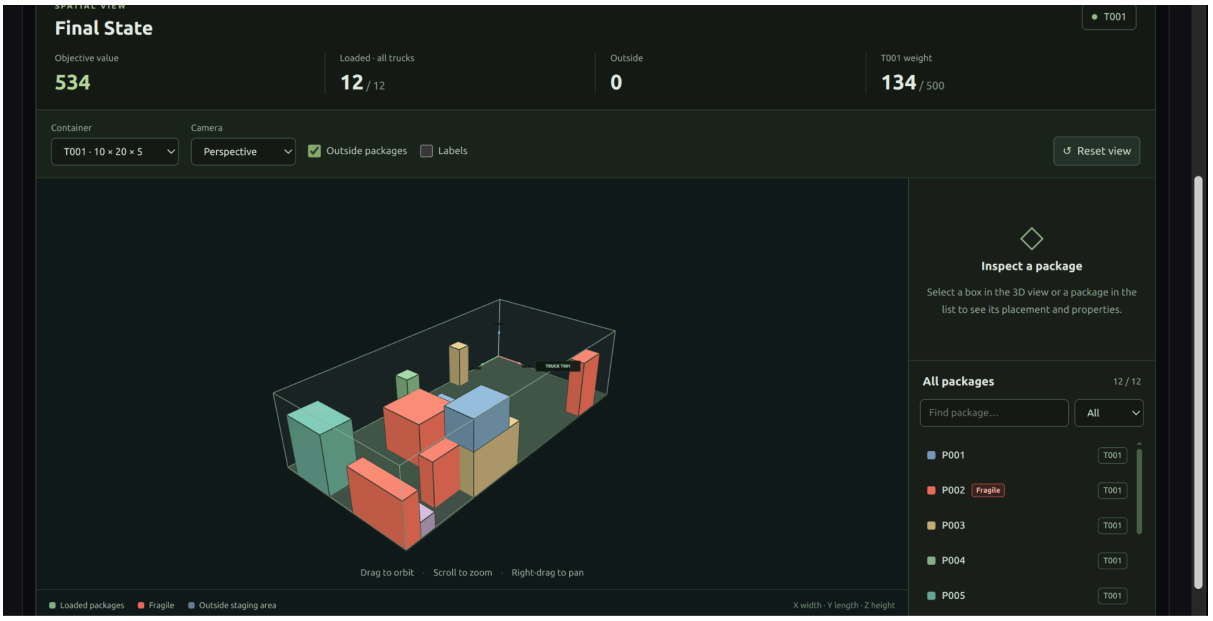

Final State
Objective value
621
Loaded · all trucks
12 / 12
Outside
0
T001 weight
127 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
T001
P003
T001
P004
T001
P005
Fragile
T001

Test Case 3 :


RUN DETAILS
Hill Climbing - Sideways Move
Total package value
Initial
342
Final
534
Best
534
Runtime
19.5479s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | HC-SW | seed 43
The search stopped after reaching the sideways-move limit.
Best improvement
+192
compared with the initial state
SEARCH
Run progress
Iterations completed
9
Configured limit
1,000
Seed
43
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
HC-SW
CONFIGURATION
Algorithm parameters
Maximum Sideways Moves
5

RUN DETAILS
Hill Climbing - Sideways Move
Total package value
Initial
342
Final
534
Best
534
Runtime
19.5479s
Overview
Plots
Initial State
Final State
Best State
Metrics
Objective over iterations
Objective
534
438
342
Objective value
0
4
9
Iteration





3.1.5. Hill Climbing - Random Restart

3.1.5.1. Parameter Input

Berikut adalah parameter input yang digunakan saat menjalankan eksperimen :

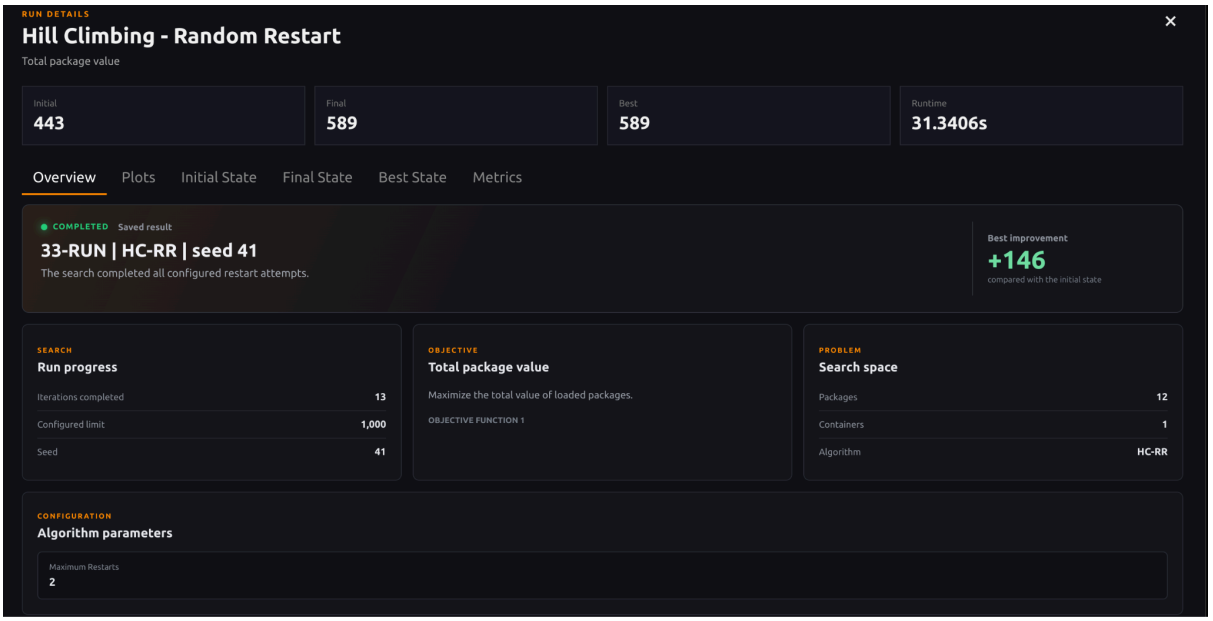
Test case	Objective function	Package count	Truck count	Maximum iterations	Seed	Maximum restarts
1	Total package value	12	1	1000	41	2

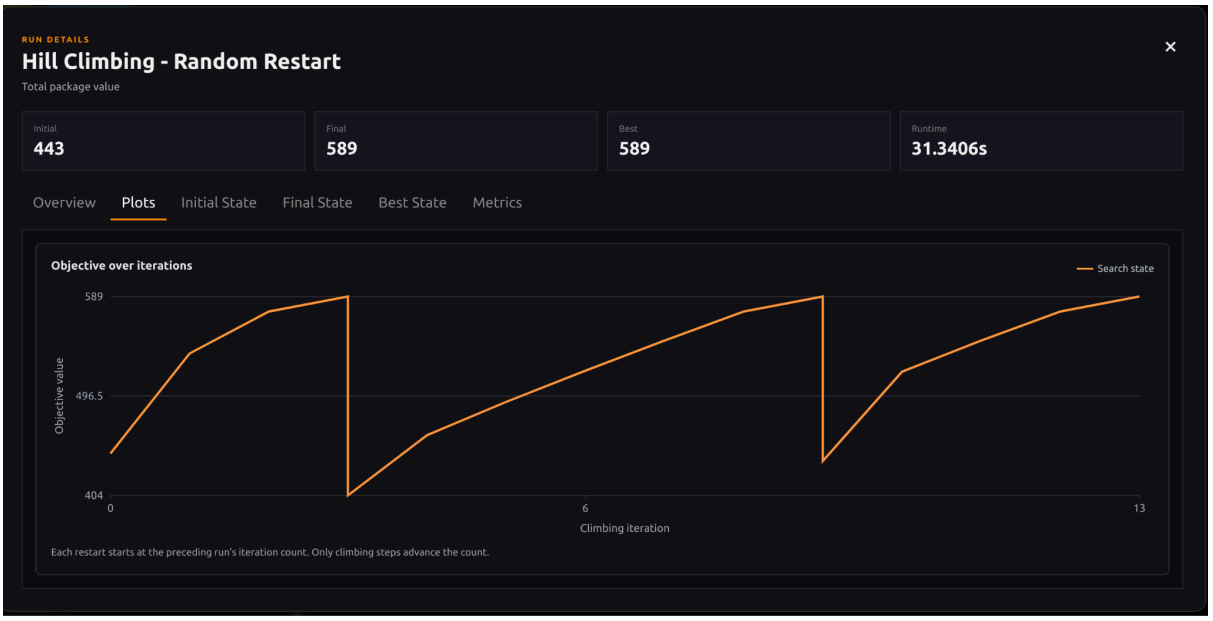
2	Total package value	12	1	1000	42	2
3	Total package value	12	1	1000	43	2

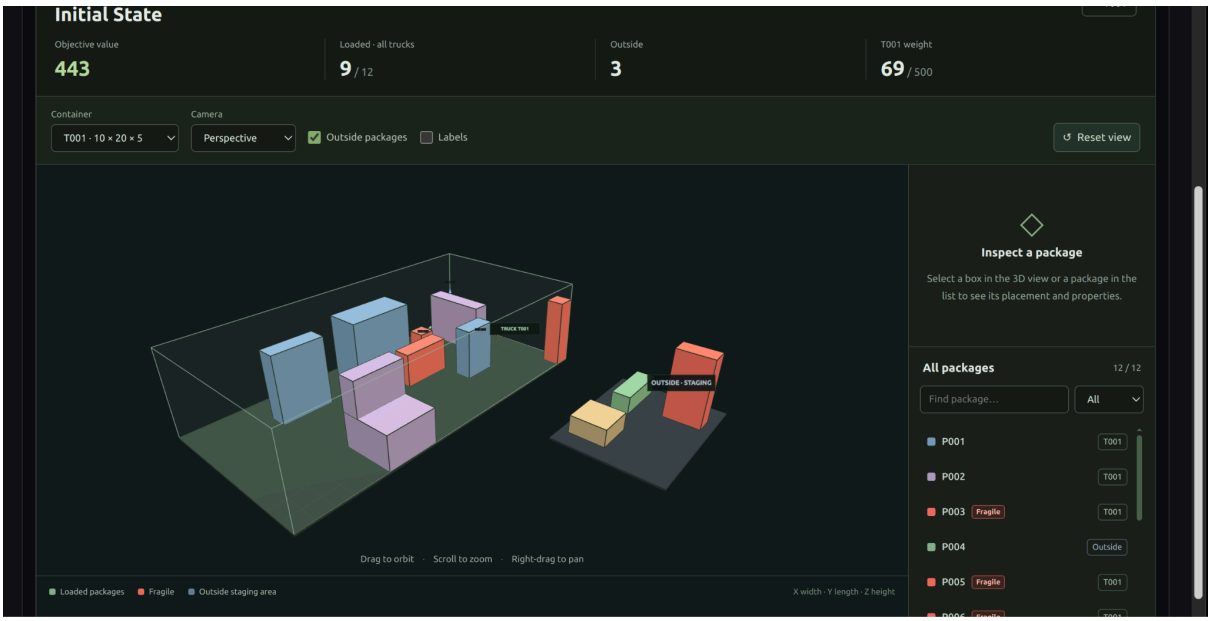
3.1.5.2. Hasil Eksperimen

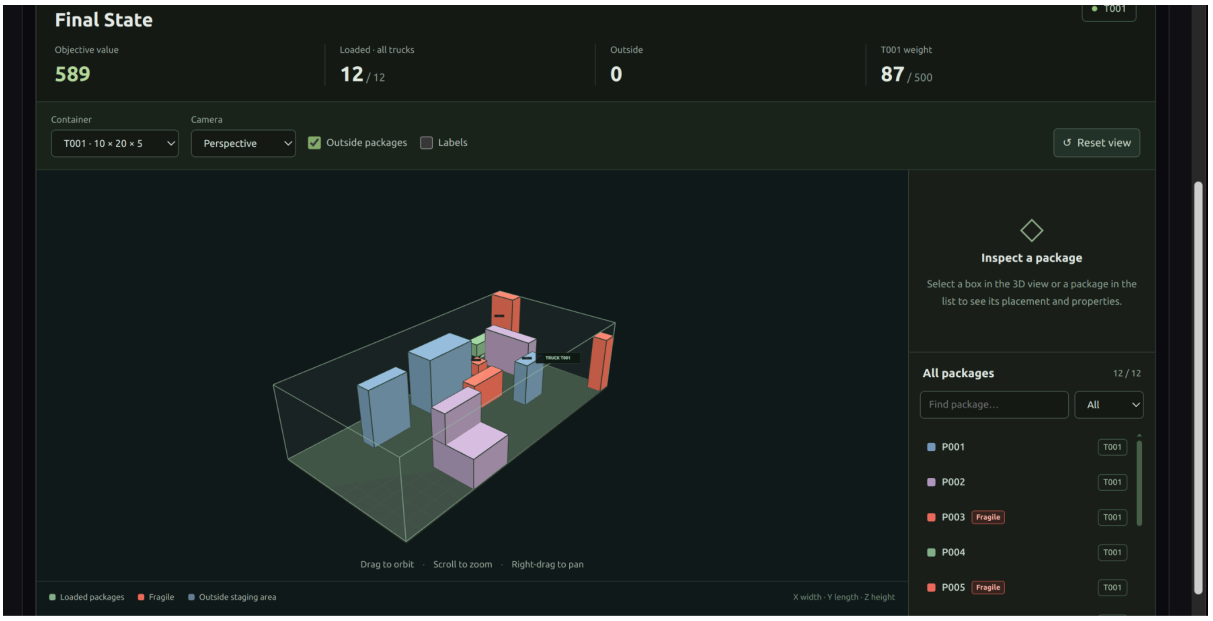
Berikut adalah hasil eksperimen untuk setiap test case :

Test Case 1 :

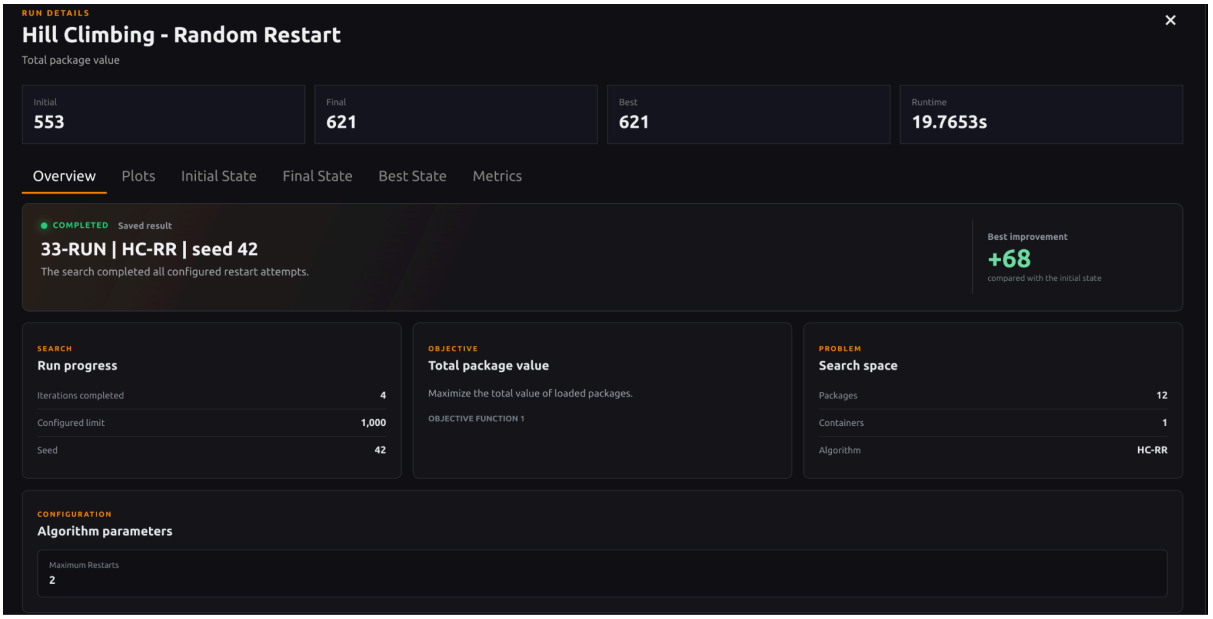


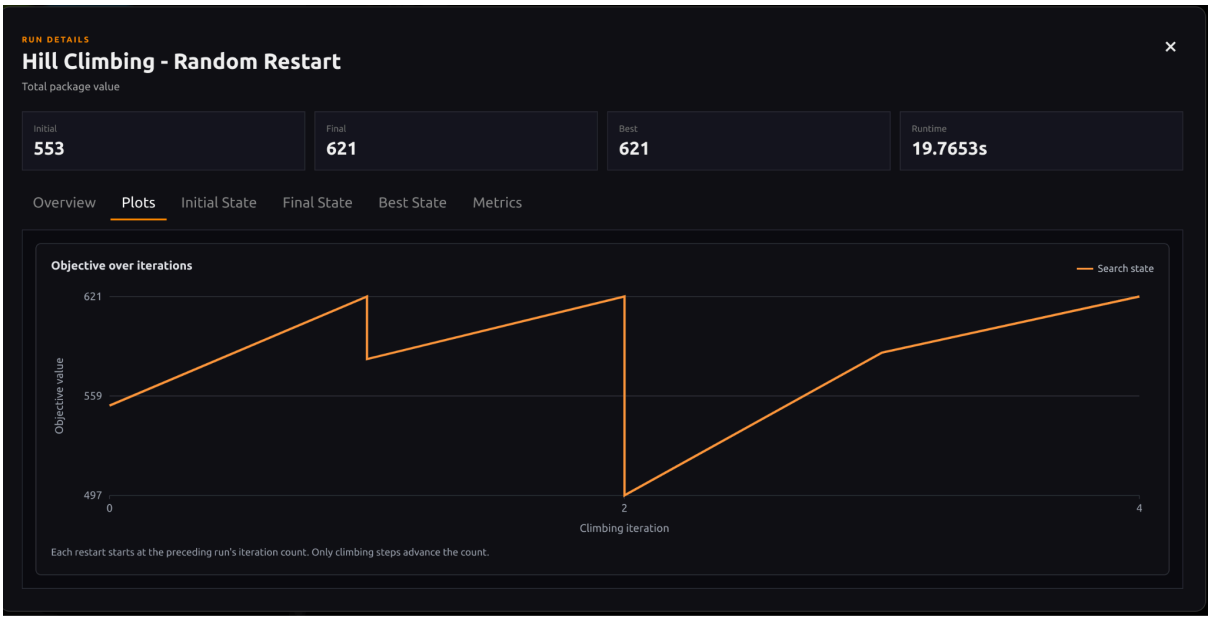


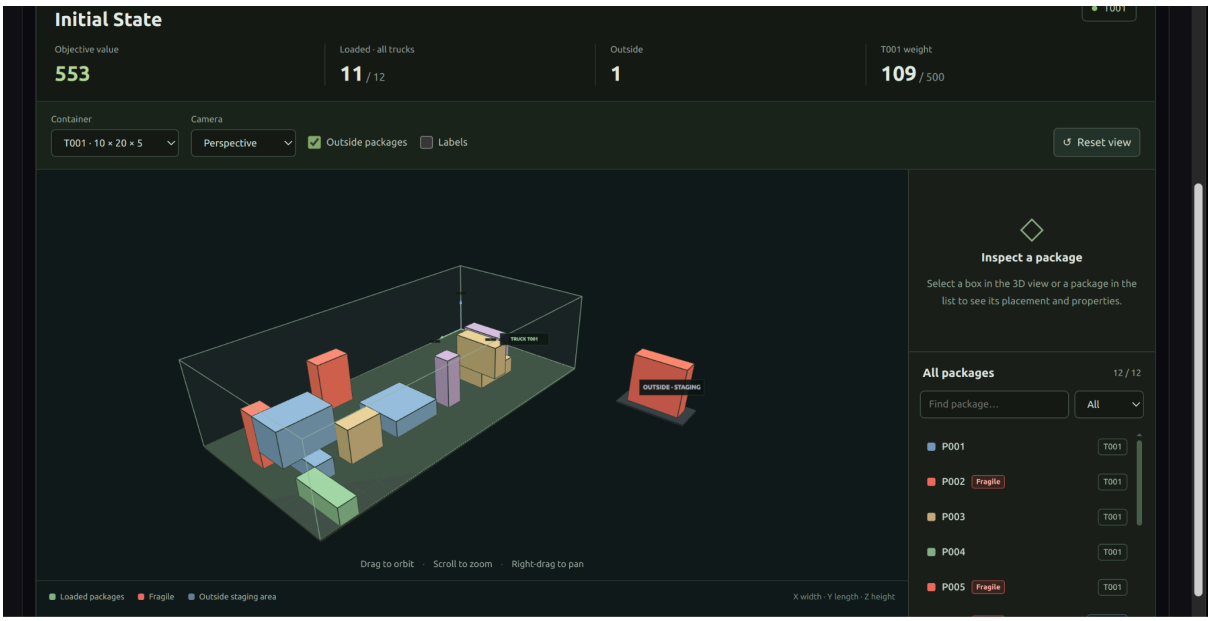

Initial State
Objective value
443
Loaded · all trucks
9 / 12
Outside
3
T001 weight
69 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 T001
P003 Fragile T001
P004 Outside
P005 Fragile T001

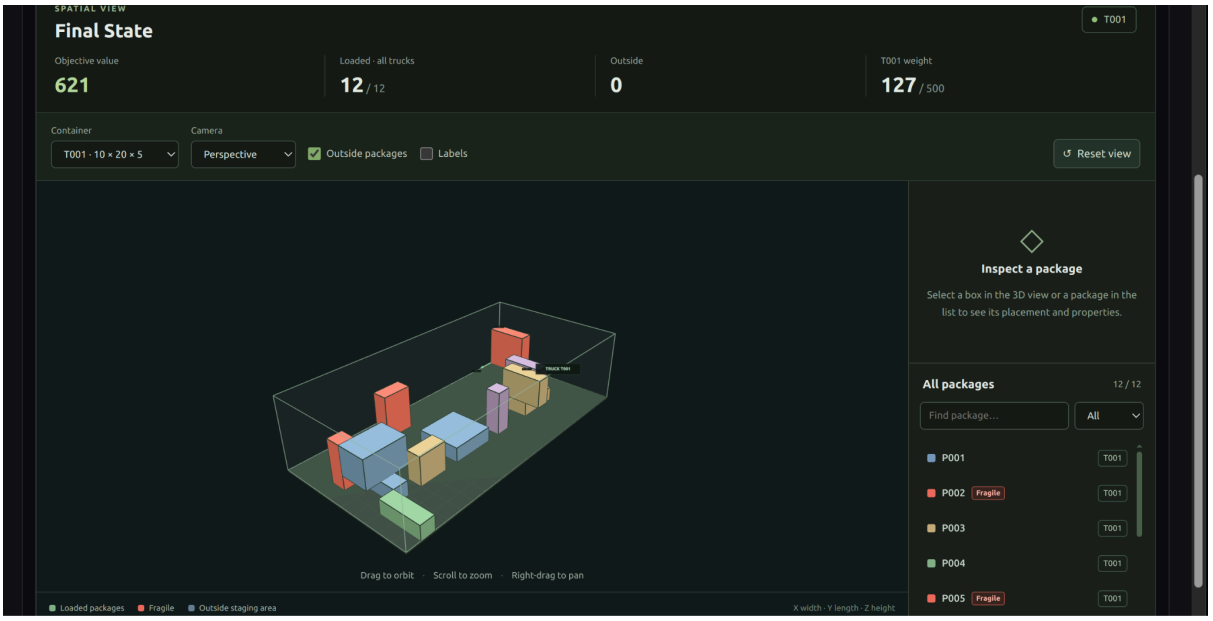

Final State
Objective value
589
Loaded · all trucks
12 / 12
Outside
0
T001 weight
87 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 T001
P003 Fragile T001
P004 T001
P005 Fragile T001

Test Case 2 :

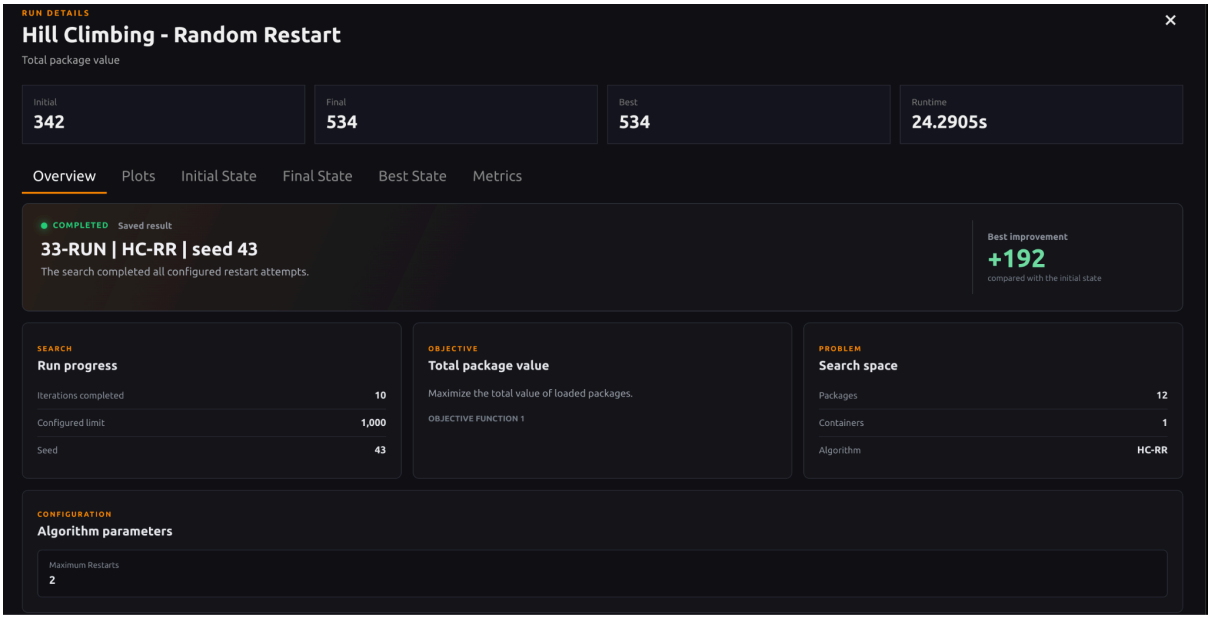

RUN DETAILS
Hill Climbing - Random Restart
Total package value
Initial
553
Final
621
Best
621
Runtime
19.7653s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | HC-RR | seed 42
The search completed all configured restart attempts.
Best improvement
+68
compared with the initial state
SEARCH
Run progress
Iterations completed
4
Configured limit
1,000
Seed
42
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
HC-RR
CONFIGURATION
Algorithm parameters
Maximum Restarts
2

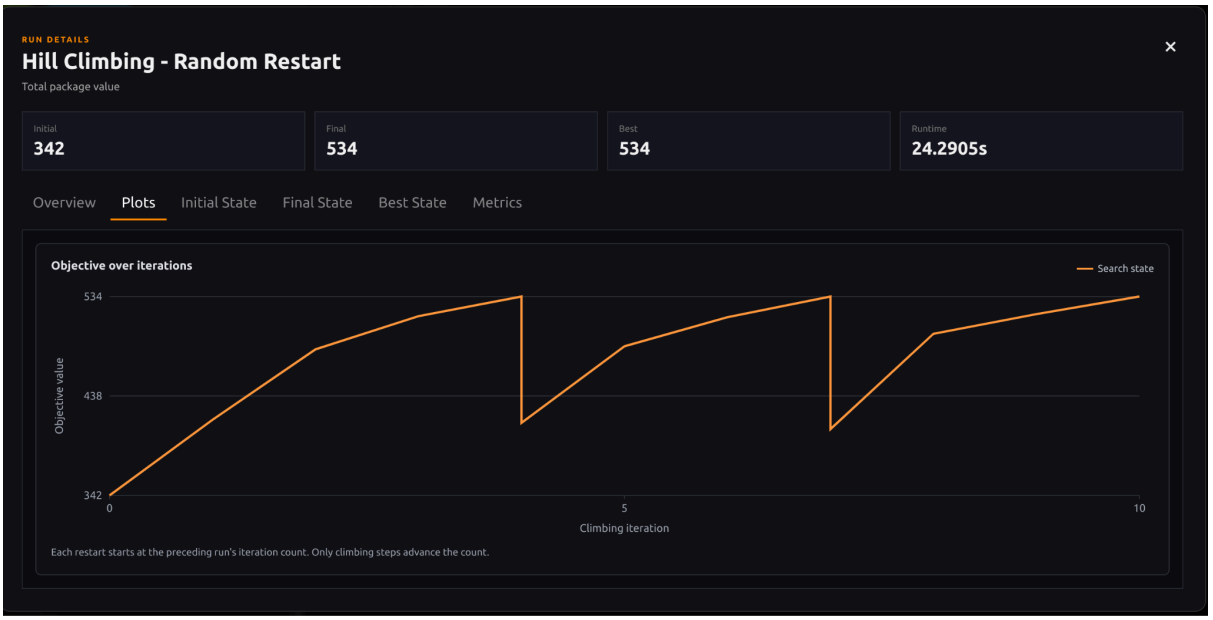

RUN DETAILS
Hill Climbing - Random Restart
Total package value
Initial
553
Final
621
Best
621
Runtime
19.7653s
Overview
Plots
Initial State
Final State
Best State
Metrics
Objective over iterations
Search state
621
559
497
Objective value
0
2
4
Climbing iteration
Each restart starts at the preceding run's iteration count. Only climbing steps advance the count.

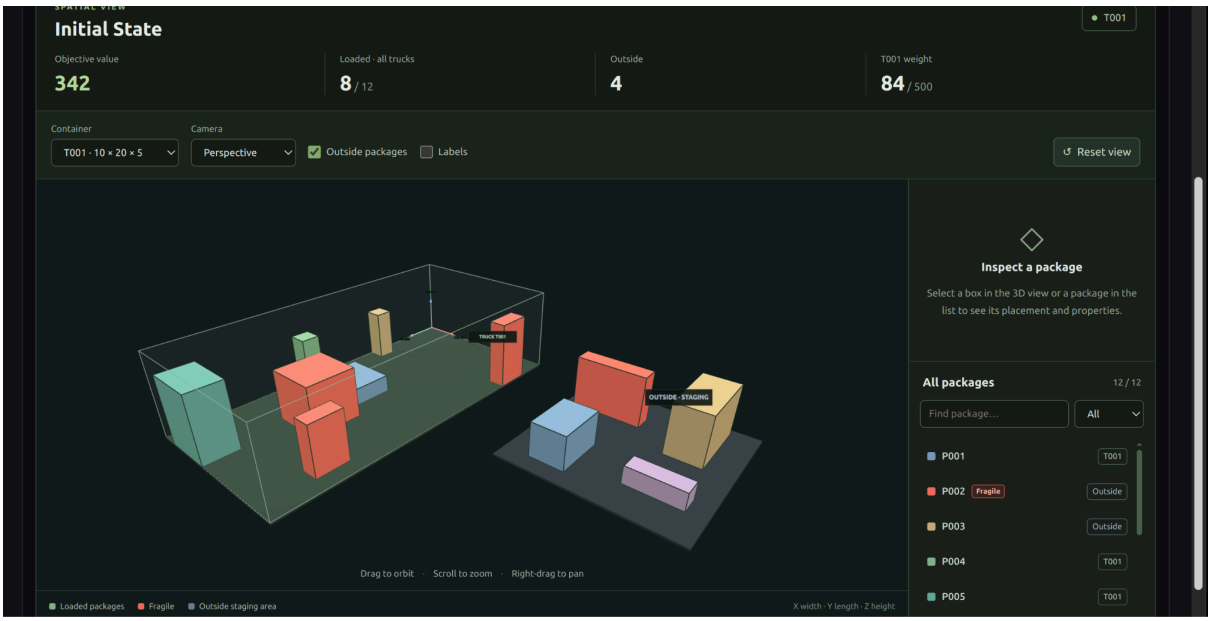

Initial State
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
OUTSIDE · STAGING
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Drag to orbit
Scroll to zoom
Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

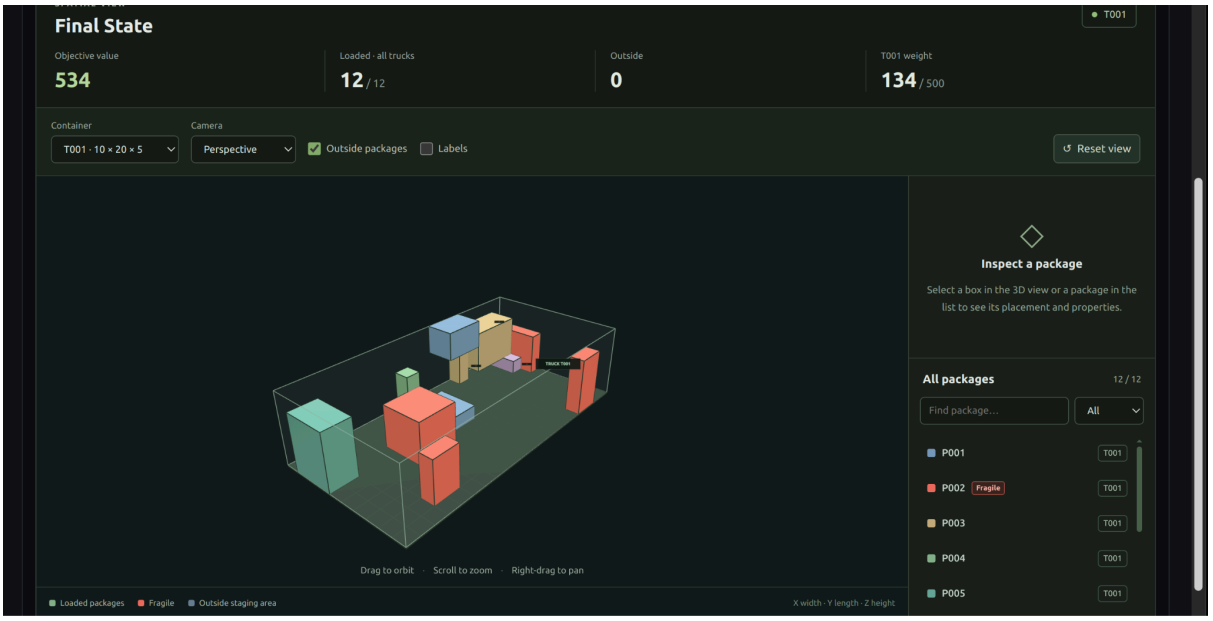

SPATIAL VIEW
Final State
T001
Objective value
621
Loaded · all trucks
12 / 12
Outside
0
T001 weight
127 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Drag to orbit
Scroll to zoom
Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

Test Case 3 :


RUN DETAILS
Hill Climbing - Random Restart
Total package value
Initial
342
Final
534
Best
534
Runtime
24.2905s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | HC-RR | seed 43
The search completed all configured restart attempts.
Best improvement
+192
compared with the initial state
SEARCH
Run progress
Iterations completed
10
Configured limit
1,000
Seed
43
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
HC-RR
CONFIGURATION
Algorithm parameters
Maximum Restarts
2


RUN DETAILS
Hill Climbing - Random Restart
Total package value
Initial
342
Final
534
Best
534
Runtime
24.2905s
Overview
Plots
Initial State
Final State
Best State
Metrics
Objective over iterations
Search state
534
438
342
Objective value
0
5
10
Climbing iteration
Each restart starts at the preceding run's iteration count. Only climbing steps advance the count.





3.2. Simulated Annealing

3.2.1. Parameter Input

Berikut adalah parameter input yang digunakan saat menjalankan eksperimen :

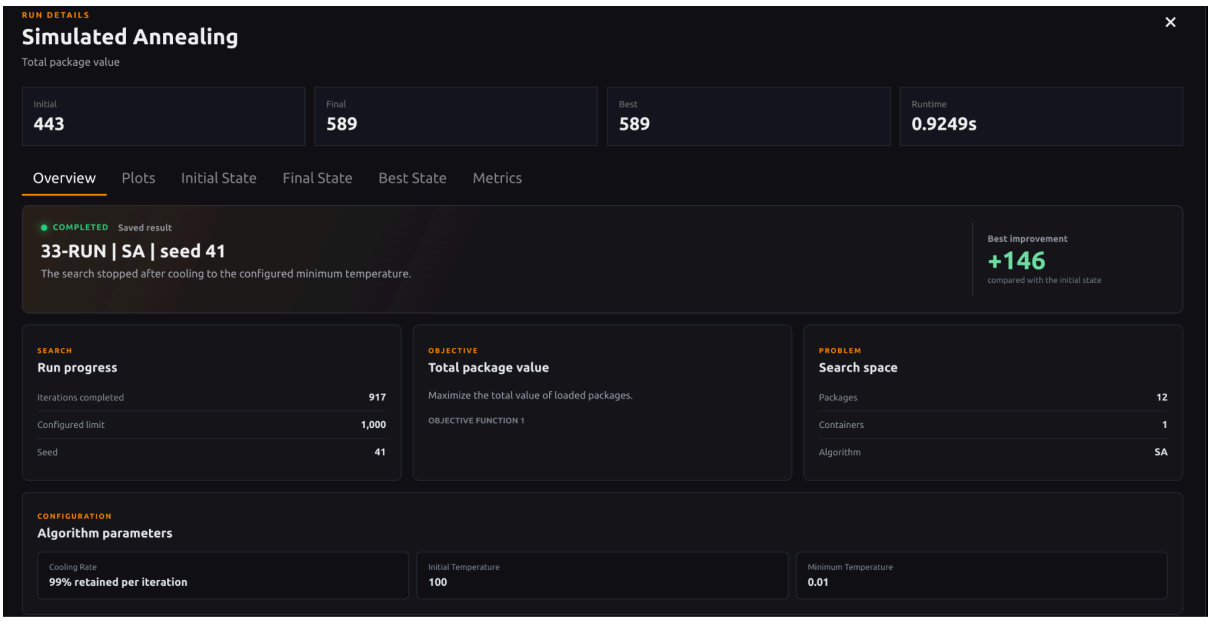
Test case	Objective function	Package count	Truck count	Maximum iterations	Seed	Initial temperature	Cooling rate	Minimum temperature
1	Total package	12	1	1000	41	100	0.99	0.01

	value							
2	Total package value	12	1	1000	42	100	0.99	0.01
3	Total package value	12	1	1000	43	100	0.99	0.01

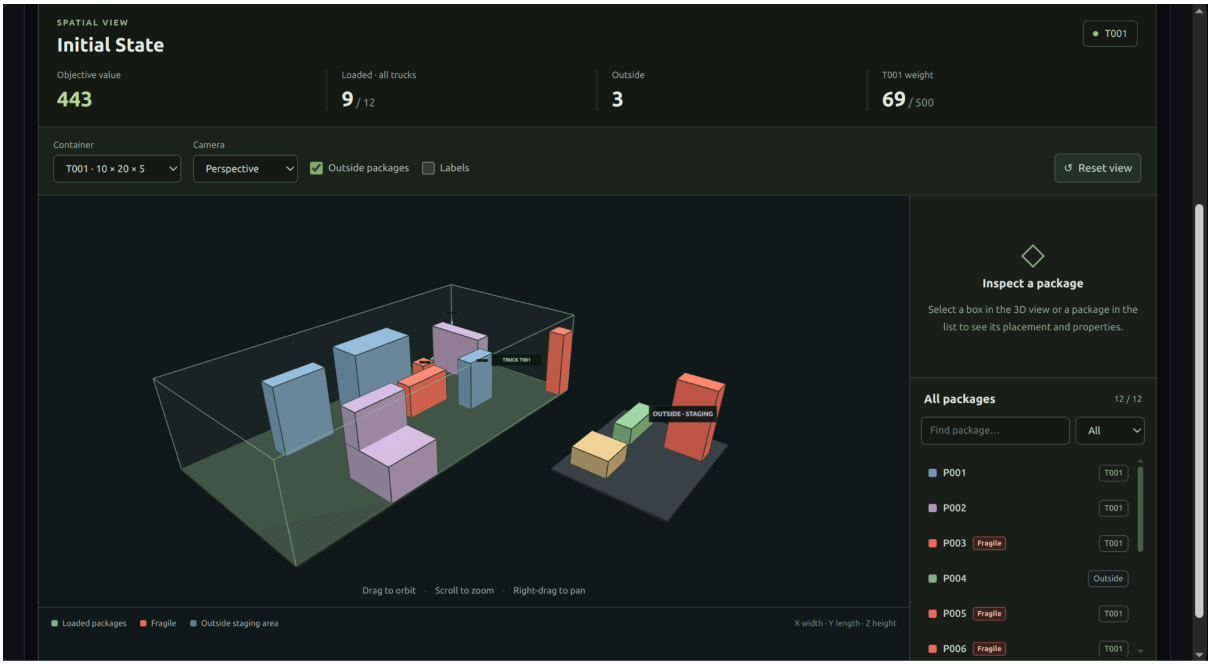
3.2.2. Hasil Eksperimen

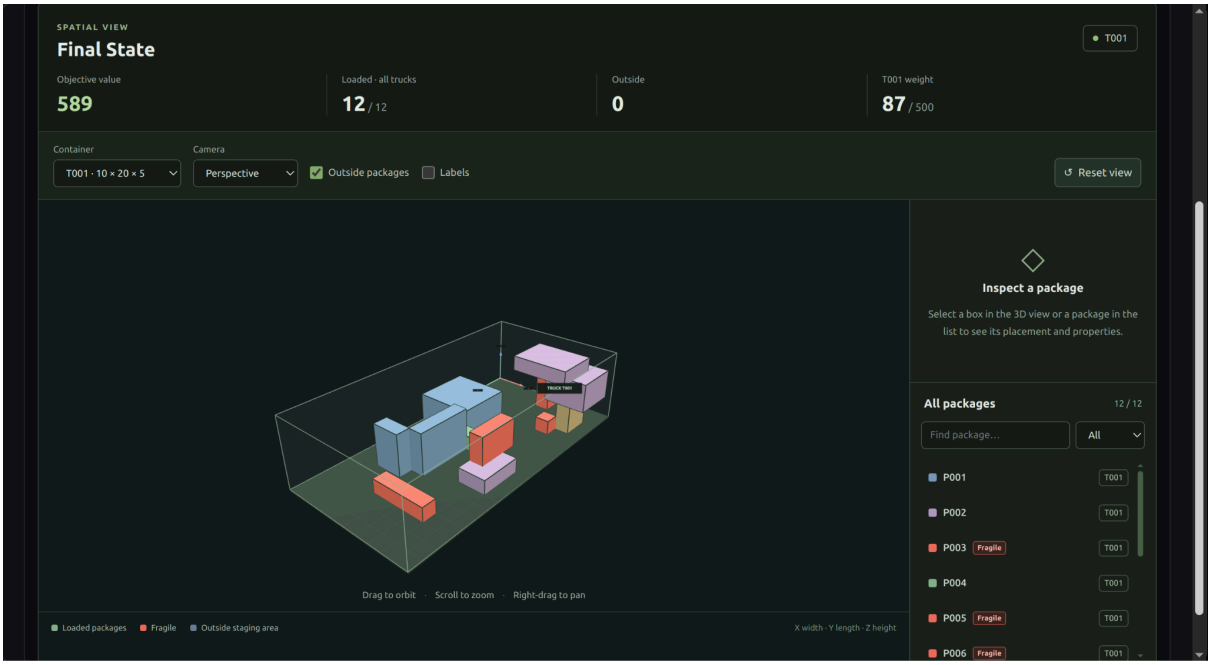
Berikut adalah hasil eksperimen untuk setiap test case :

Test Case 1 :

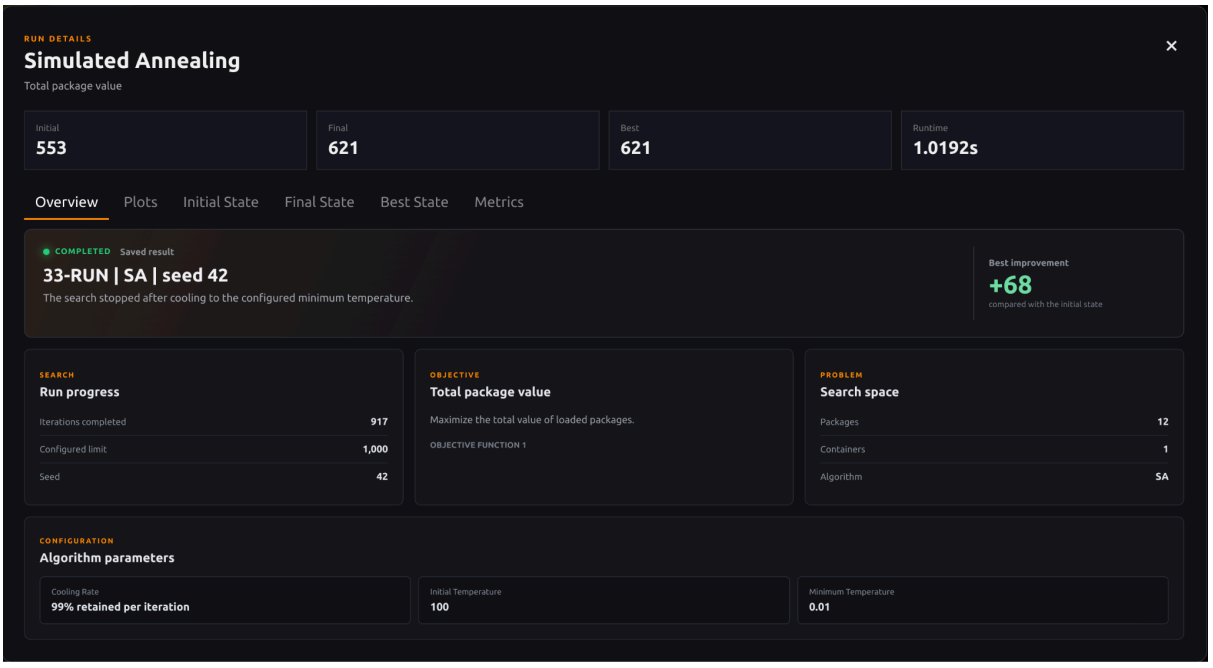


Objective over iterations
Current state
Best so far
589
438
287
Objective value
0
458
917
Iteration
SA acceptance probability
Acceptance probability
1
0.5
0
Probability
1
459
917
Iteration
Non-worse proposals have probability 1. Worse proposals use exp(ΔE / T).


SPATIAL VIEW
Initial State
T001
Objective value
443
Loaded · all trucks
9 / 12
Outside
3
T001 weight
69 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
T001
P003
Fragile
T001
P004
Outside
P005
Fragile
T001
P006
Fragile
T001

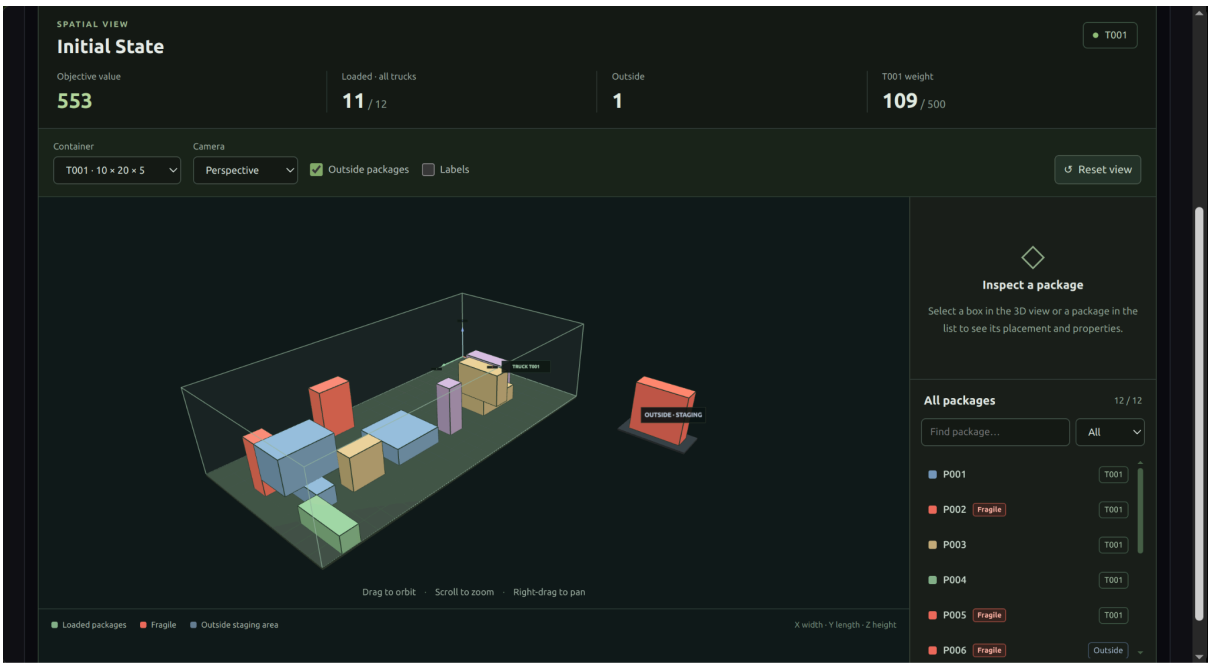

SPATIAL VIEW
Final State
T001
Objective value
589
Loaded · all trucks
12 / 12
Outside
0
T001 weight
87 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 T001
P003 Fragile T001
P004 T001
P005 Fragile T001
P006 Fragile T001

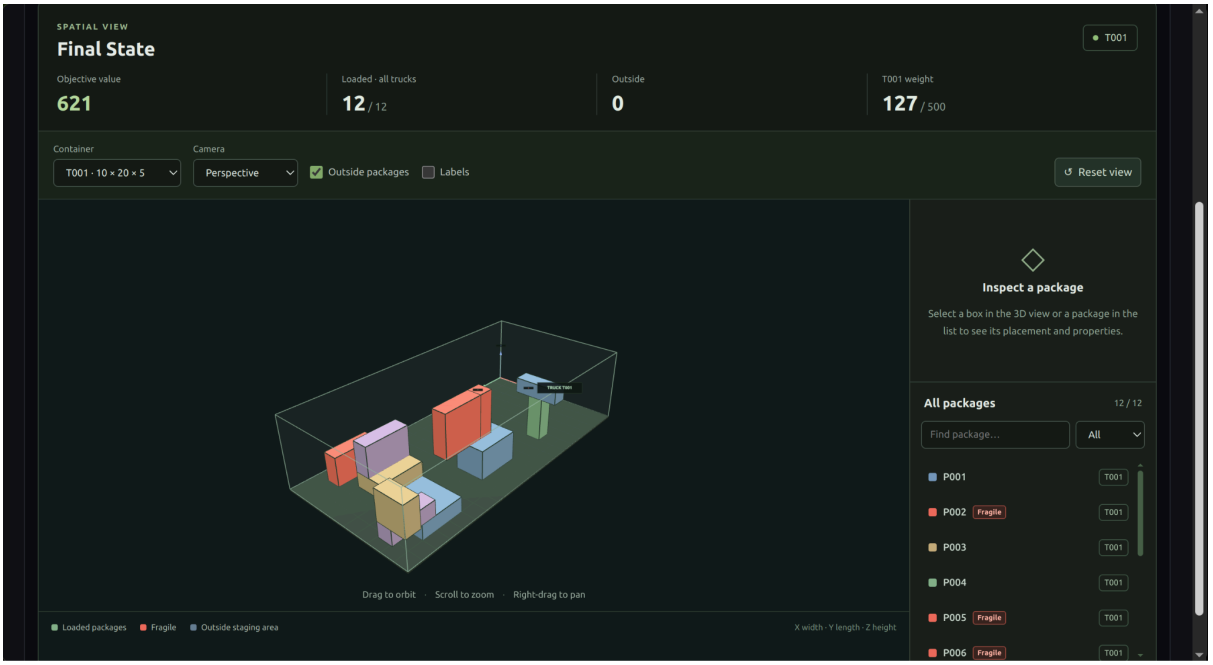
Test Case 2 :


RUN DETAILS
Simulated Annealing
Total package value
Initial
553
Final
621
Best
621
Runtime
1.0192s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | SA | seed 42
The search stopped after cooling to the configured minimum temperature.
Best improvement
+68
compared with the initial state
SEARCH
Run progress
Iterations completed 917
Configured limit 1,000
Seed 42
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages 12
Containers 1
Algorithm SA
CONFIGURATION
Algorithm parameters
Cooling Rate
99% retained per iteration
Initial Temperature
100
Minimum Temperature
0.01

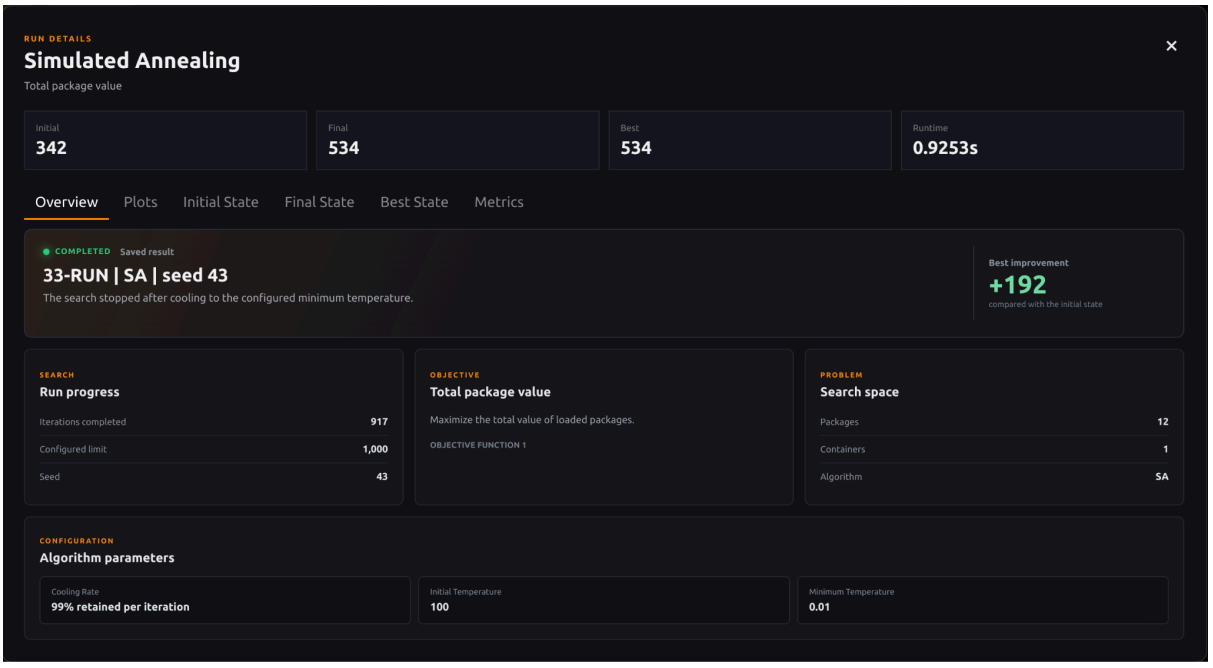
SIMULATED ANNEALING
Movement and stagnation
Stuck Frequency
10.47%
Stuck / Completed Iterations
96 / 917
Accepted Proposals
821
Rejected Proposals
96
An iteration counts as stuck when its proposed neighbor is rejected, so the current state does not change. Frequency is rejected proposals divided by completed iterations.

Objective over iterations
Current state
Best so far
621
537.5
454
Objective value
0
458
917
Iteration
SA acceptance probability
Acceptance probability
1
0.5
0
Probability
1
459
917
Iteration
Non-worse proposals have probability 1. Worse proposals use exp(ΔE / T).

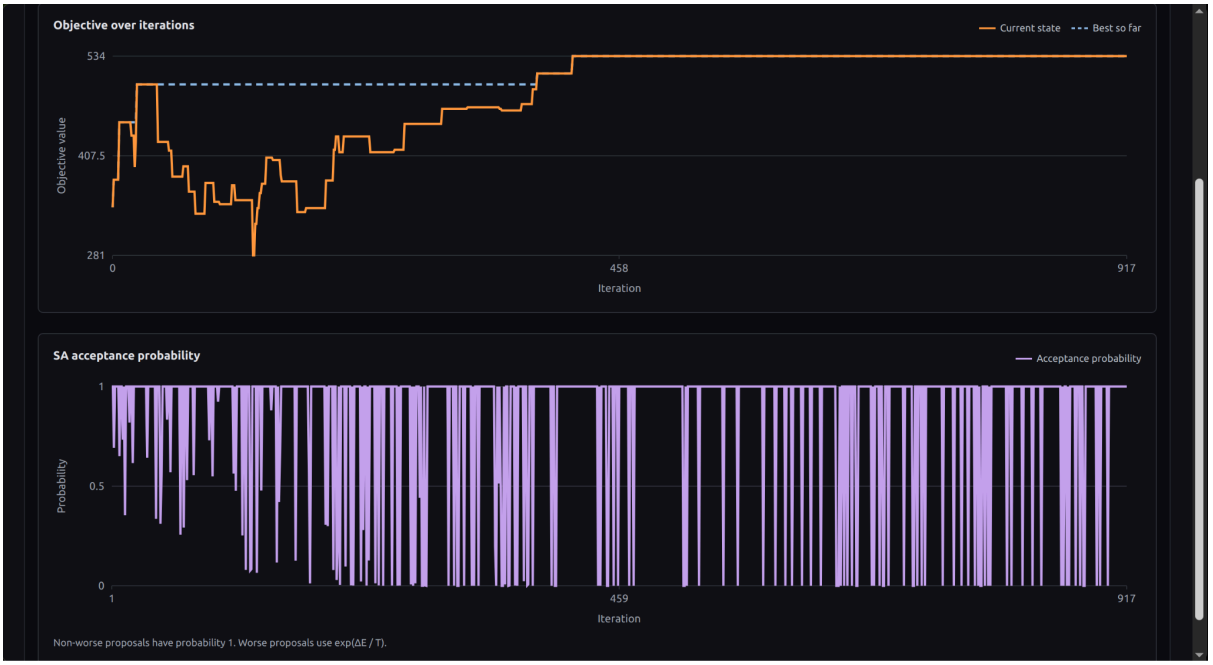

SPATIAL VIEW
Initial State
T001
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
T001
P003
T001
P004
T001
P005
Fragile
T001
P006
Fragile
Outside

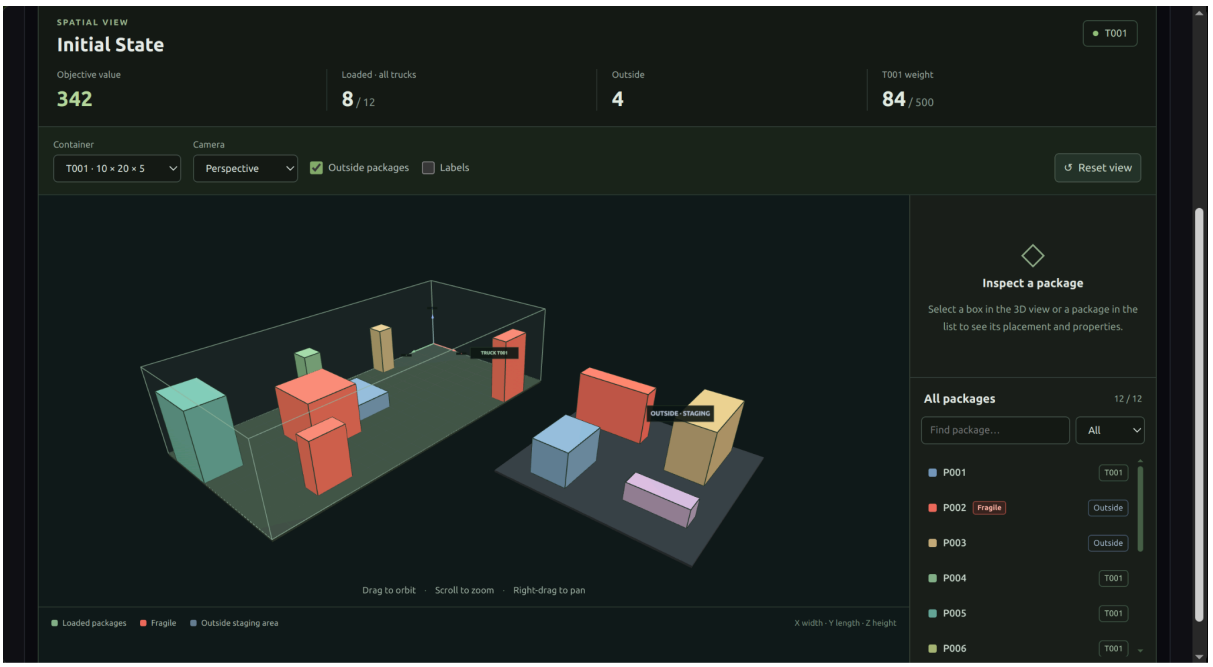

SPATIAL VIEW
Final State
T001
Objective value
621
Loaded · all trucks
12 / 12
Outside
0
T001 weight
127 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Fragile
P006
Fragile
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

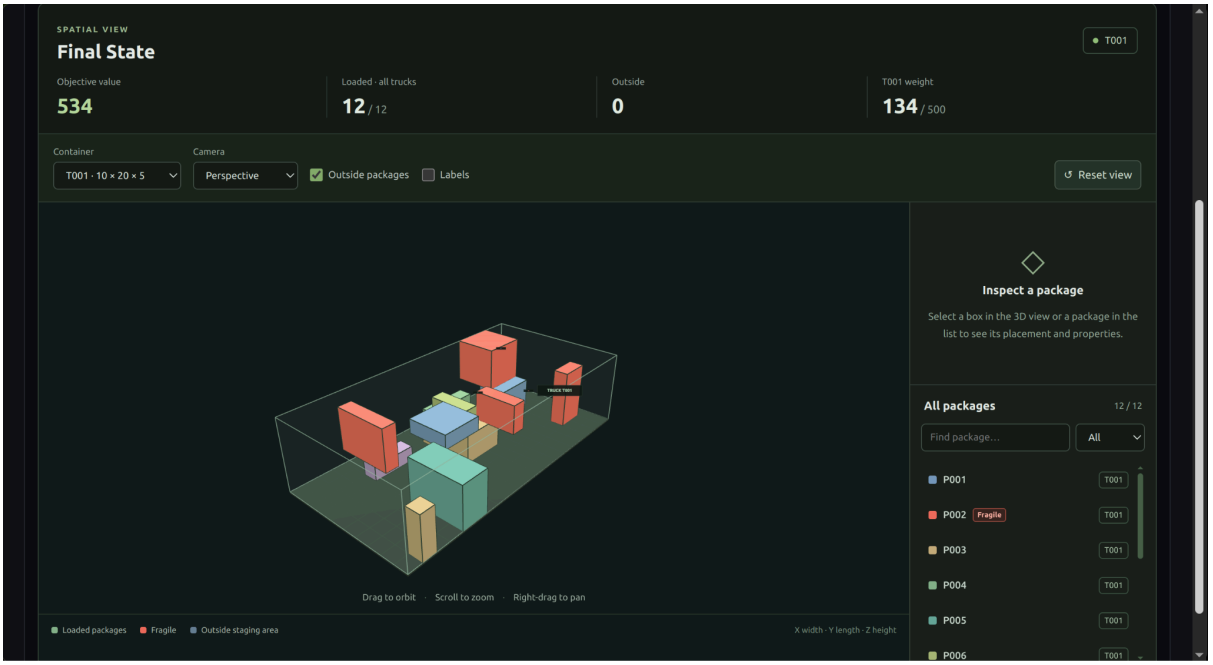
Test Case 3 :


RUN DETAILS
Simulated Annealing
Total package value
Initial
342
Final
534
Best
534
Runtime
0.9253s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | SA | seed 43
The search stopped after cooling to the configured minimum temperature.
Best improvement
+192
compared with the initial state
SEARCH
Run progress
Iterations completed
917
Configured limit
1,000
Seed
43
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
SA
CONFIGURATION
Algorithm parameters
Cooling Rate
99% retained per iteration
Initial Temperature
100
Minimum Temperature
0.01

SIMULATED ANNEALING
Movement and stagnation
Stuck Frequency
14.94%
Stuck / Completed Iterations
137 / 917
Accepted Proposals
780
Rejected Proposals
137
An iteration counts as stuck when its proposed neighbor is rejected, so the current state does not change. Frequency is rejected proposals divided by completed iterations.


Objective over iterations
Current state
Best so far
534
407.5
281
Objective value
0
458
917
Iteration
SA acceptance probability
Acceptance probability
1
0.5
0
Probability
1
459
917
Iteration
Non-worse proposals have probability 1. Worse proposals use exp(ΔE / T).


SPATIAL VIEW
Initial State
T001
Objective value
342
Loaded · all trucks
8 / 12
Outside
4
T001 weight
84 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
Outside
P003
Outside
P004
T001
P005
T001
P006
T001



3.3. Genetic Algorithm

3.3.1. Jumlah Populasi sebagai Kontrol

3.3.1.1. Parameter Input

Berikut adalah parameter input yang digunakan saat menjalankan eksperimen :

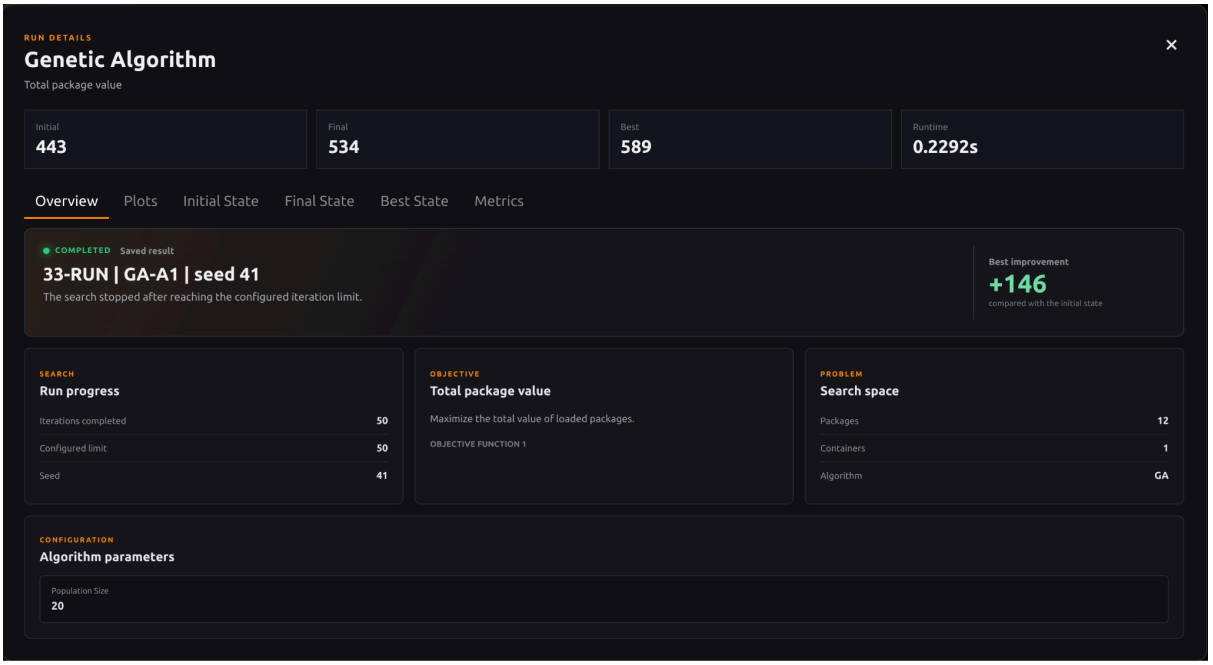
Test case	Objective function	Package count	Truck count	Maximum iterations	Seed	Population size
1	Total package value	12	1	50	41	20
2	Total package value	12	1	50	42	20
3	Total package value	12	1	50	43	20
4	Total package value	12	1	100	41	20
5	Total package value	12	1	100	42	20
6	Total package	12	1	100	43	20

	value					
7	Total package value	12	1	200	41	20
8	Total package value	12	1	200	42	20
9	Total package value	12	1	200	43	20

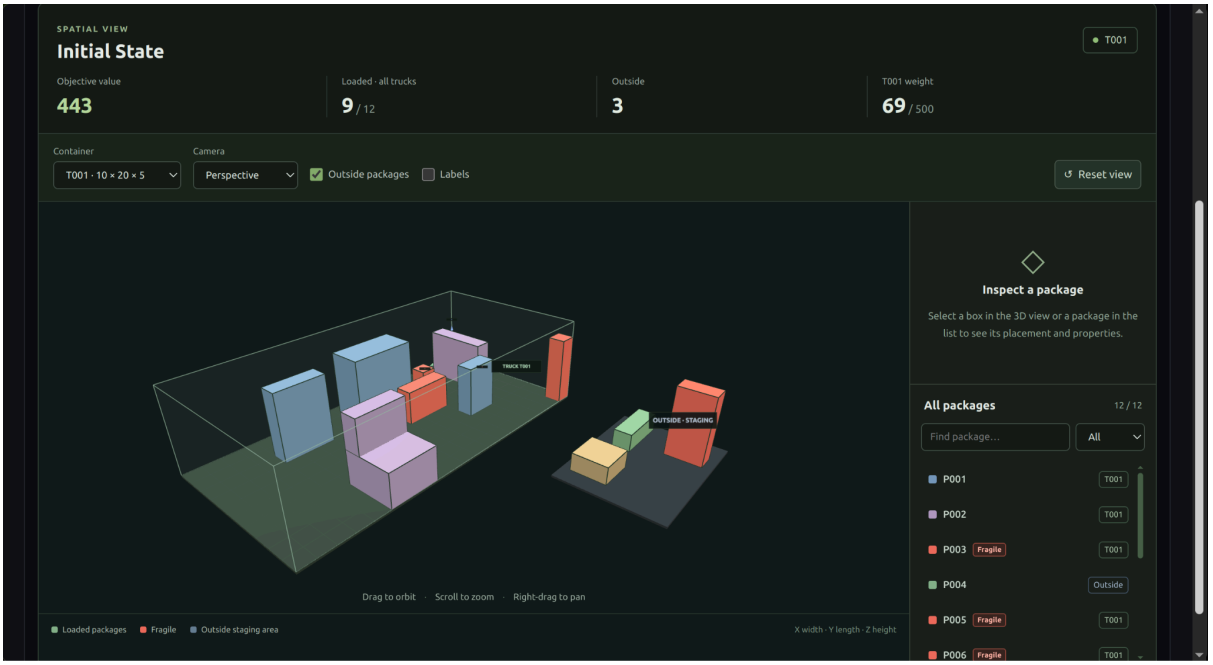
3.3.1.2. Hasil Eksperimen

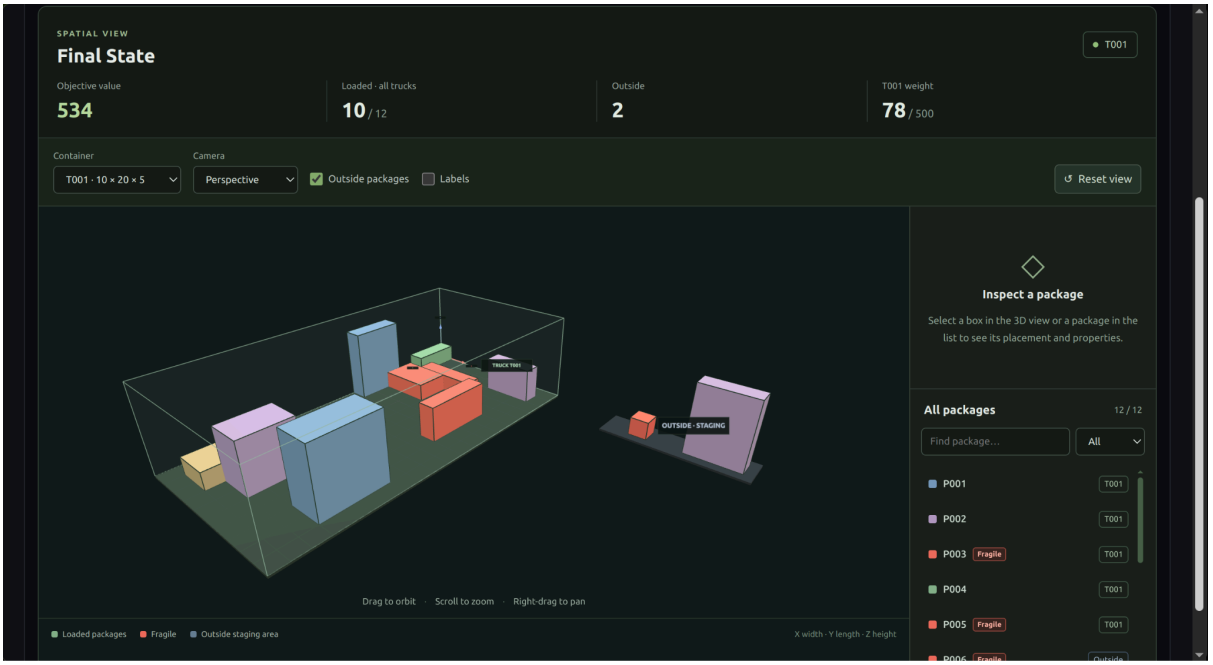
Berikut adalah hasil eksperimen untuk setiap test case :

Test Case 1 :

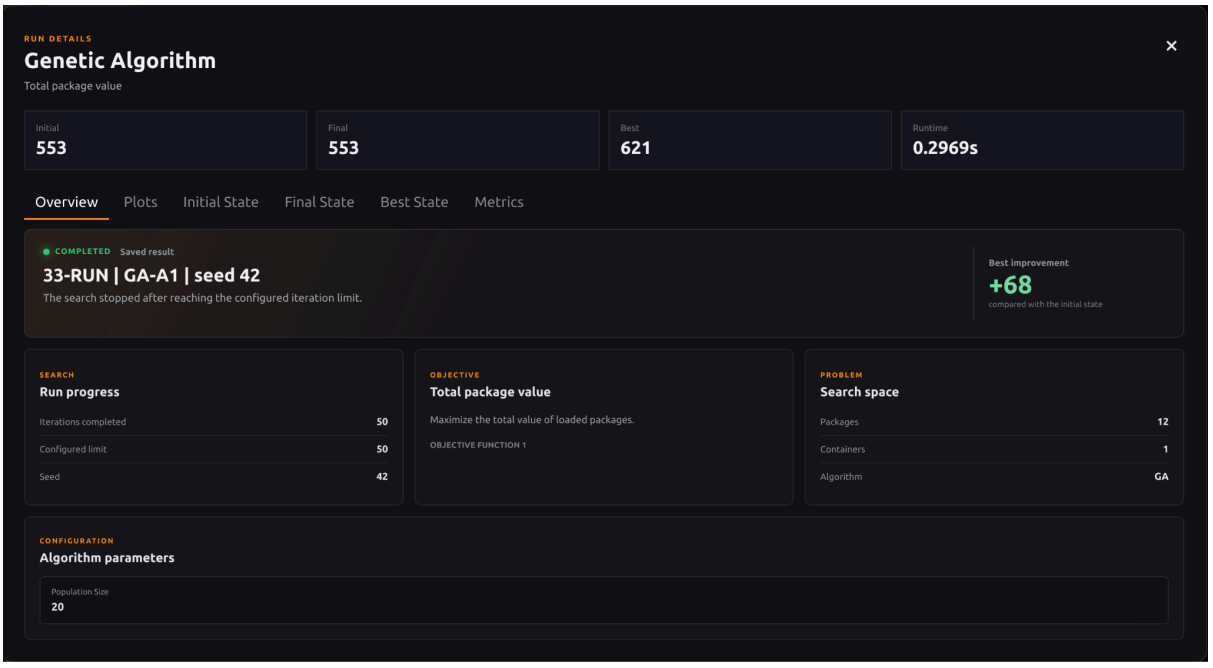


RUN DETAILS
Genetic Algorithm
Total package value
Initial
443
Final
534
Best
589
Runtime
0.2292s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
589
499.525
410.05
Objective value
0
25
50
Iteration (generation)

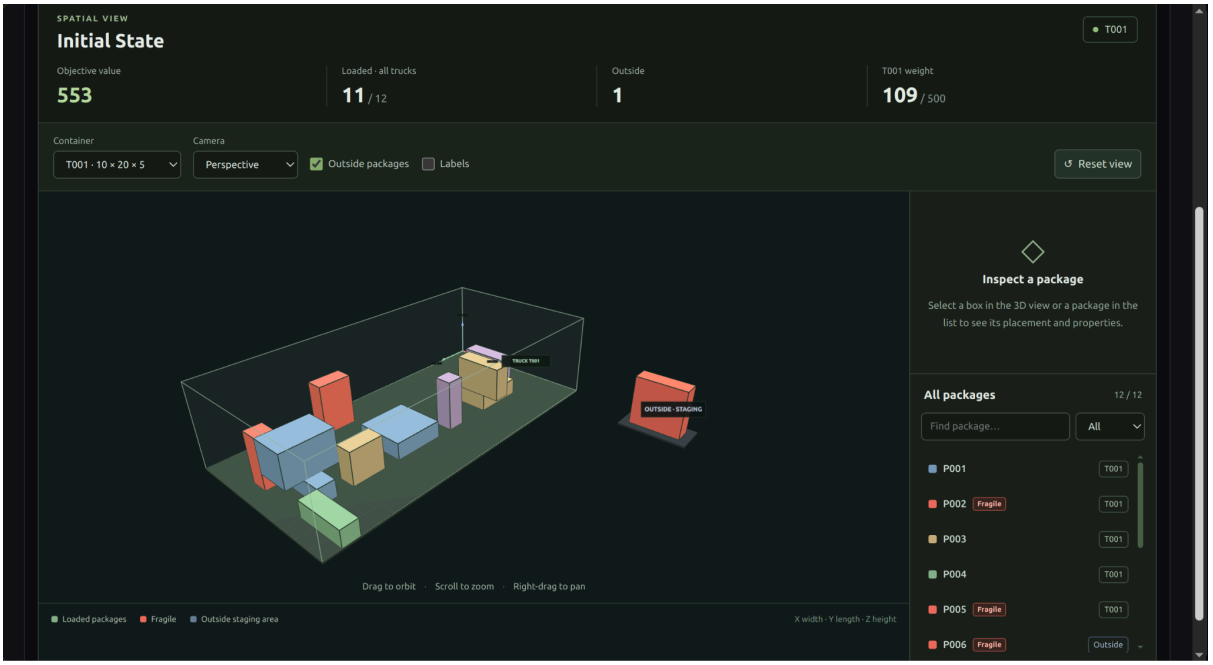

SPATIAL VIEW
Initial State
T001
Objective value
443
Loaded · all trucks
9 / 12
Outside
3
T001 weight
69 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
T001
P003
Fragile
T001
P004
Outside
P005
Fragile
T001
P006
Fragile
T001

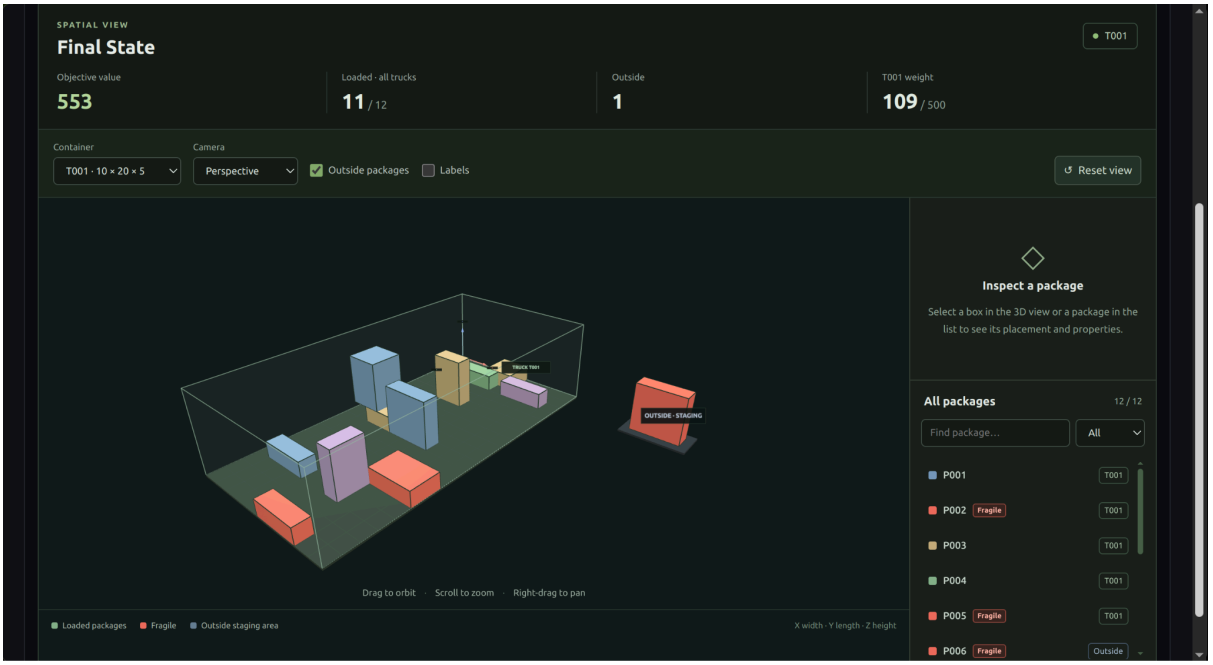

SPATIAL VIEW
Final State
T001
Objective value
534
Loaded · all trucks
10 / 12
Outside
2
T001 weight
78 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
P003
Fragile
P004
P005
Fragile

Test Case 2 :

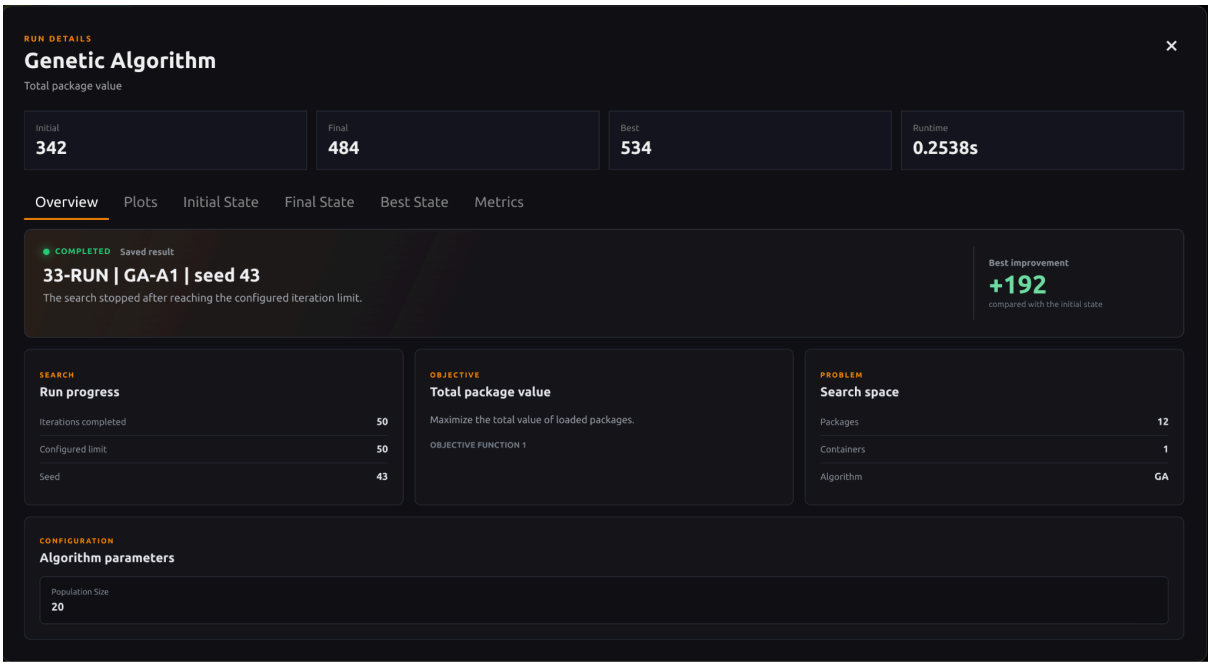

RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
553
Best
621
Runtime
0.2969s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-A1 | seed 42
The search stopped after reaching the configured iteration limit.
Best improvement
+68
compared with the initial state
SEARCH
Run progress
Iterations completed
50
Configured limit
50
Seed
42
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
GA
CONFIGURATION
Algorithm parameters
Population Size
20

RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
553
Best
621
Runtime
0.2969s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
621
535.5
450
Objective value
0
25
50
Iteration (generation)

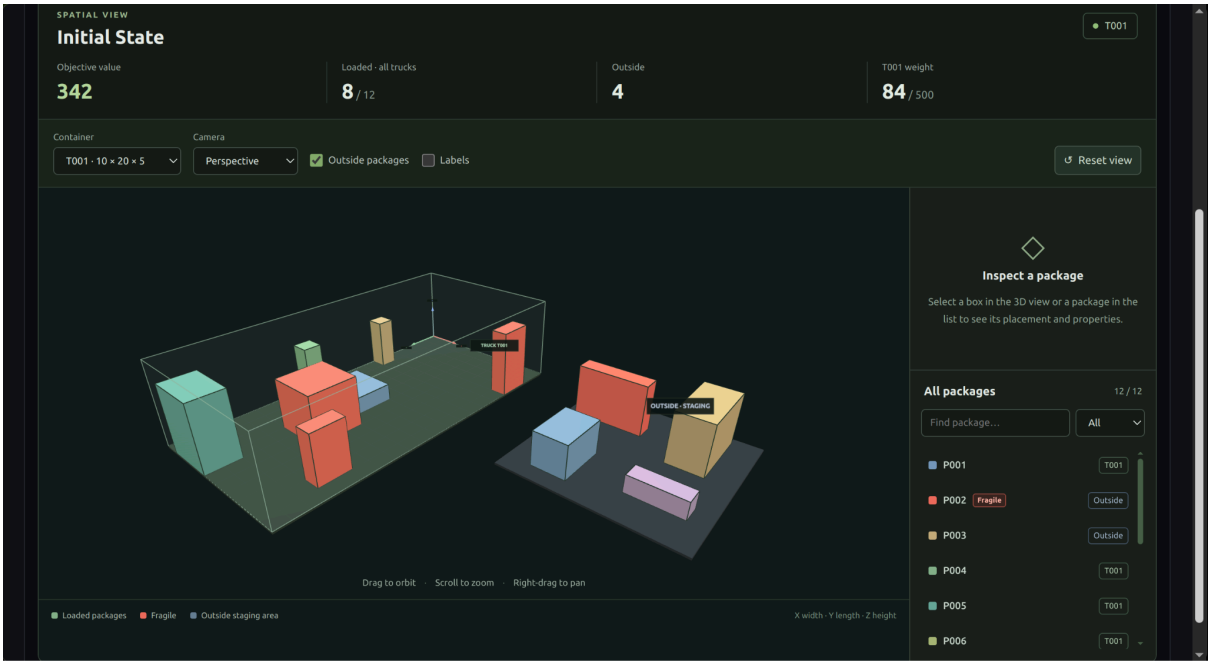

SPATIAL VIEW
Initial State
T001
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Fragile
P006
Fragile
Outside
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

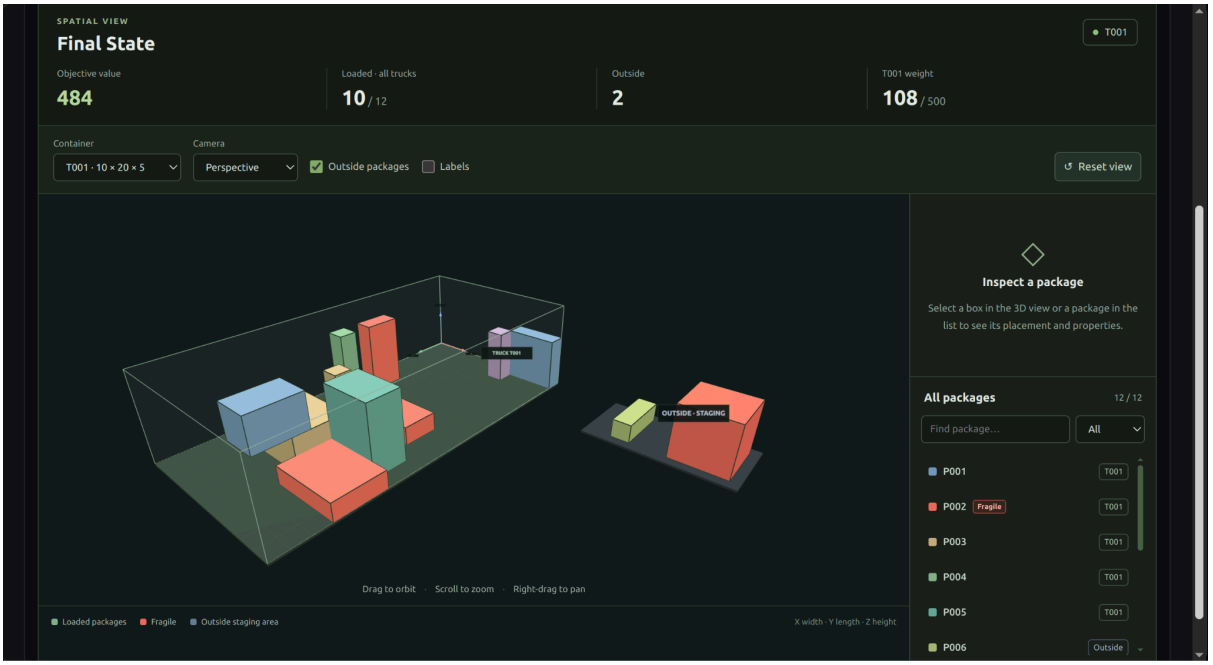

SPATIAL VIEW
Final State
T001
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 Fragile T001
P003 T001
P004 T001
P005 Fragile T001
P006 Fragile Outside

Test Case 3 :

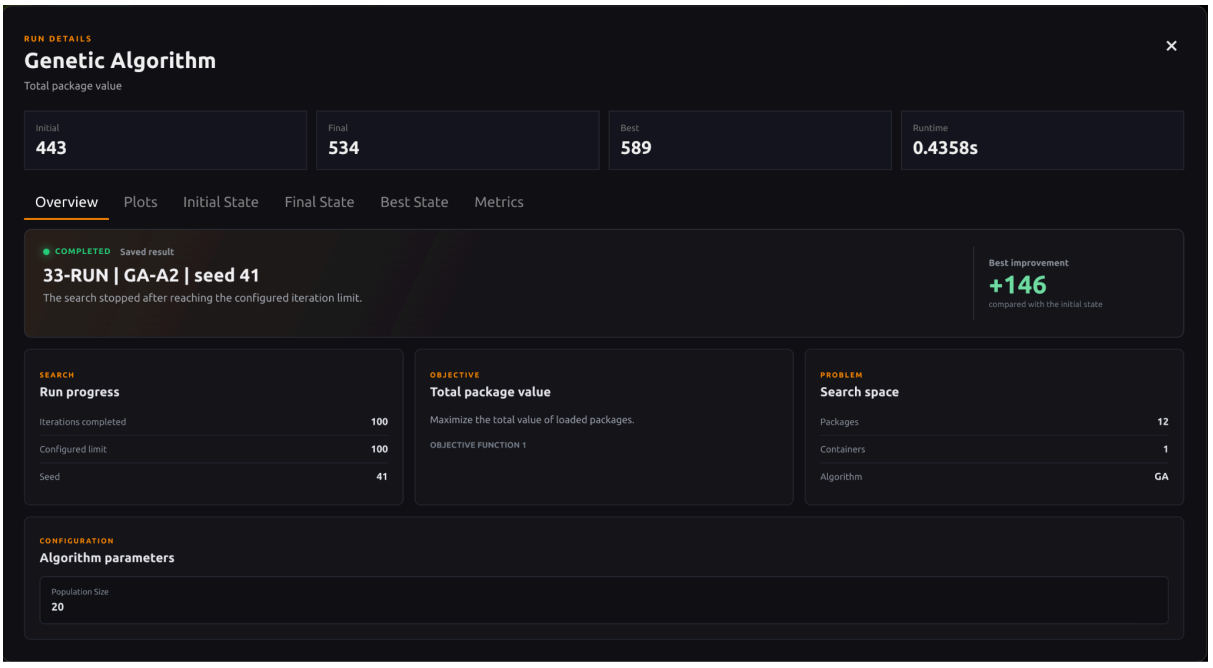

RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
484
Best
534
Runtime
0.2538s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-A1 | seed 43
The search stopped after reaching the configured iteration limit.
Best improvement
+192
compared with the initial state
SEARCH
Run progress
Iterations completed 50
Configured limit 50
Seed 43
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages 12
Containers 1
Algorithm GA
CONFIGURATION
Algorithm parameters
Population Size
20

RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
484
Best
534
Runtime
0.2538s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
534
439.1
344.2
Objective value
0
25
50
Iteration (generation)

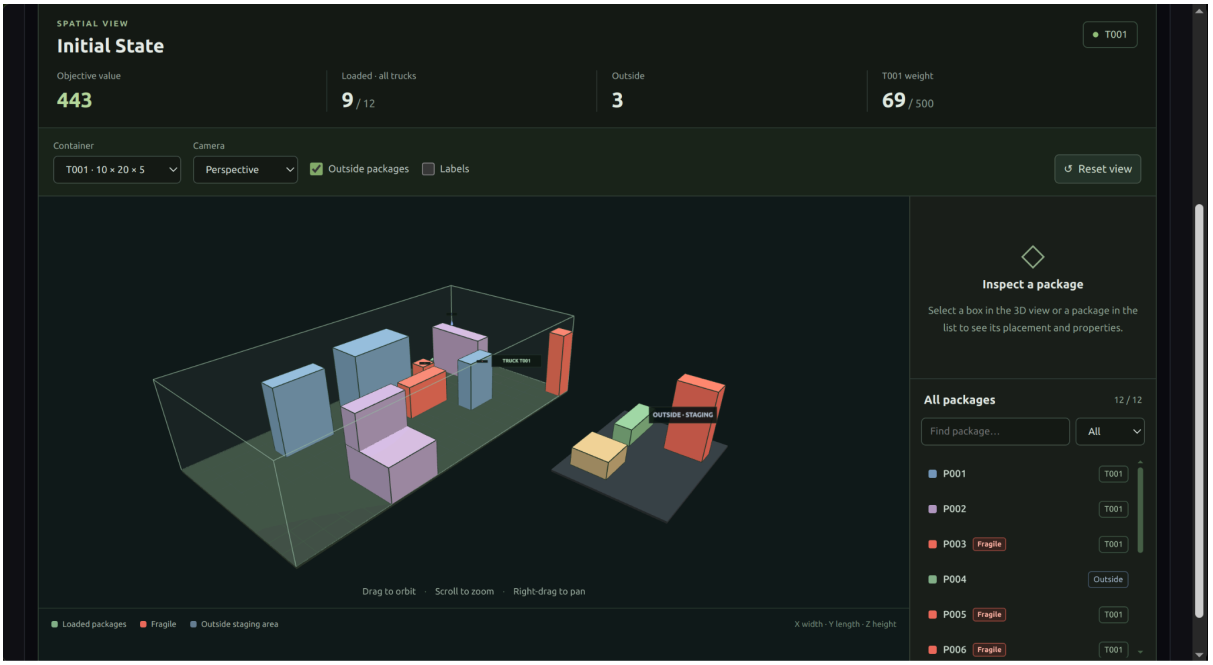

SPATIAL VIEW
Initial State
T001
Objective value
342
Loaded · all trucks
8 / 12
Outside
4
T001 weight
84 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
Outside
P003
Outside
P004
T001
P005
T001
P006
T001
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

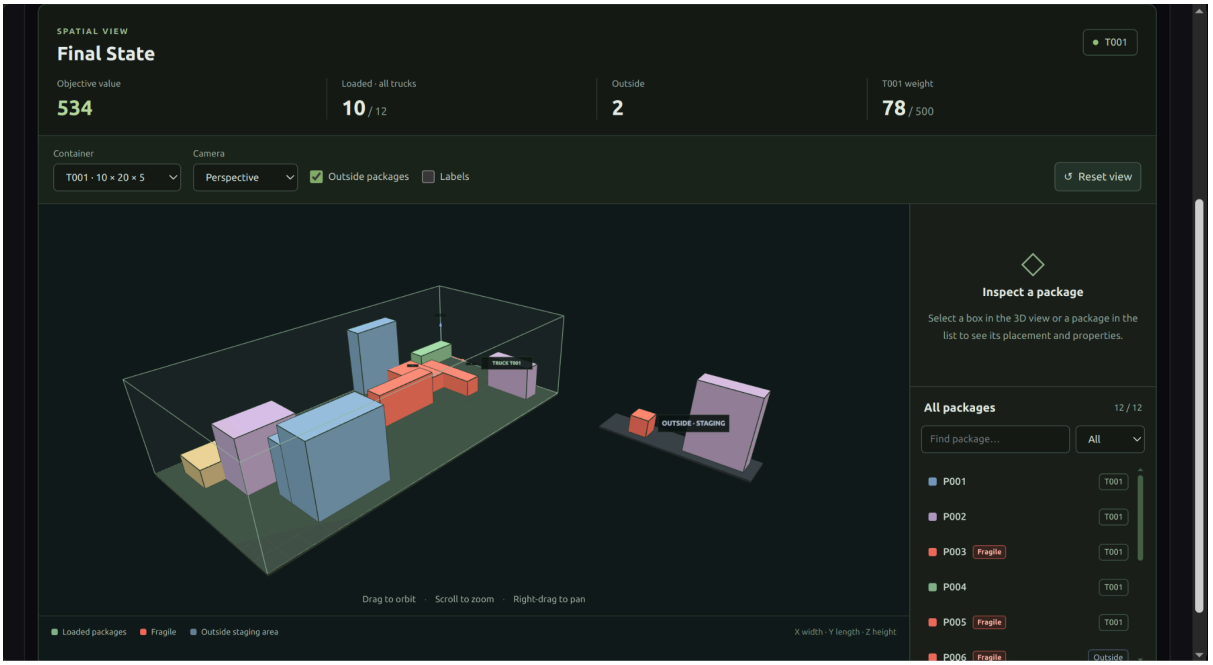

SPATIAL VIEW
Final State
T001
Objective value
484
Loaded · all trucks
10 / 12
Outside
2
T001 weight
108 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 Fragile T001
P003 T001
P004 T001
P005 T001
P006 Outside

Test Case 4 :

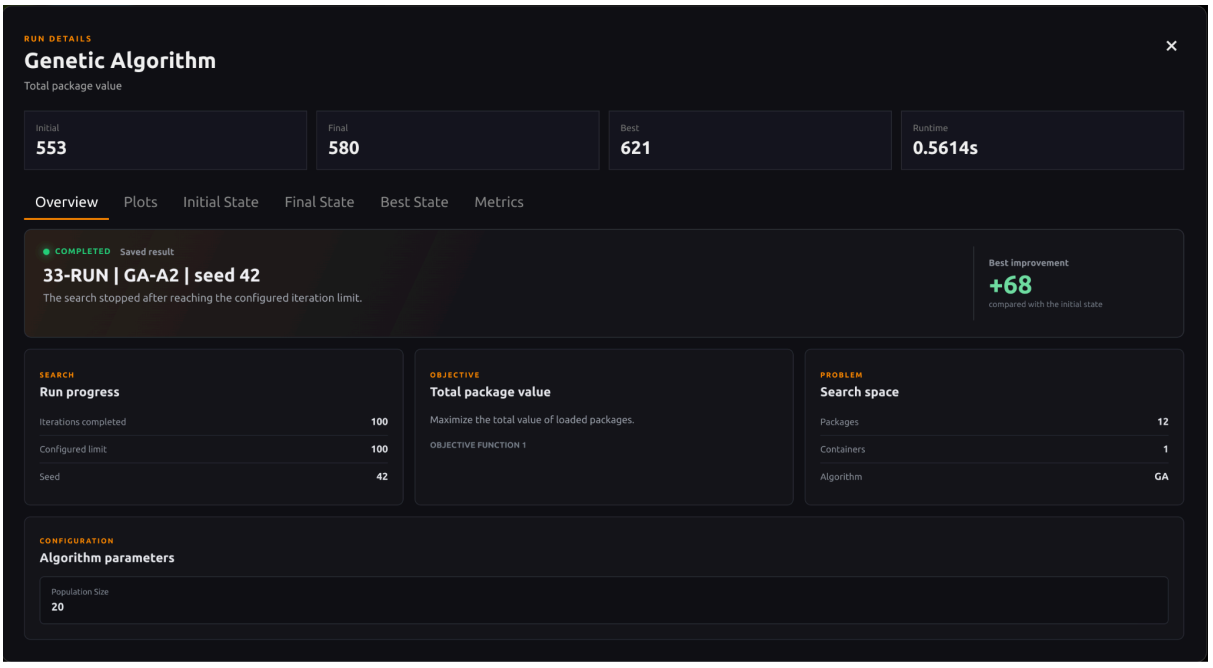

RUN DETAILS
Genetic Algorithm
Total package value
Initial
443
Final
534
Best
589
Runtime
0.4358s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-A2 | seed 41
The search stopped after reaching the configured iteration limit.
Best improvement
+146
compared with the initial state
SEARCH
Run progress
Iterations completed 100
Configured limit 100
Seed 41
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages 12
Containers 1
Algorithm GA
CONFIGURATION
Algorithm parameters
Population Size
20

RUN DETAILS
Genetic Algorithm
Total package value
Initial
443
Final
534
Best
589
Runtime
0.4358s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
589
499.525
410.05
Objective value
0
50
100
Iteration (generation)

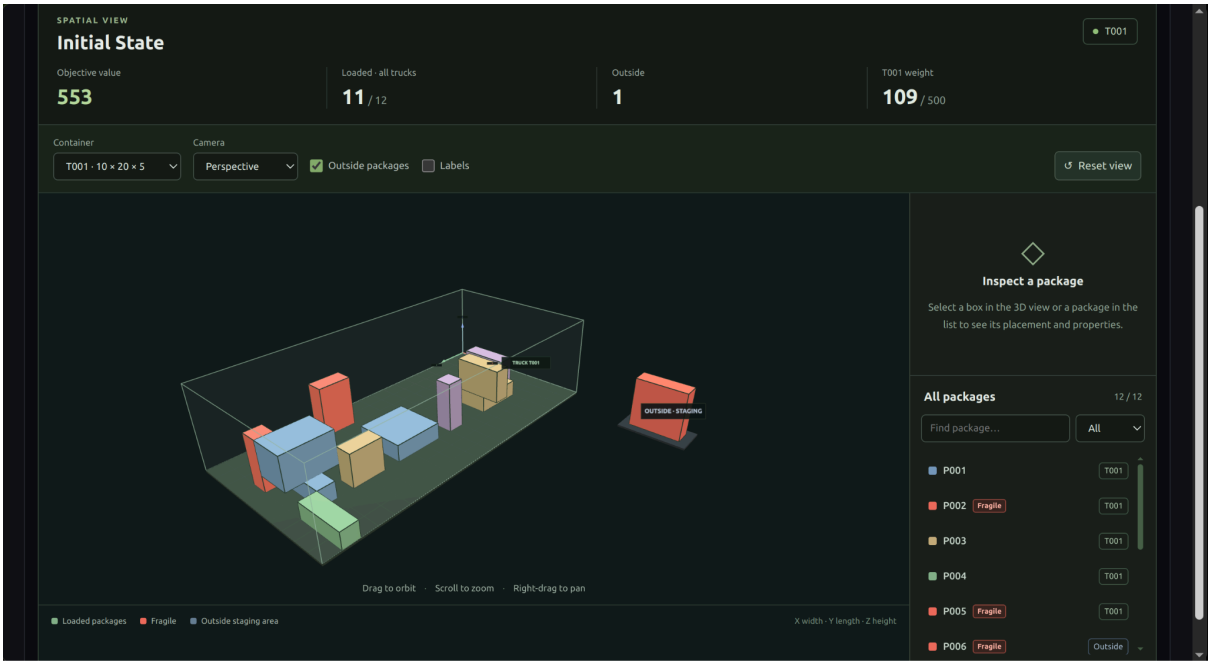

SPATIAL VIEW
Initial State
T001
Objective value
443
Loaded · all trucks
9 / 12
Outside
3
T001 weight
69 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
T001
P003
Fragile
T001
P004
Outside
P005
Fragile
T001
P006
Fragile
T001

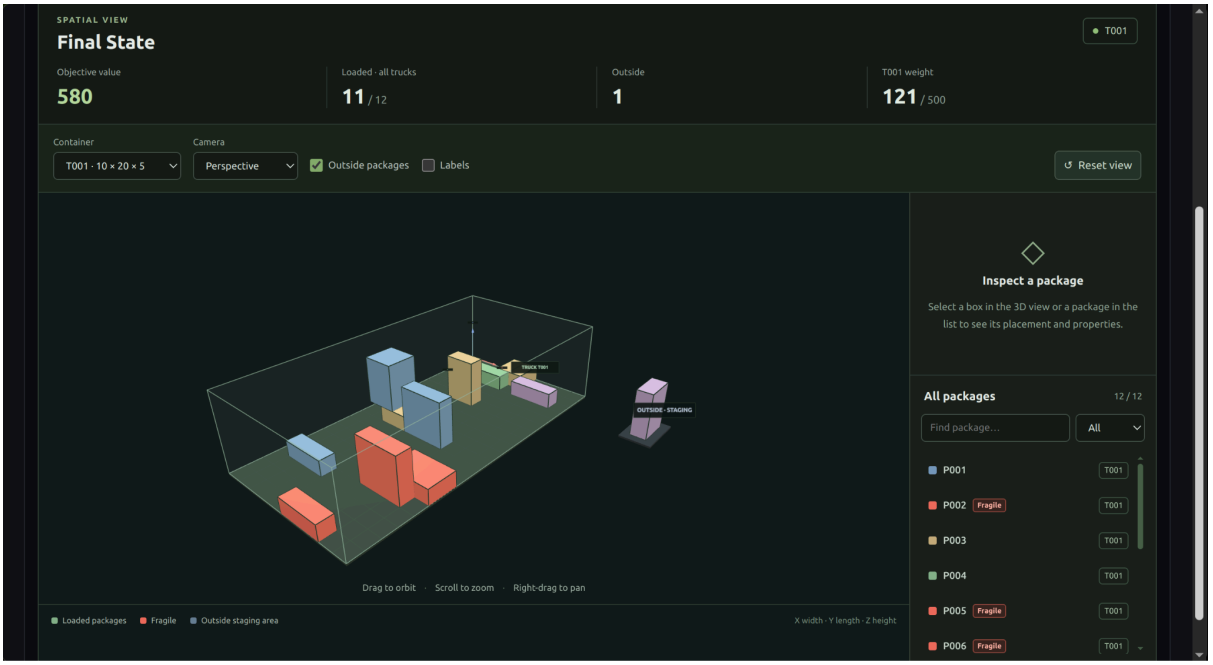

SPATIAL VIEW
Final State
T001
Objective value
534
Loaded · all trucks
10 / 12
Outside
2
T001 weight
78 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
P003
Fragile
P004
P005
P006
Outside

Test Case 5 :

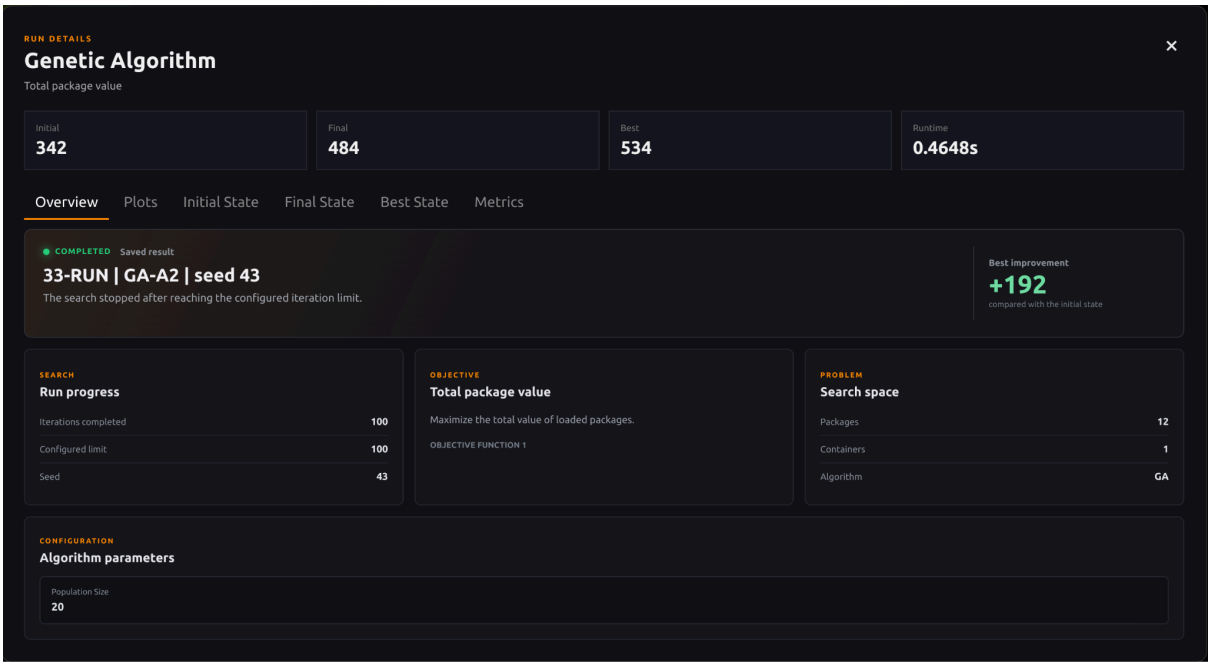

RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
580
Best
621
Runtime
0.5614s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-A2 | seed 42
The search stopped after reaching the configured iteration limit.
Best improvement
+68
compared with the initial state
SEARCH
Run progress
Iterations completed
100
Configured limit
100
Seed
42
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
GA
CONFIGURATION
Algorithm parameters
Population Size
20

RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
580
Best
621
Runtime
0.5614s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
621
535.5
450
Objective value
0
50
100
Iteration (generation)

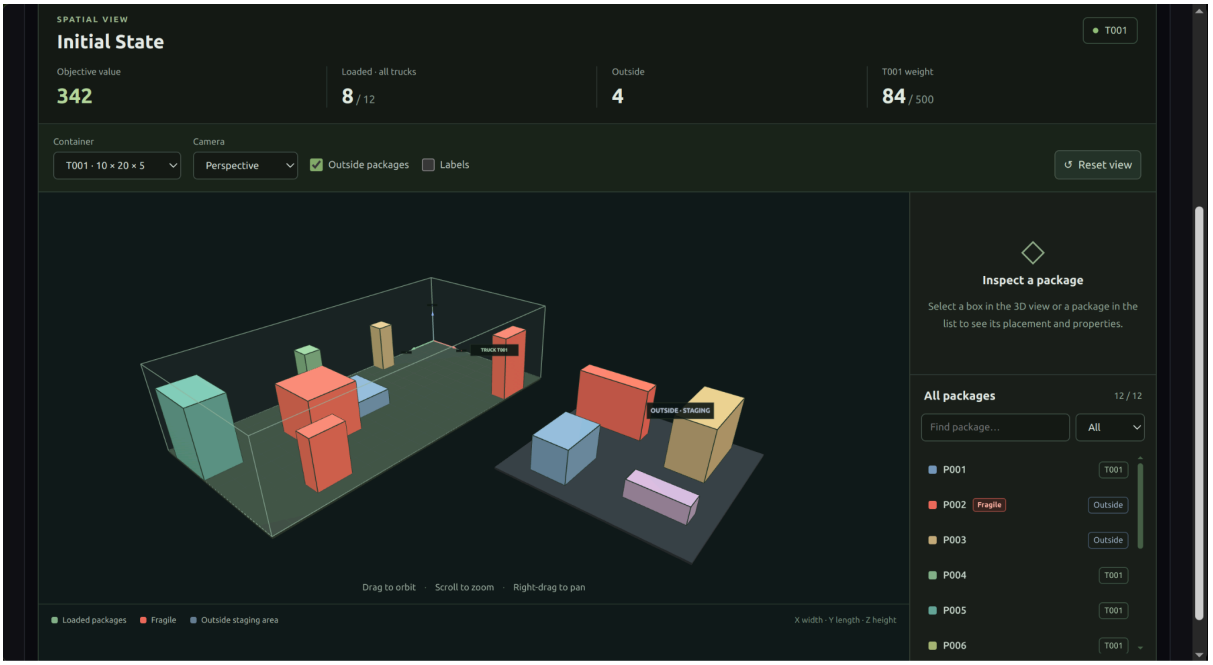

SPATIAL VIEW
Initial State
T001
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Fragile
P006
Fragile
Outside
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

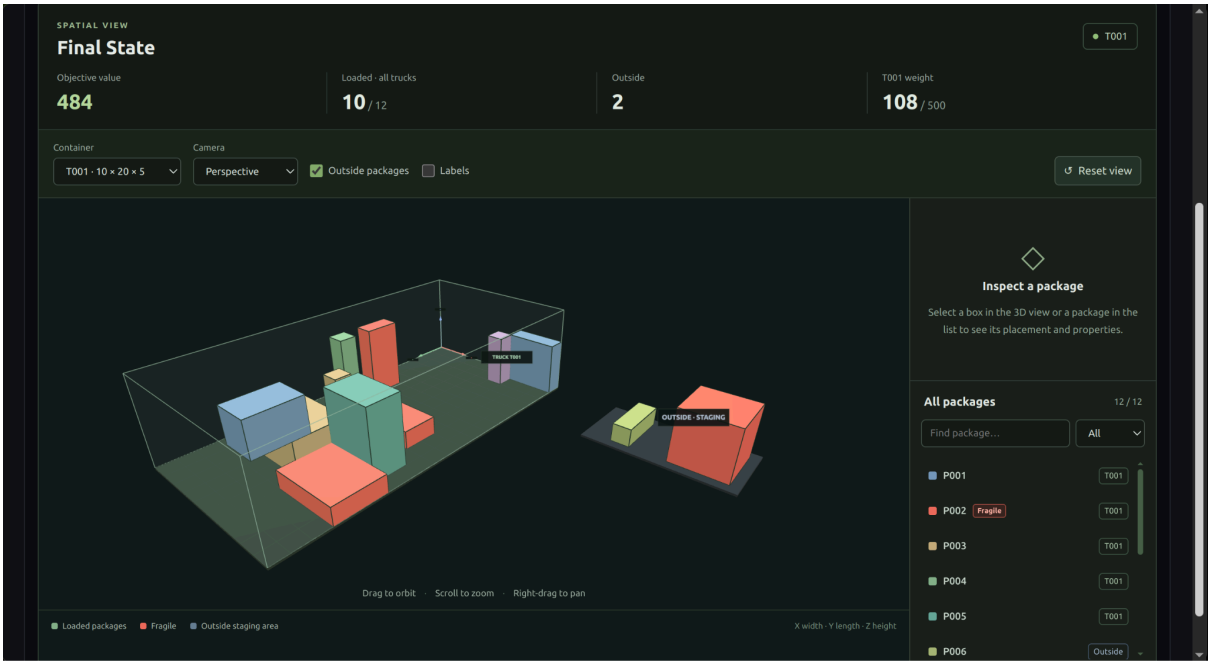

SPATIAL VIEW
Final State
T001
Objective value
580
Loaded · all trucks
11 / 12
Outside
1
T001 weight
121 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Fragile
P006
Fragile
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

Test Case 6 :

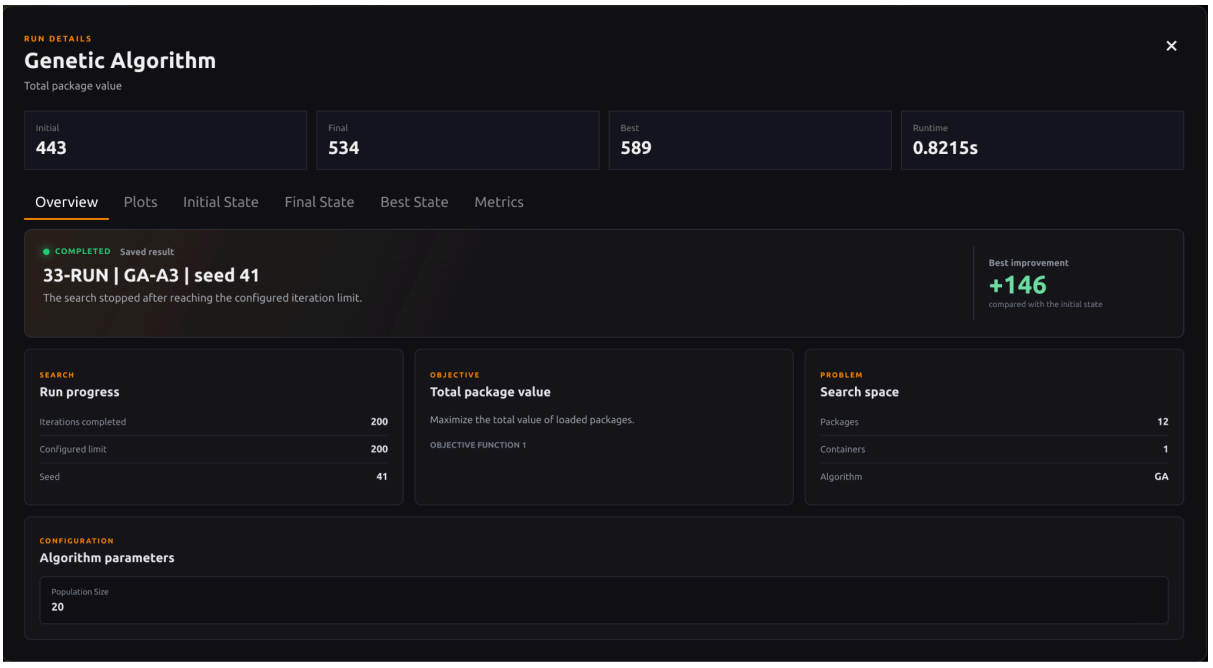

RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
484
Best
534
Runtime
0.4648s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-A2 | seed 43
The search stopped after reaching the configured iteration limit.
Best improvement
+192
compared with the initial state
SEARCH
Run progress
Iterations completed
100
Configured limit
100
Seed
43
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
GA
CONFIGURATION
Algorithm parameters
Population Size
20

RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
484
Best
534
Runtime
0.4648s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
534
439.1
344.2
Objective value
0
50
100
Iteration (generation)

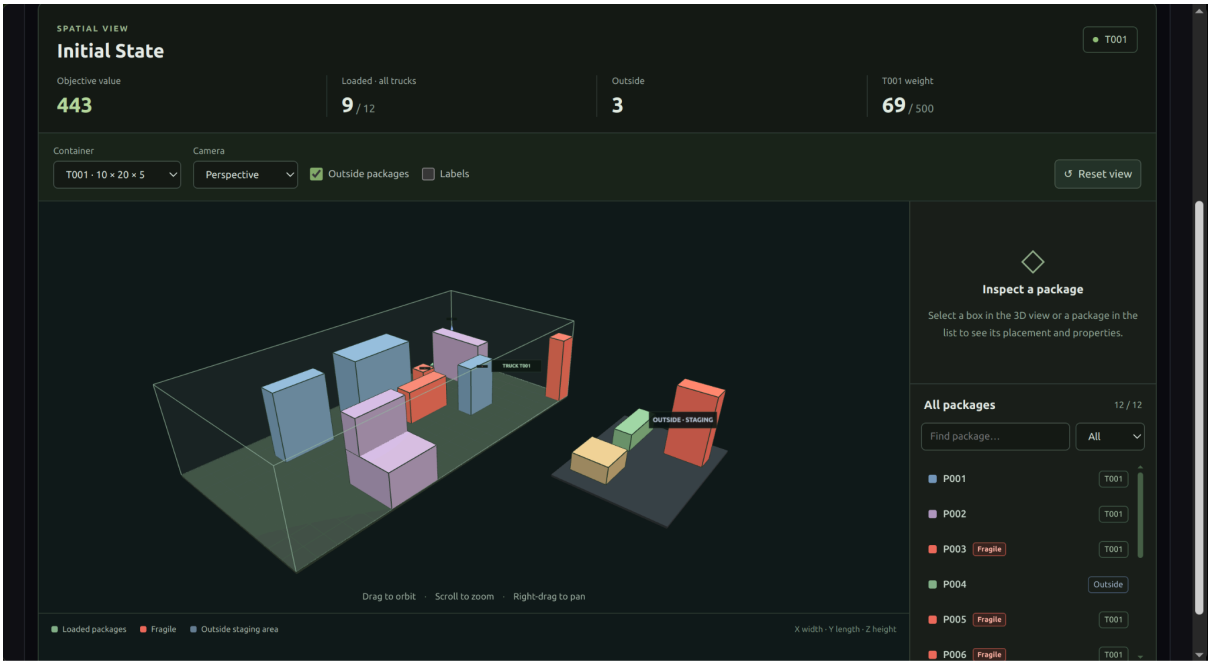

SPATIAL VIEW
Initial State
T001
Objective value
342
Loaded · all trucks
8 / 12
Outside
4
T001 weight
84 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
Outside
P003
Outside
P004
T001
P005
T001
P006
T001

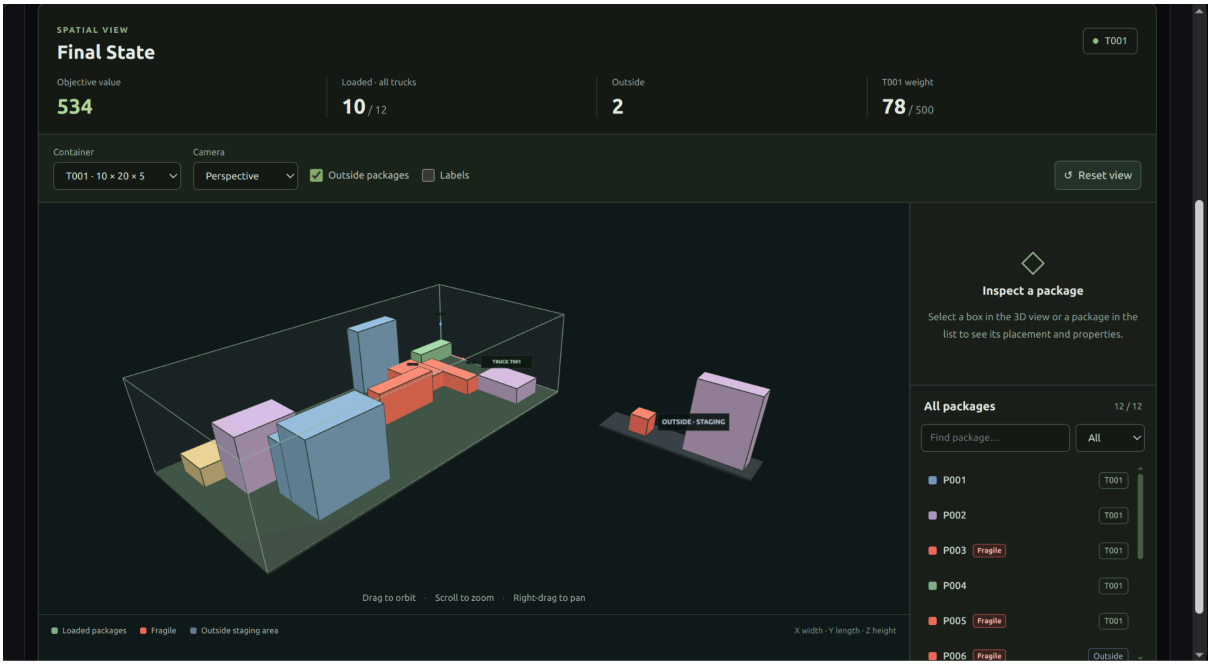

SPATIAL VIEW
Final State
T001
Objective value
484
Loaded · all trucks
10 / 12
Outside
2
T001 weight
108 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 Fragile T001
P003 T001
P004 T001
P005 T001
P006 Outside

Test Case 7 :

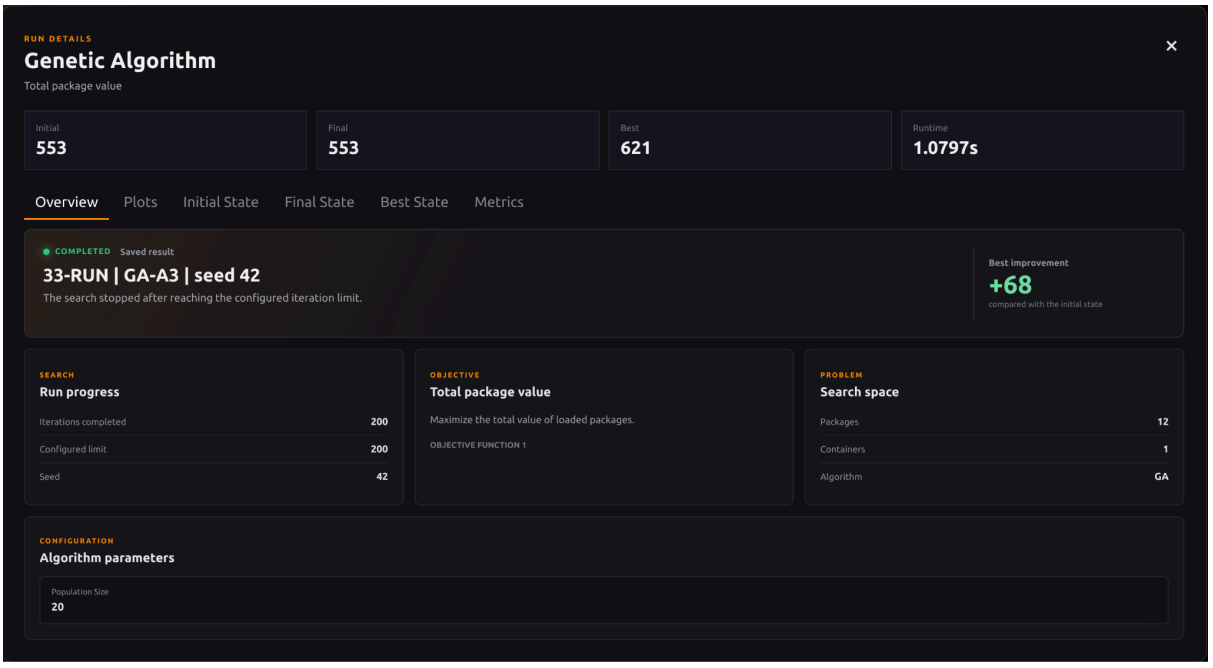

RUN DETAILS
Genetic Algorithm
Total package value
Initial
443
Final
534
Best
589
Runtime
0.8215s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-A3 | seed 41
The search stopped after reaching the configured iteration limit.
Best improvement
+146
compared with the initial state
SEARCH
Run progress
Iterations completed 200
Configured limit 200
Seed 41
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages 12
Containers 1
Algorithm GA
CONFIGURATION
Algorithm parameters
Population Size
20

RUN DETAILS
Genetic Algorithm
Total package value
Initial
443
Final
534
Best
589
Runtime
0.8215s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
589
499.525
410.05
Objective value
0
100
200
Iteration (generation)

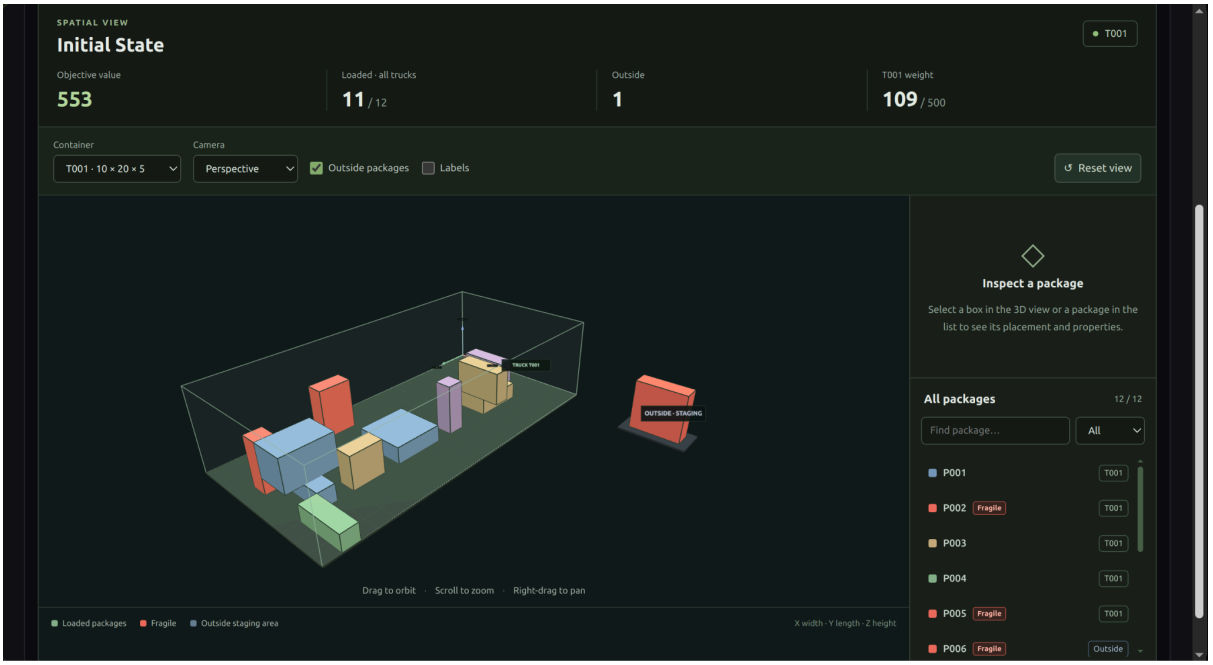

SPATIAL VIEW
Initial State
T001
Objective value
443
Loaded · all trucks
9 / 12
Outside
3
T001 weight
69 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
T001
P003
Fragile
T001
P004
Outside
P005
Fragile
T001
P006
Fragile
T001

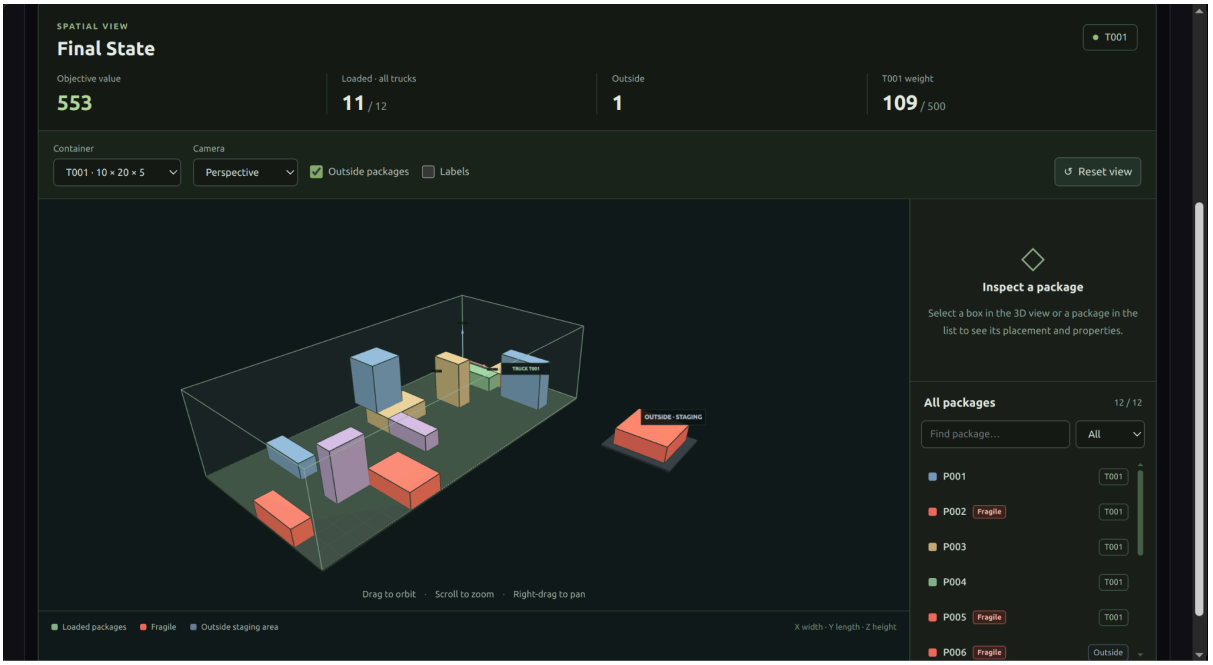

SPATIAL VIEW
Final State
T001
Objective value
534
Loaded · all trucks
10 / 12
Outside
2
T001 weight
78 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 T001
P003 Fragile T001
P004 T001
P005 Fragile T001
P006 Fragile Outside

Test Case 8 :

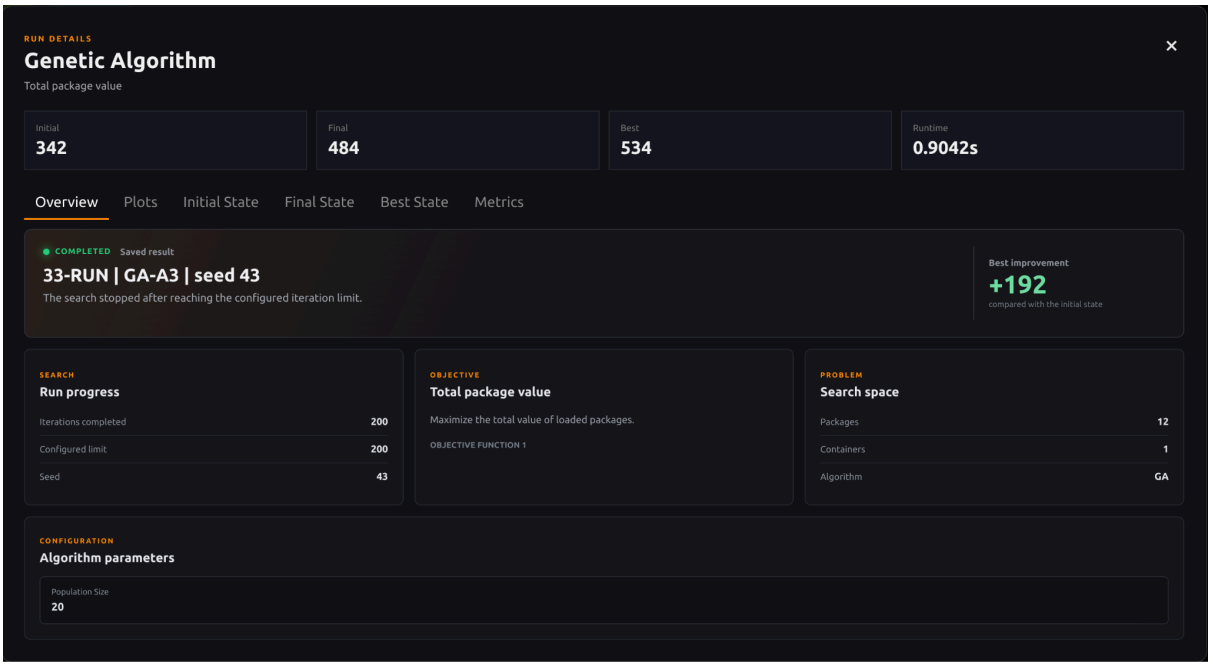

RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
553
Best
621
Runtime
1.0797s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-A3 | seed 42
The search stopped after reaching the configured iteration limit.
Best improvement
+68
compared with the initial state
SEARCH
Run progress
Iterations completed 200
Configured limit 200
Seed 42
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages 12
Containers 1
Algorithm GA
CONFIGURATION
Algorithm parameters
Population Size
20

RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
553
Best
621
Runtime
1.0797s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
621
535.5
450
Objective value
0
100
200
Iteration (generation)

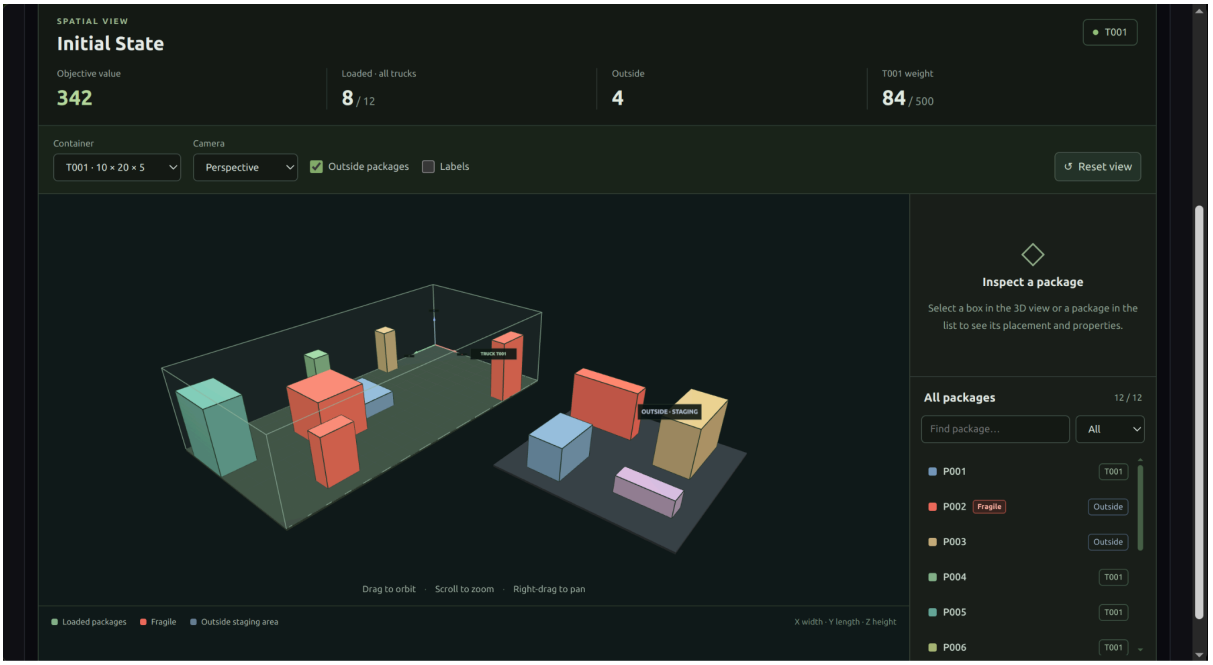

SPATIAL VIEW
Initial State
T001
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Fragile
P006
Fragile
Outside
Drag to orbit
Scroll to zoom
Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

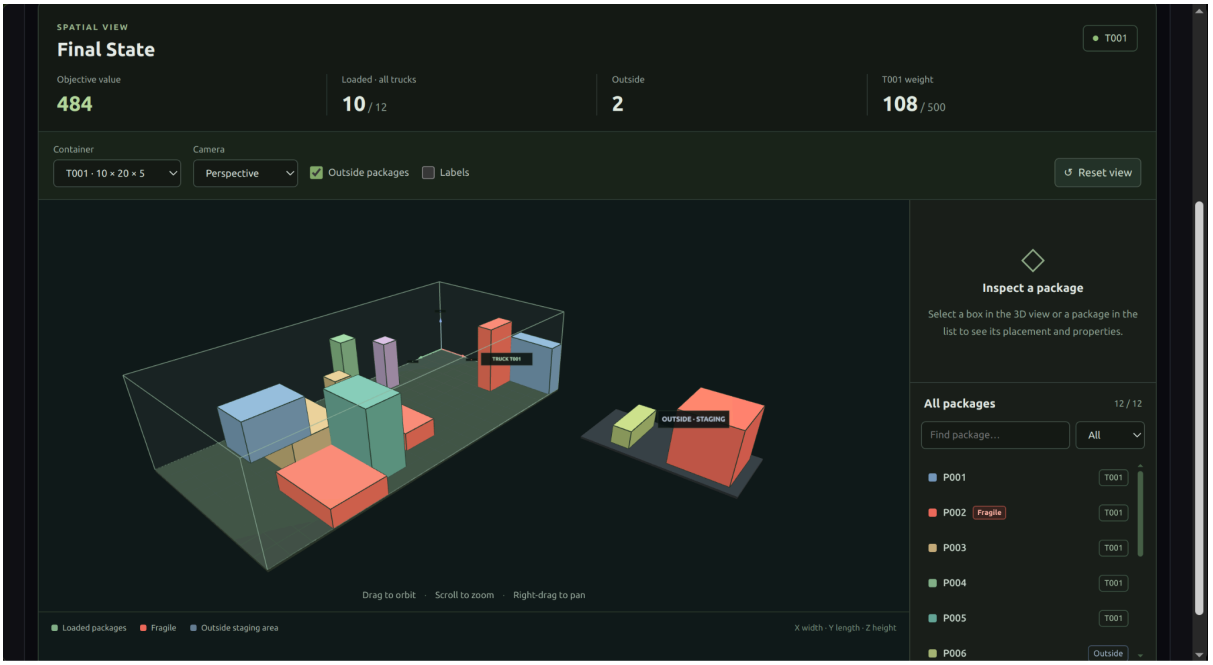

SPATIAL VIEW
Final State
T001
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 Fragile T001
P003 T001
P004 T001
P005 Fragile T001
P006 Fragile Outside

Test Case 9 :


RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
484
Best
534
Runtime
0.9042s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-A3 | seed 43
The search stopped after reaching the configured iteration limit.
Best improvement
+192
compared with the initial state
SEARCH
Run progress
Iterations completed 200
Configured limit 200
Seed 43
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages 12
Containers 1
Algorithm GA
CONFIGURATION
Algorithm parameters
Population Size
20

RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
484
Best
534
Runtime
0.9042s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
534
439.1
344.2
Objective value
0
100
200
Iteration (generation)


SPATIAL VIEW
Initial State
T001
Objective value
342
Loaded · all trucks
8 / 12
Outside
4
T001 weight
84 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
Outside
P003
Outside
P004
T001
P005
T001
P006
T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height



3.3.2. Banyak Iterasi sebagai Kontrol

3.3.2.1. Parameter Input

Berikut adalah parameter input yang digunakan saat menjalankan eksperimen :

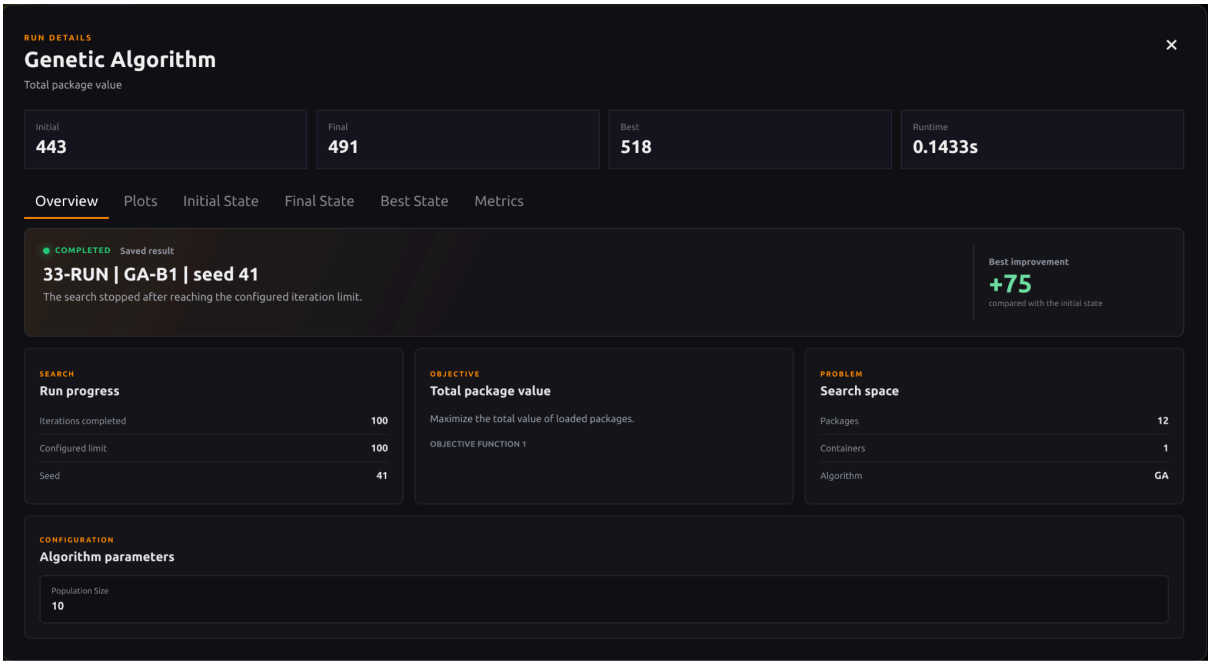
Test case	Objective function	Package count	Truck count	Maximum iterations	Seed	Population size
1	Total package value	12	1	100	41	10
2	Total package value	12	1	100	42	10
3	Total package value	12	1	100	43	10
4	Total package value	12	1	100	41	30
5	Total package value	12	1	100	42	30
6	Total package value	12	1	100	43	30
7	Total	12	1	100	41	40

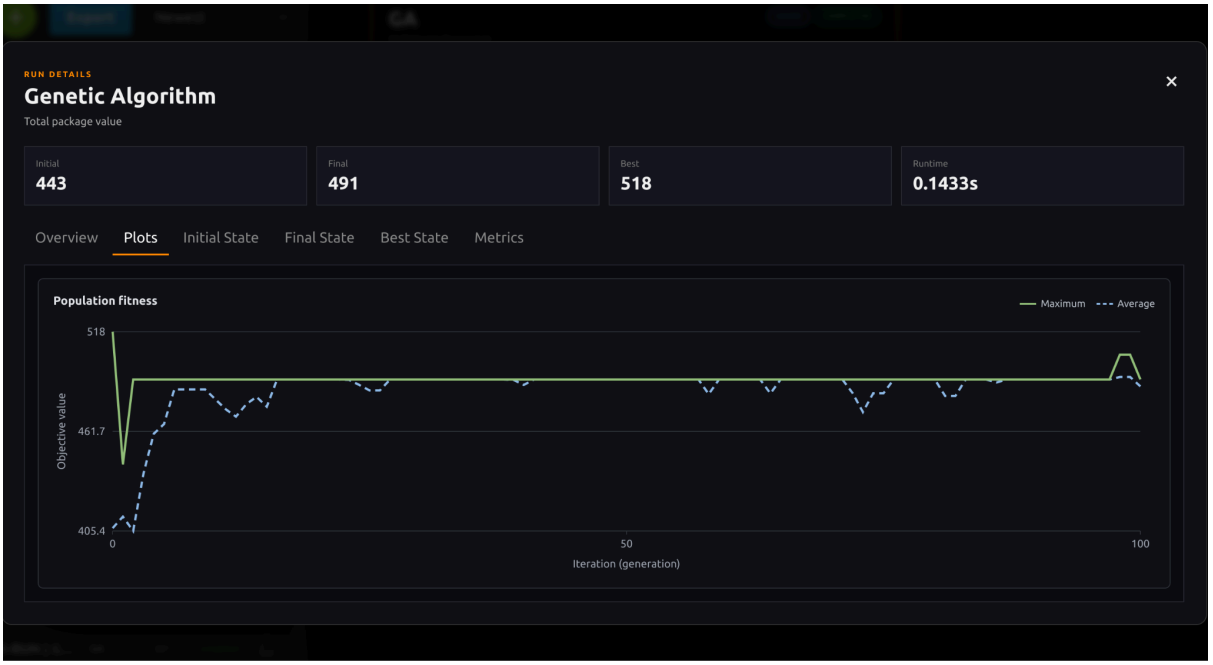
	package value					
8	Total package value	12	1	100	42	40
9	Total package value	12	1	100	43	40

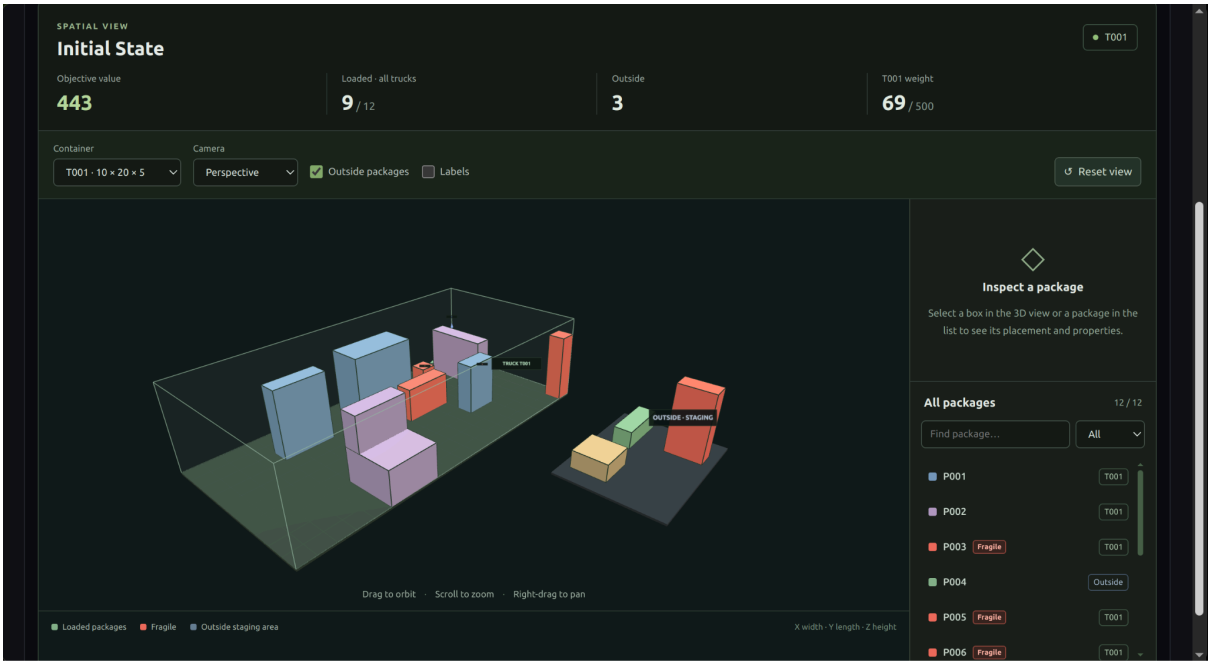
3.3.2.2. Hasil Eksperimen

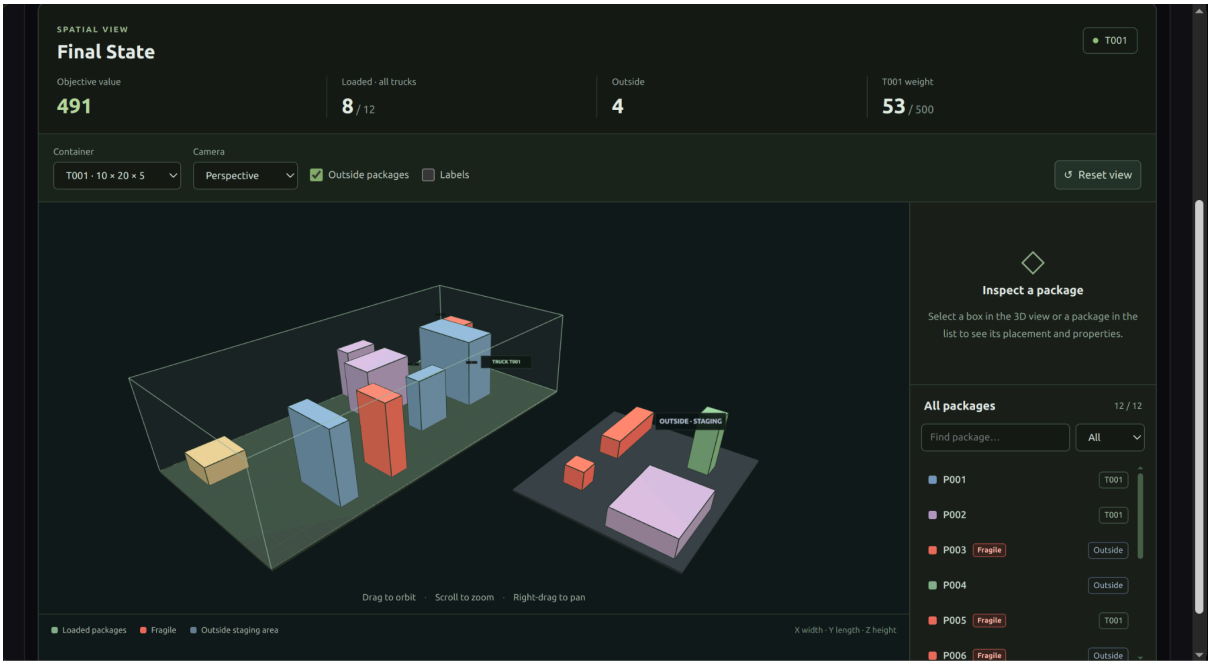
Berikut adalah hasil eksperimen untuk setiap test case :

Test Case 1 :

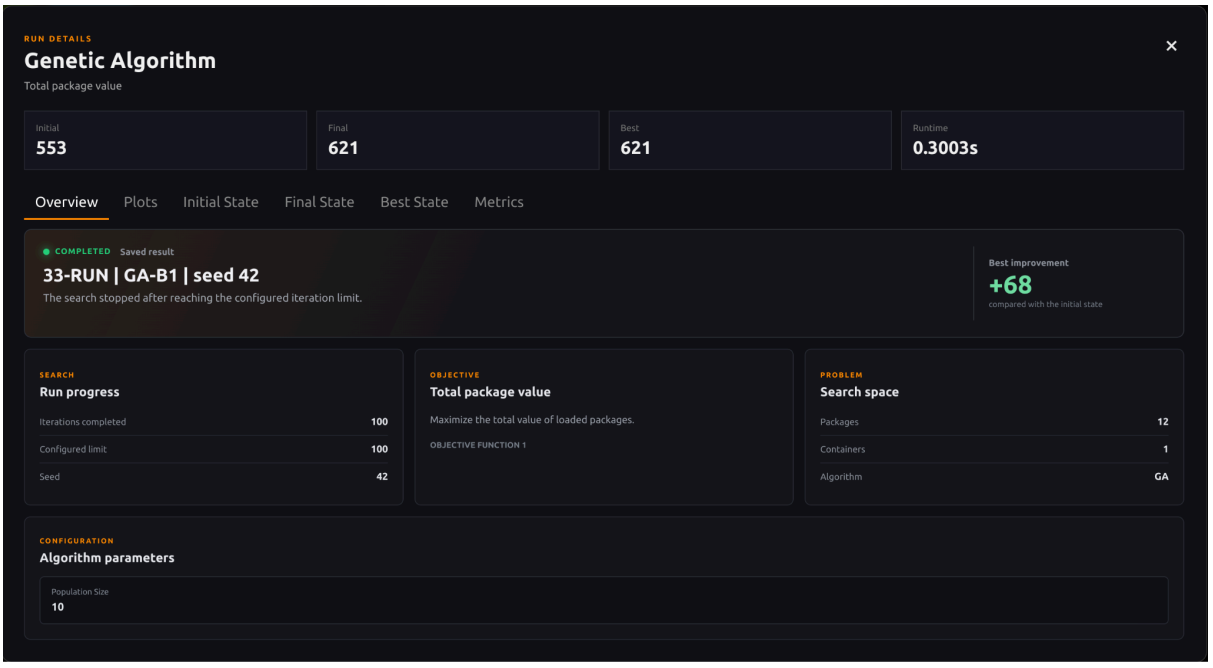



RUN DETAILS
Genetic Algorithm
Total package value
Initial
443
Final
491
Best
518
Runtime
0.1433s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
518
461.7
405.4
Objective value
0
50
100
Iteration (generation)

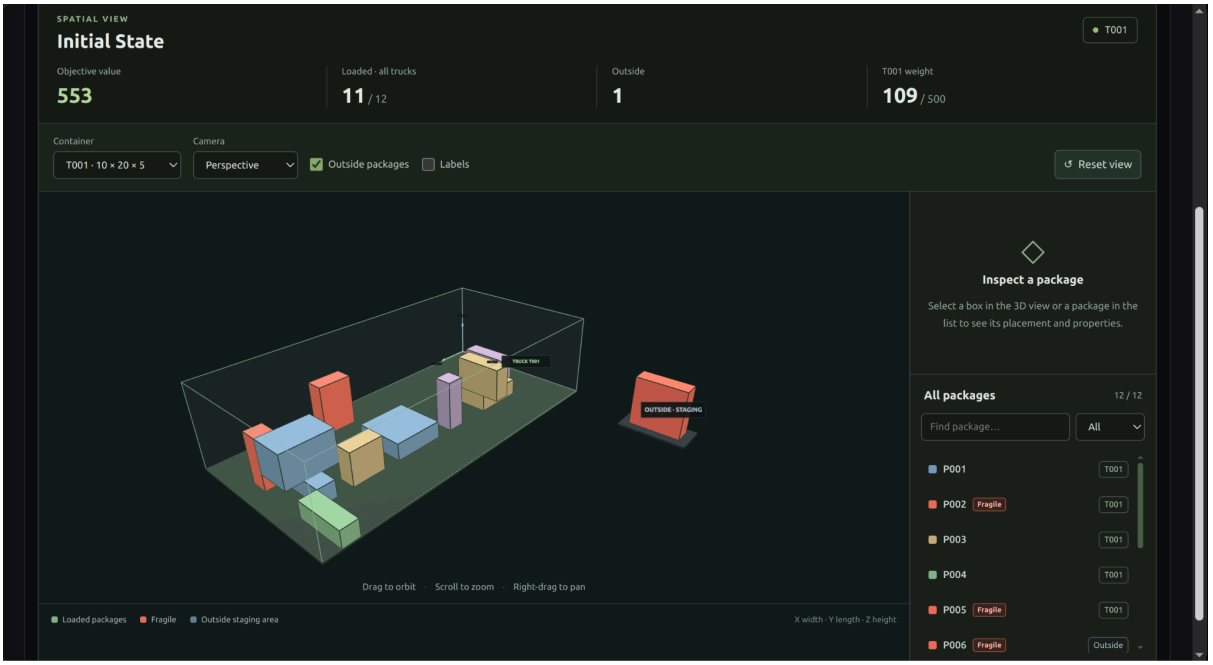

SPATIAL VIEW
Initial State
T001
Objective value
443
Loaded · all trucks
9 / 12
Outside
3
T001 weight
69 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
T001
P003
Fragile
T001
P004
Outside
P005
Fragile
T001
P006
Fragile
T001

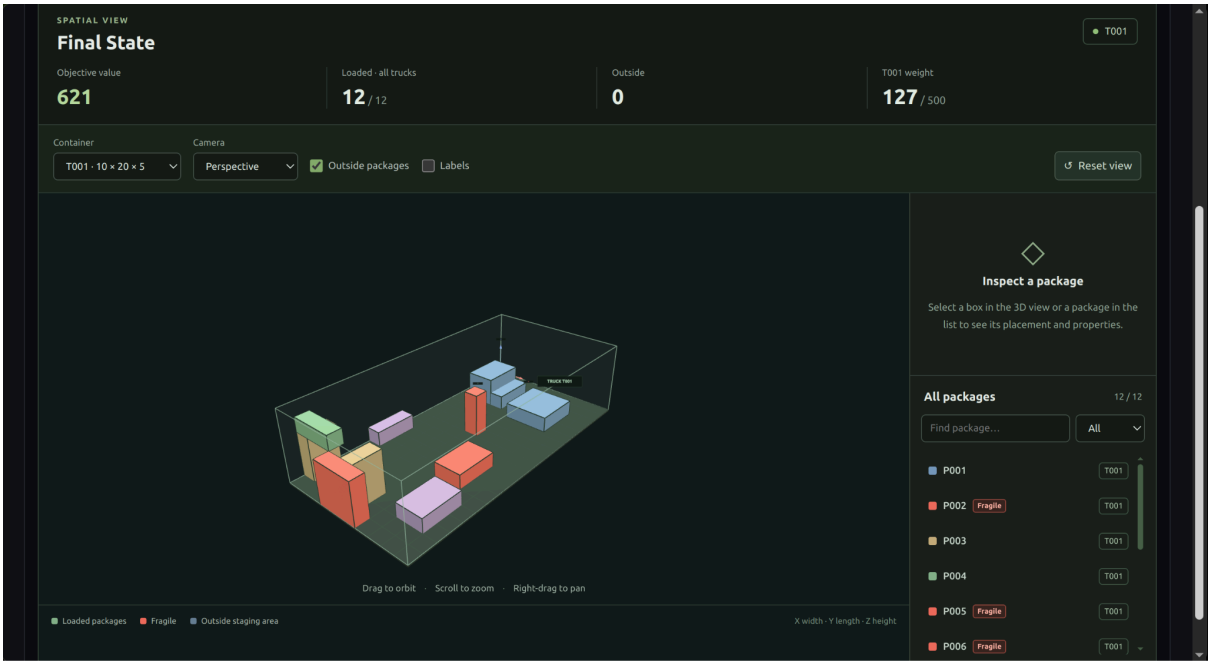

SPATIAL VIEW
Final State
T001
Objective value
491
Loaded · all trucks
8 / 12
Outside
4
T001 weight
53 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001 T001
P002 T001
P003 Fragile Outside
P004 Outside
P005 Fragile T001
P006 Fragile Outside

Test Case 2 :

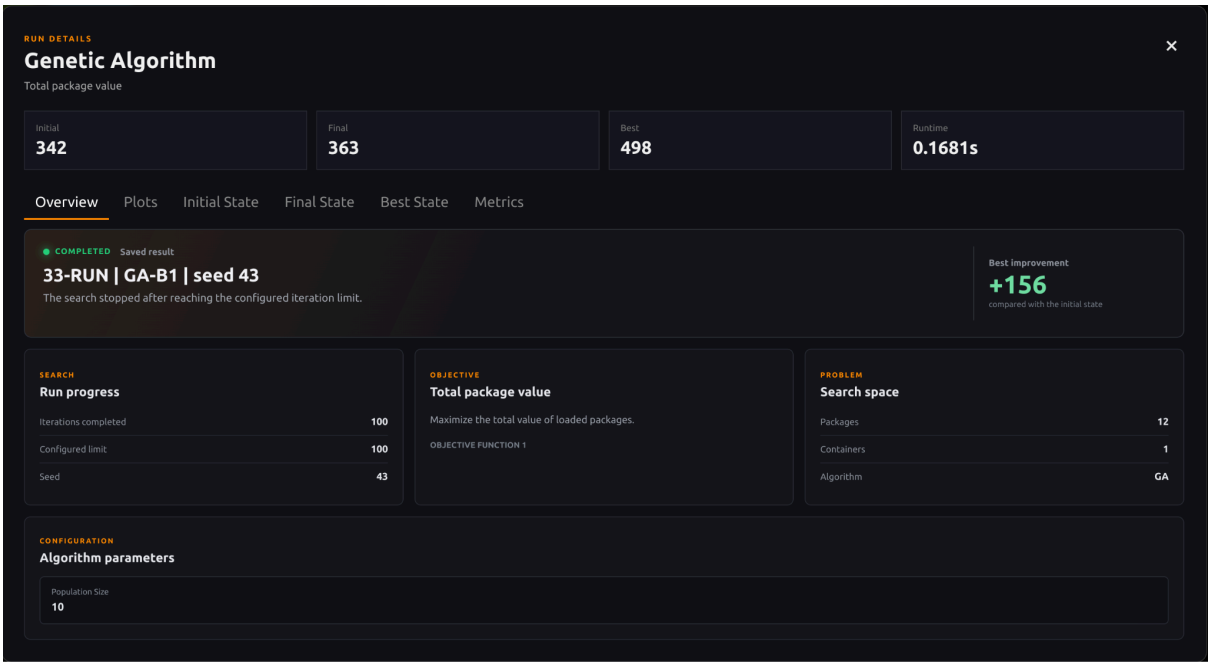

RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
621
Best
621
Runtime
0.3003s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-B1 | seed 42
The search stopped after reaching the configured iteration limit.
Best improvement
+68
compared with the initial state
SEARCH
Run progress
Iterations completed 100
Configured limit 100
Seed 42
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages 12
Containers 1
Algorithm GA
CONFIGURATION
Algorithm parameters
Population Size
10

RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
621
Best
621
Runtime
0.3003s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
621
536.5
452
Objective value
0
50
100
Iteration (generation)

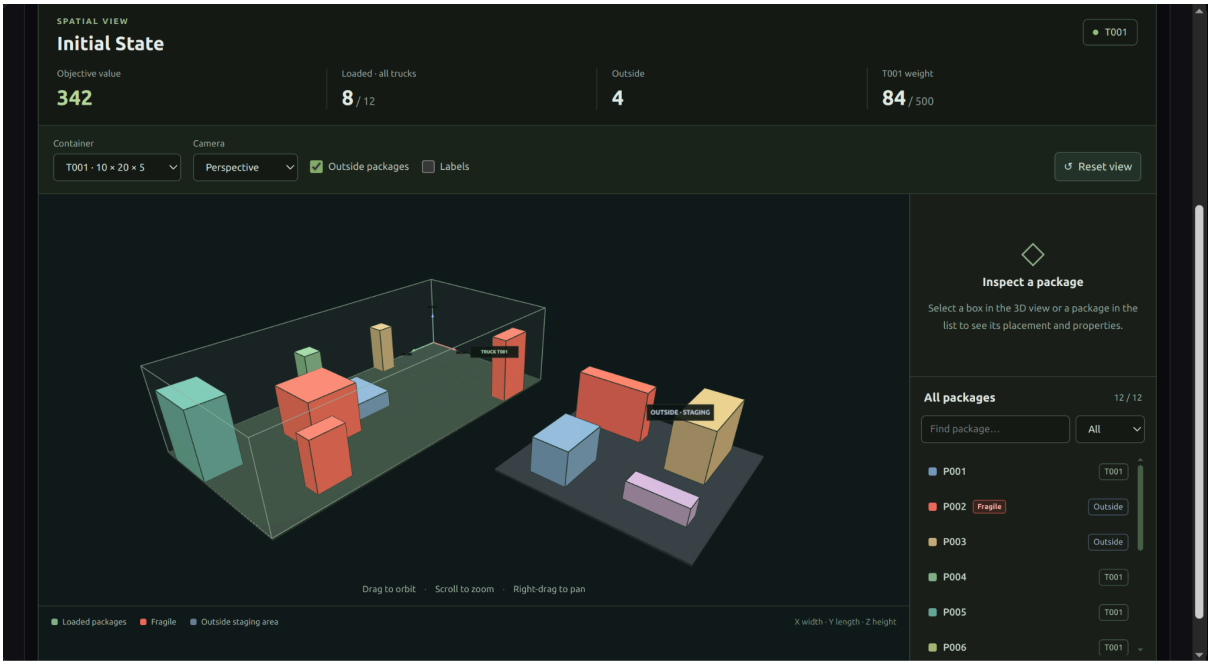

SPATIAL VIEW
Initial State
T001
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Fragile
P006
Fragile
Outside
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

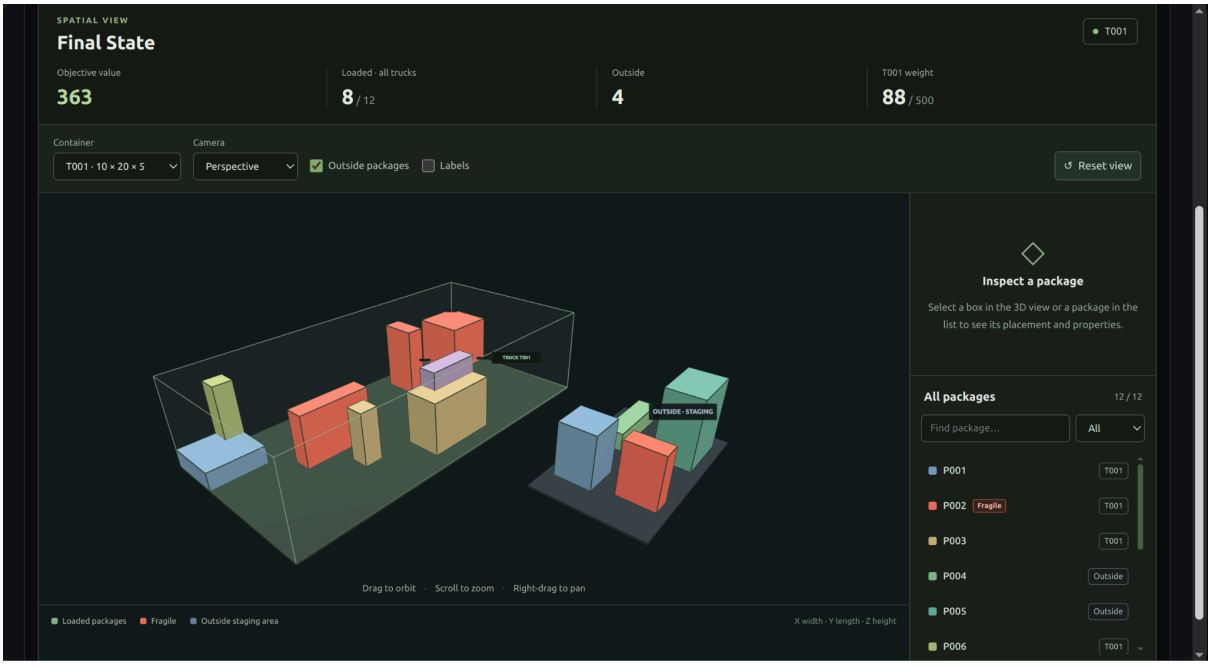

SPATIAL VIEW
Final State
T001
Objective value
621
Loaded · all trucks
12 / 12
Outside
0
T001 weight
127 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Fragile
P006
Fragile
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

Test Case 3 :

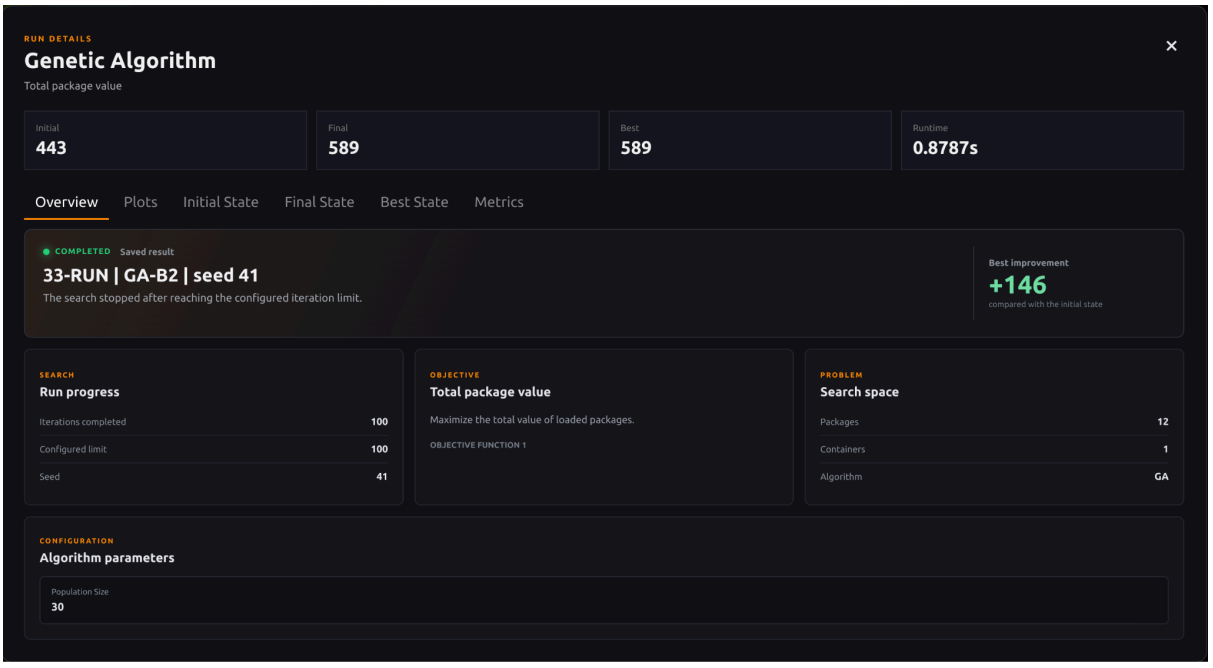

RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
363
Best
498
Runtime
0.1681s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-B1 | seed 43
The search stopped after reaching the configured iteration limit.
Best improvement
+156
compared with the initial state
SEARCH
Run progress
Iterations completed
100
Configured limit
100
Seed
43
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
GA
CONFIGURATION
Algorithm parameters
Population Size
10

RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
363
Best
498
Runtime
0.1681s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
498
407.5
317
Objective value
0
50
100
Iteration (generation)

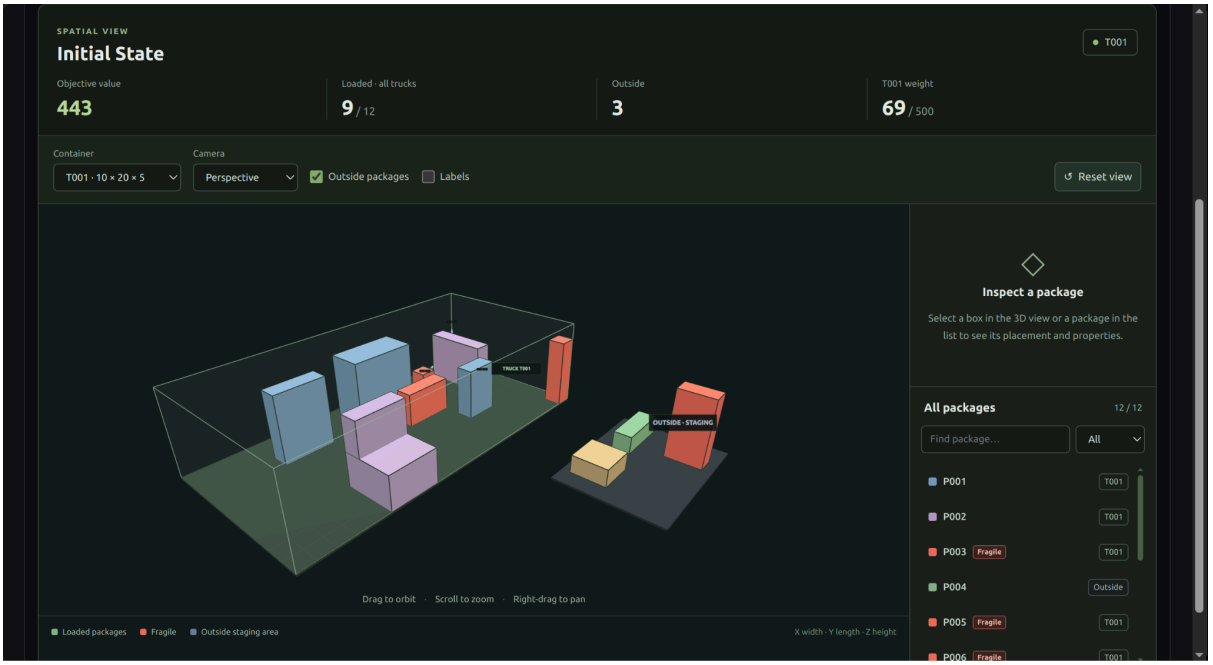

SPATIAL VIEW
Initial State
T001
Objective value
342
Loaded · all trucks
8 / 12
Outside
4
T001 weight
84 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
Outside
P003
Outside
P004
T001
P005
T001
P006
T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

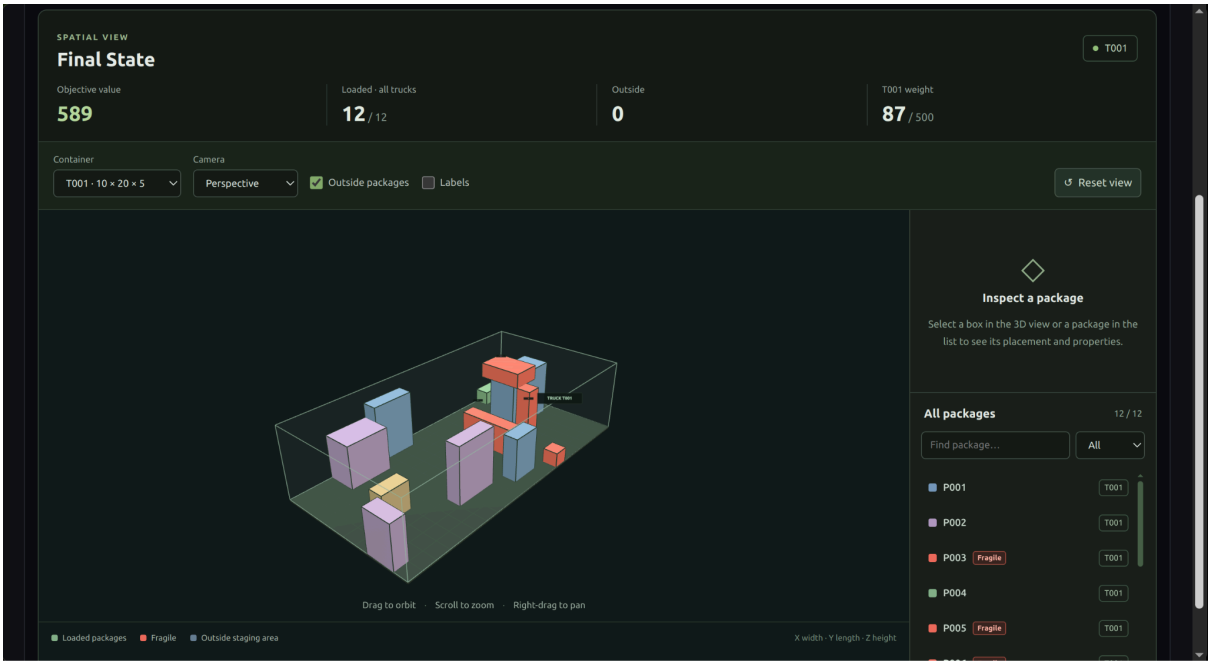

SPATIAL VIEW
Final State
T001
Objective value
363
Loaded · all trucks
8 / 12
Outside
4
T001 weight
88 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE - STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package...
All
P001 T001
P002 Fragile T001
P003 T001
P004 Outside
P005 Outside
P006 T001

Test Case 4 :

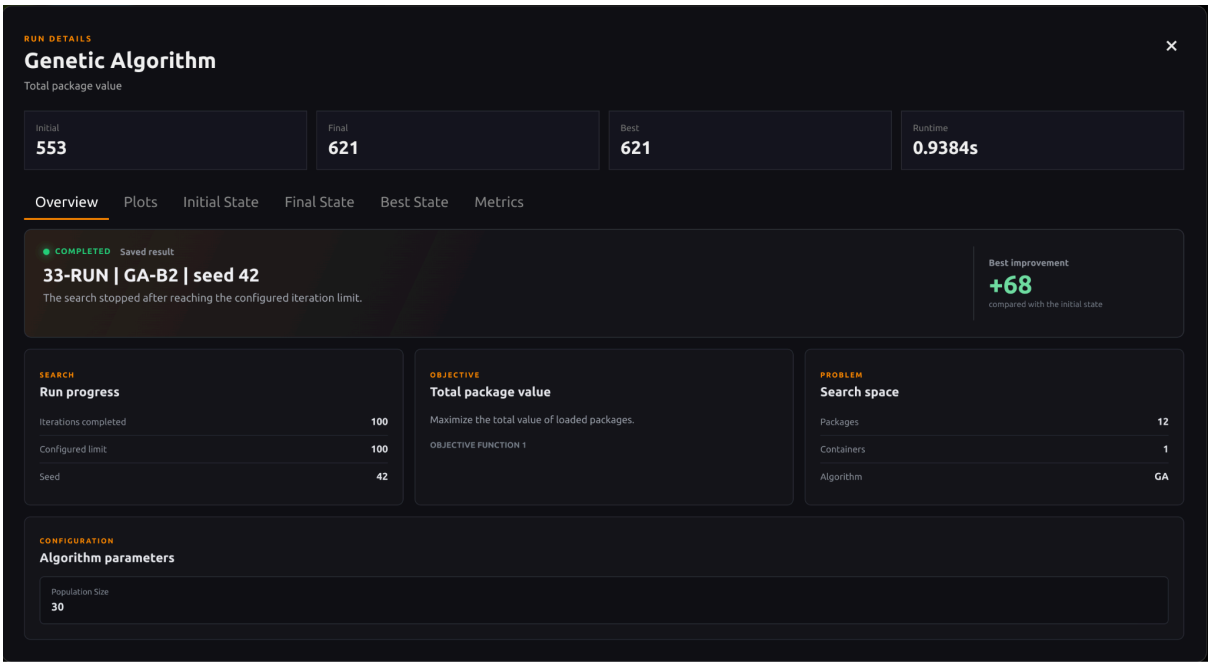

RUN DETAILS
Genetic Algorithm
Total package value
Initial
443
Final
589
Best
589
Runtime
0.8787s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-B2 | seed 41
The search stopped after reaching the configured iteration limit.
Best improvement
+146
compared with the initial state
SEARCH
Run progress
Iterations completed 100
Configured limit 100
Seed 41
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages 12
Containers 1
Algorithm GA
CONFIGURATION
Algorithm parameters
Population Size
30

RUN DETAILS
Genetic Algorithm
Total package value
Initial
443
Final
589
Best
589
Runtime
0.8787s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
589
501.683
414.367
Objective value
0
50
100
Iteration (generation)

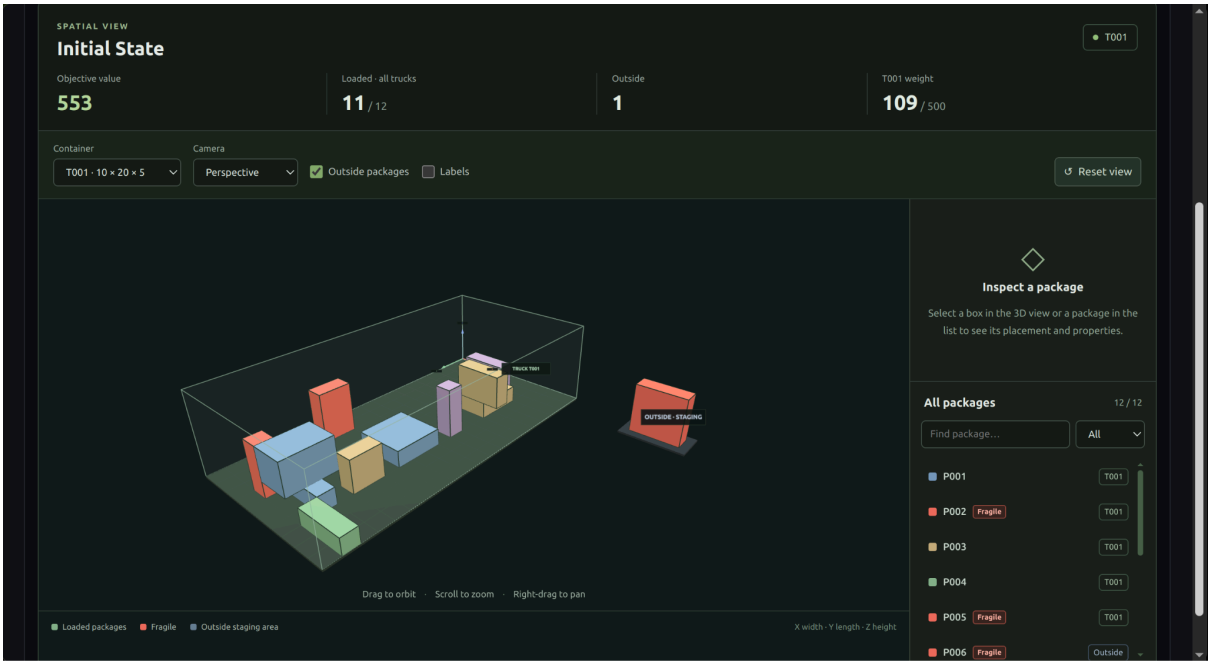

SPATIAL VIEW
Initial State
T001
Objective value
443
Loaded · all trucks
9 / 12
Outside
3
T001 weight
69 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
T001
P003
Fragile
T001
P004
Outside
P005
Fragile
T001
P006
Fragile
T001

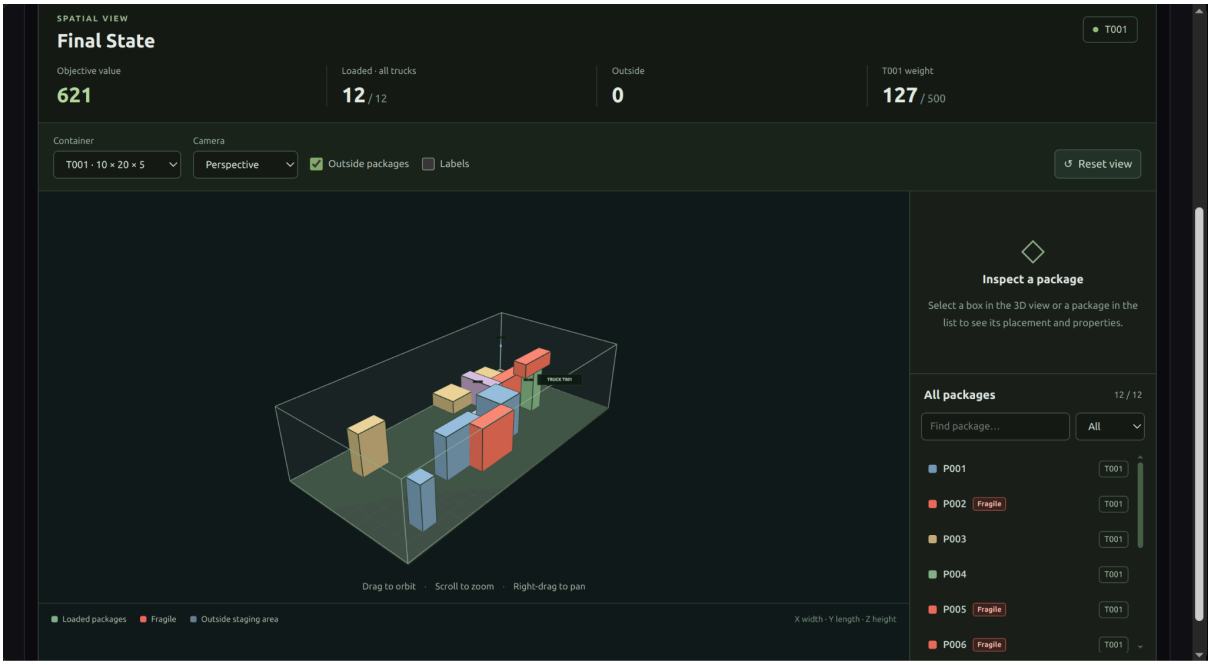

SPATIAL VIEW
Final State
T001
Objective value
589
Loaded · all trucks
12 / 12
Outside
0
T001 weight
87 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
P003
Fragile
P004
P005
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

Test Case 5 :

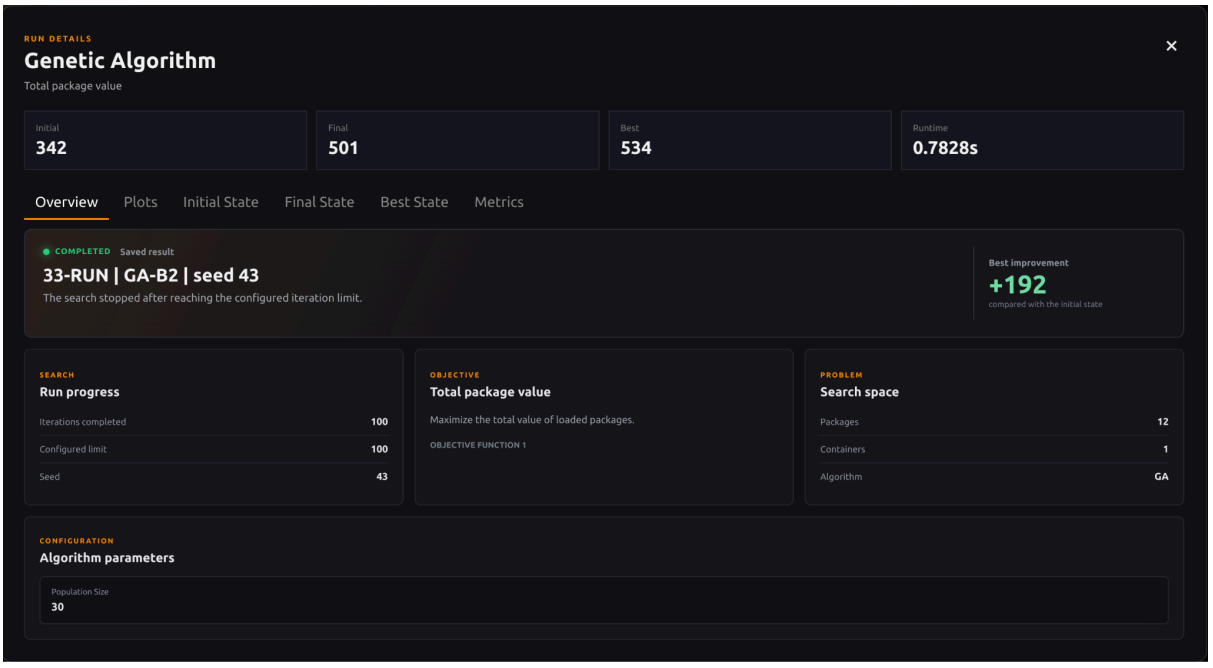

RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
621
Best
621
Runtime
0.9384s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-B2 | seed 42
The search stopped after reaching the configured iteration limit.
Best improvement
+68
compared with the initial state
SEARCH
Run progress
Iterations completed
100
Configured limit
100
Seed
42
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
GA
CONFIGURATION
Algorithm parameters
Population Size
30

RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
621
Best
621
Runtime
0.9384s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
621
531.783
442.567
Objective value
0
50
100
Iteration (generation)

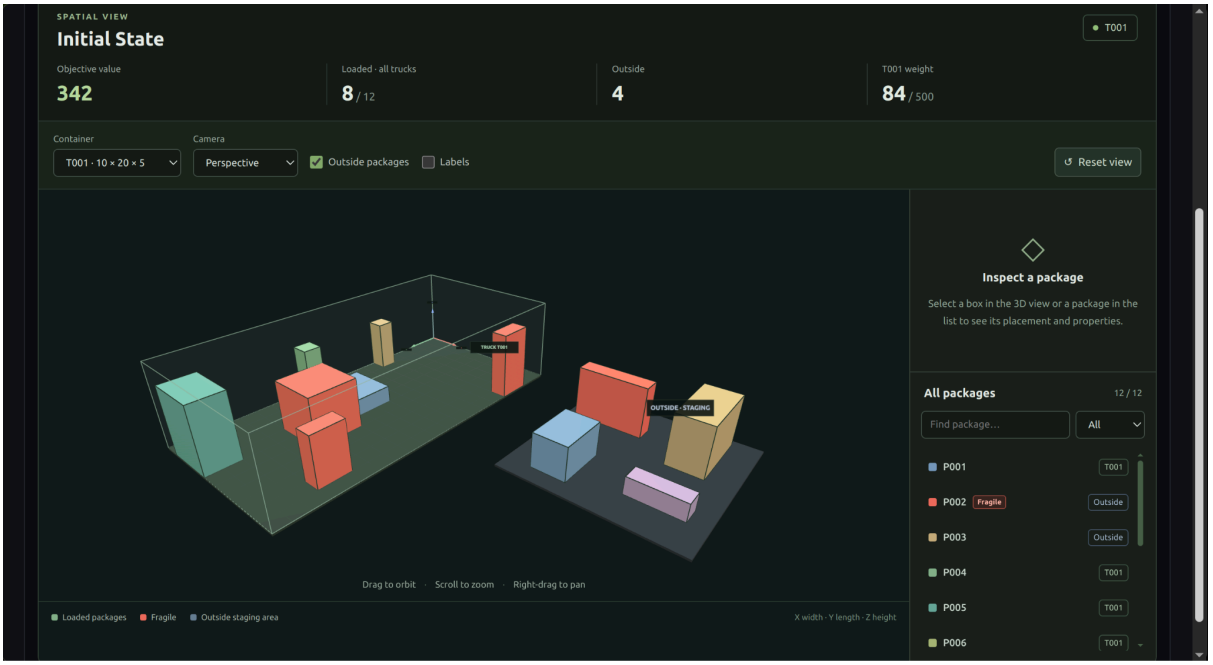

SPATIAL VIEW
Initial State
T001
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
T001
P003
T001
P004
T001
P005
Fragile
T001
P006
Fragile
Outside

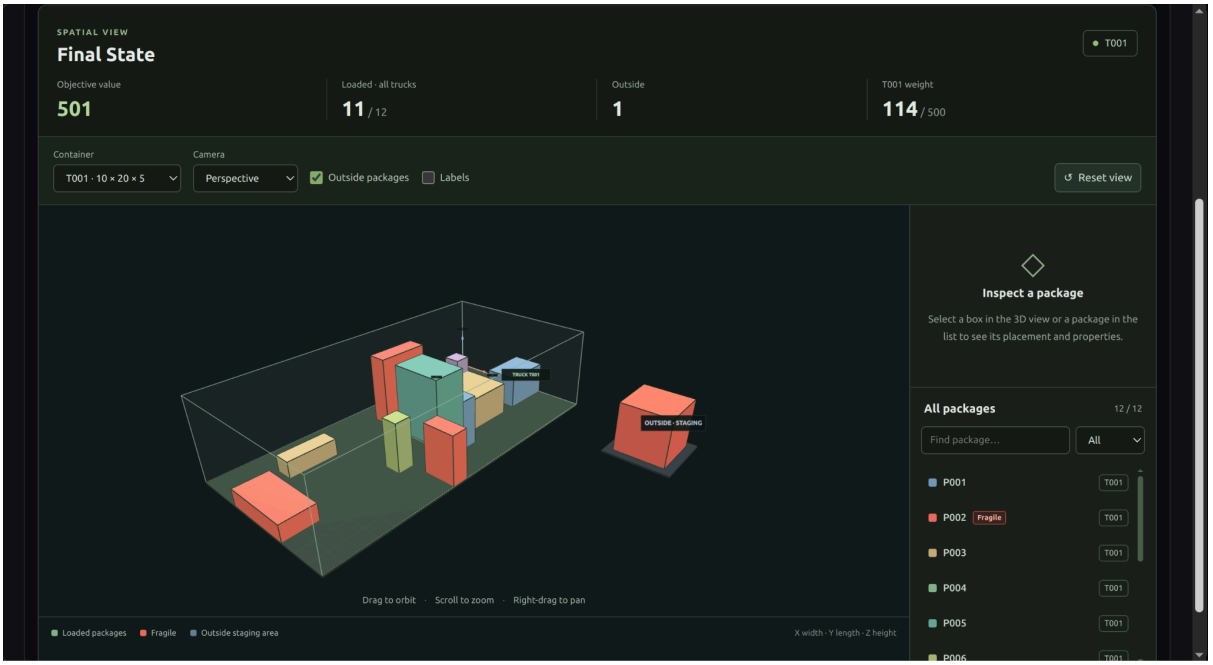

SPATIAL VIEW
Final State
T001
Objective value
621
Loaded · all trucks
12 / 12
Outside
0
T001 weight
127 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Fragile
P006
Fragile
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

Test Case 6 :

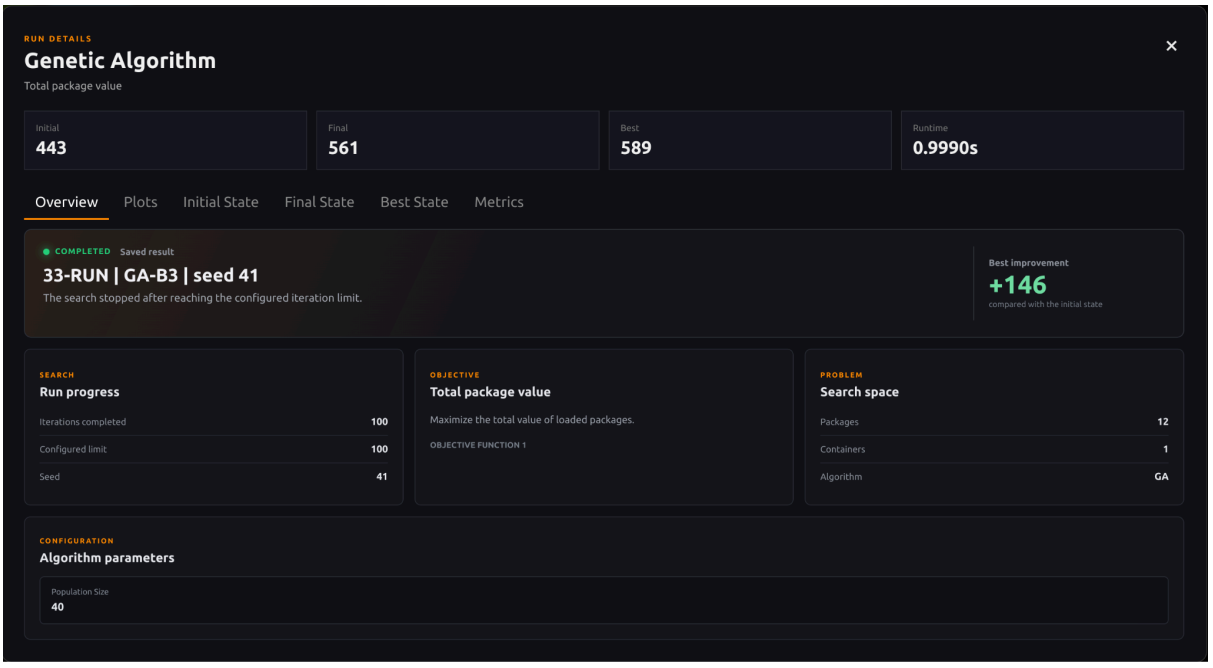

RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
501
Best
534
Runtime
0.7828s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-B2 | seed 43
The search stopped after reaching the configured iteration limit.
Best improvement
+192
compared with the initial state
SEARCH
Run progress
Iterations completed
100
Configured limit
100
Seed
43
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
GA
CONFIGURATION
Algorithm parameters
Population Size
30

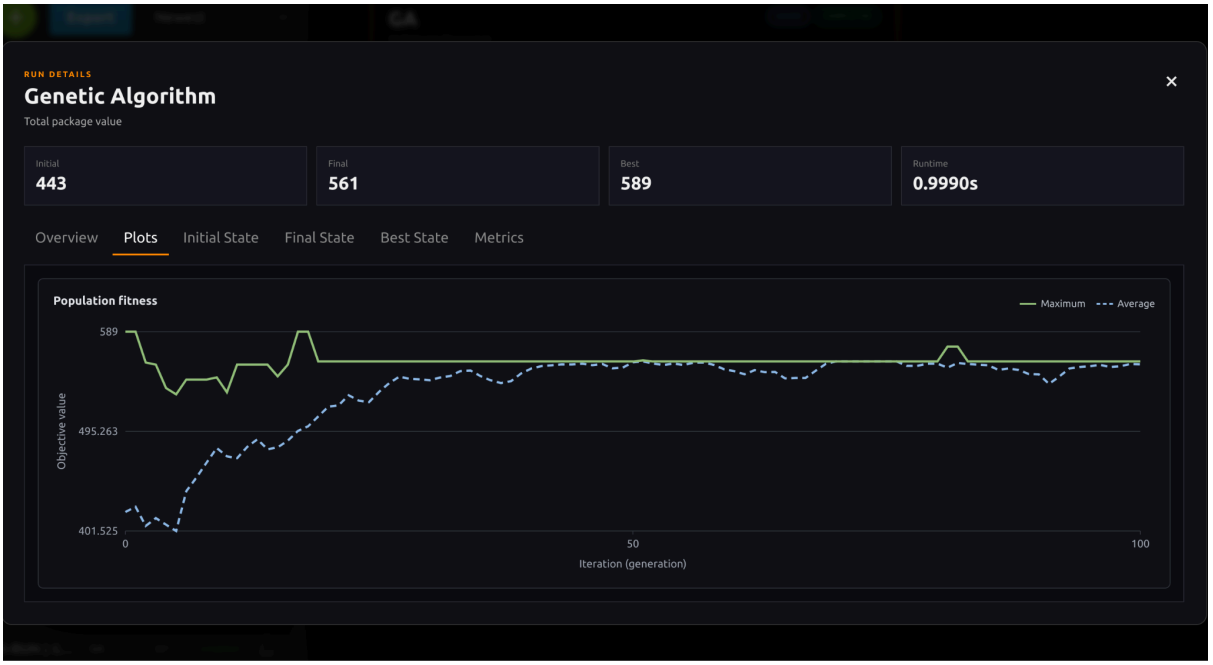
RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
501
Best
534
Runtime
0.7828s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
534
453.35
372.7
Objective value
0
50
100
Iteration (generation)

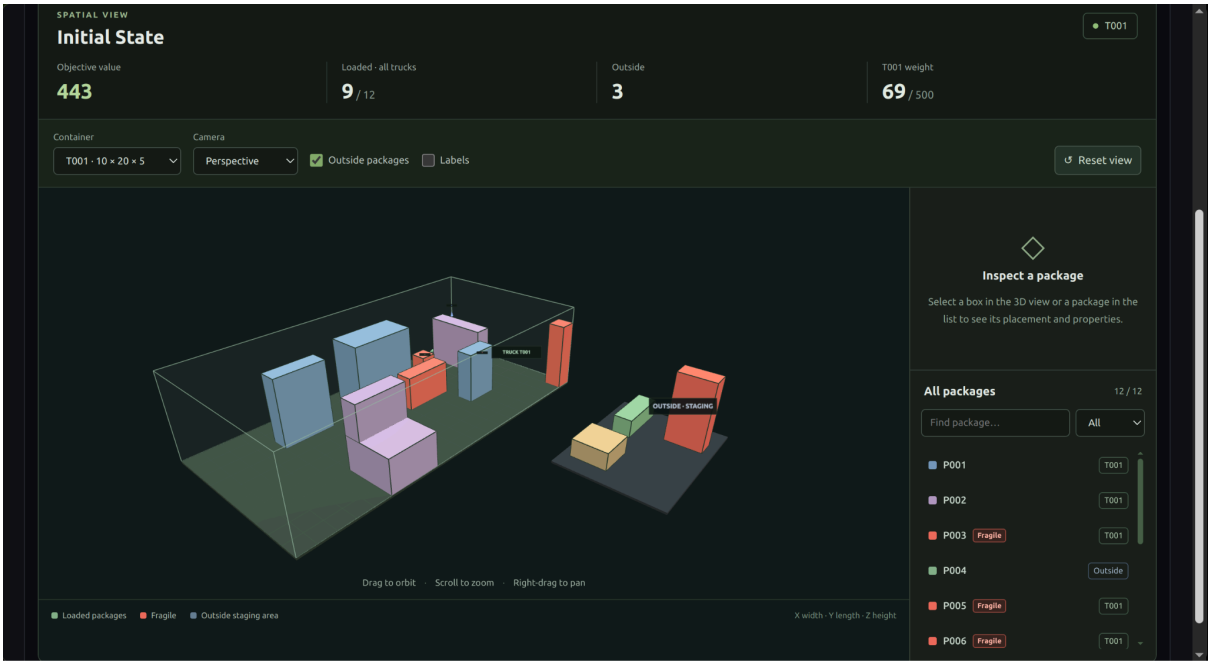

SPATIAL VIEW
Initial State
T001
Objective value
342
Loaded · all trucks
8 / 12
Outside
4
T001 weight
84 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
Outside
P003
Outside
P004
T001
P005
T001
P006
T001

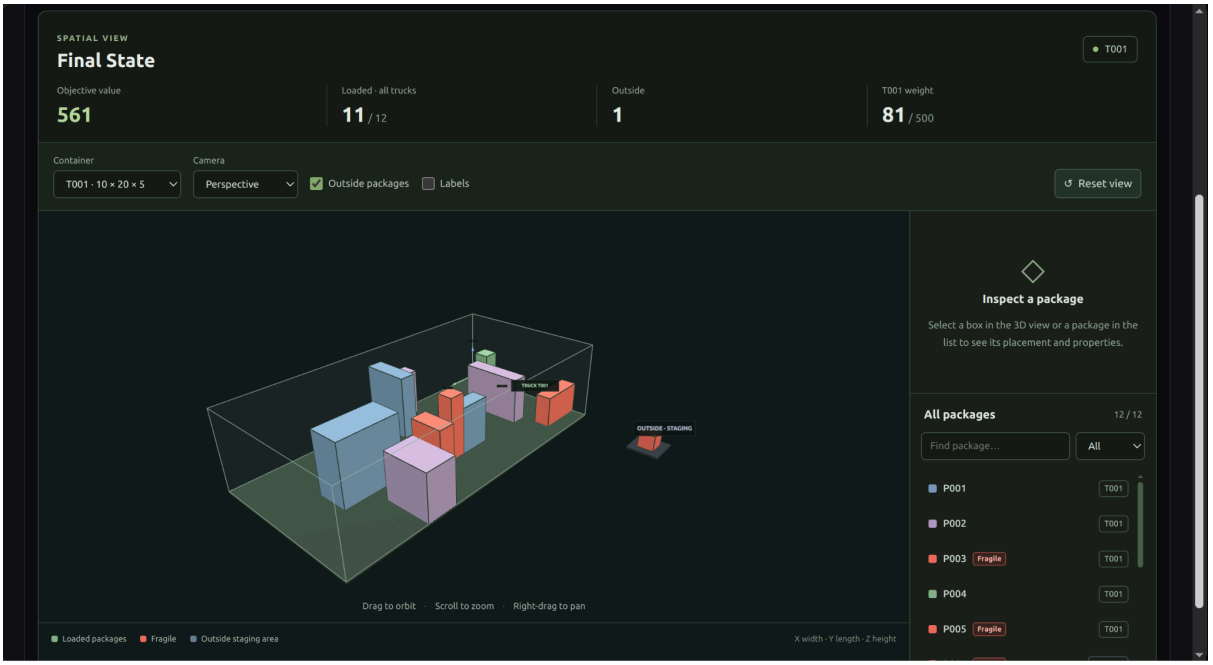

SPATIAL VIEW
Final State
T001
Objective value
501
Loaded · all trucks
11 / 12
Outside
1
T001 weight
114 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
T001
P003
T001
P004
T001
P005
T001
P006
T001

Test Case 7 :

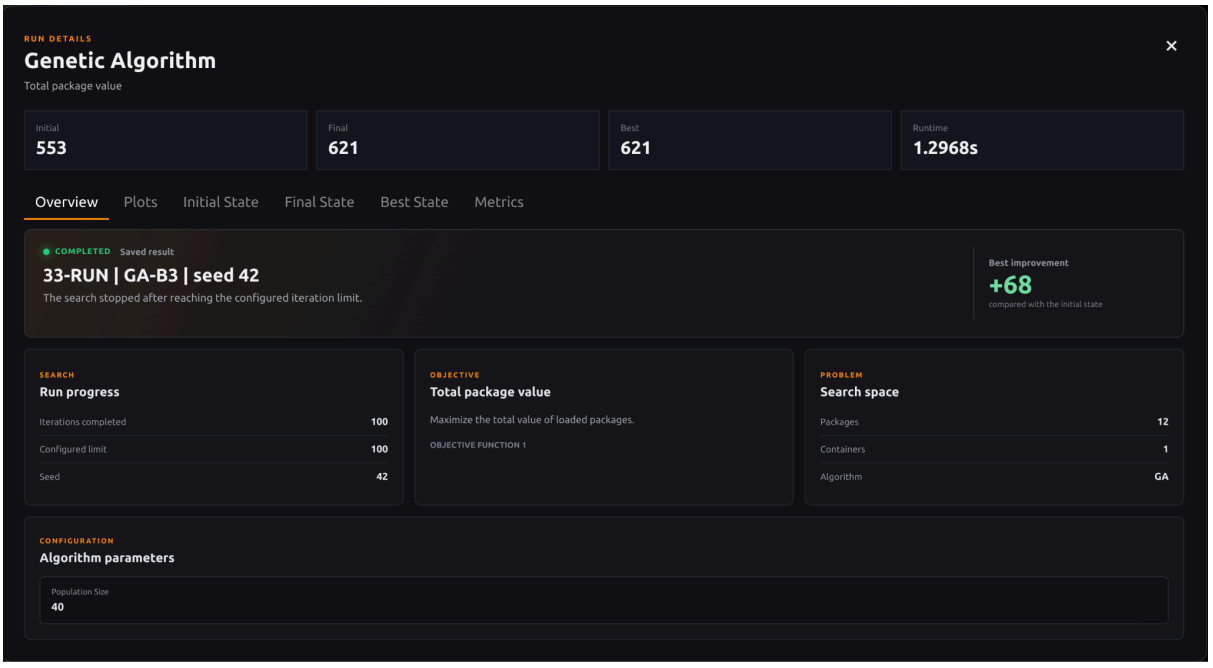

RUN DETAILS
Genetic Algorithm
Total package value
Initial
443
Final
561
Best
589
Runtime
0.9990s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-B3 | seed 41
The search stopped after reaching the configured iteration limit.
Best improvement
+146
compared with the initial state
SEARCH
Run progress
Iterations completed
100
Configured limit
100
Seed
41
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
GA
CONFIGURATION
Algorithm parameters
Population Size
40


RUN DETAILS
Genetic Algorithm
Total package value
Initial
443
Final
561
Best
589
Runtime
0.9990s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
589
495.263
401.525
Objective value
0
50
100
Iteration (generation)

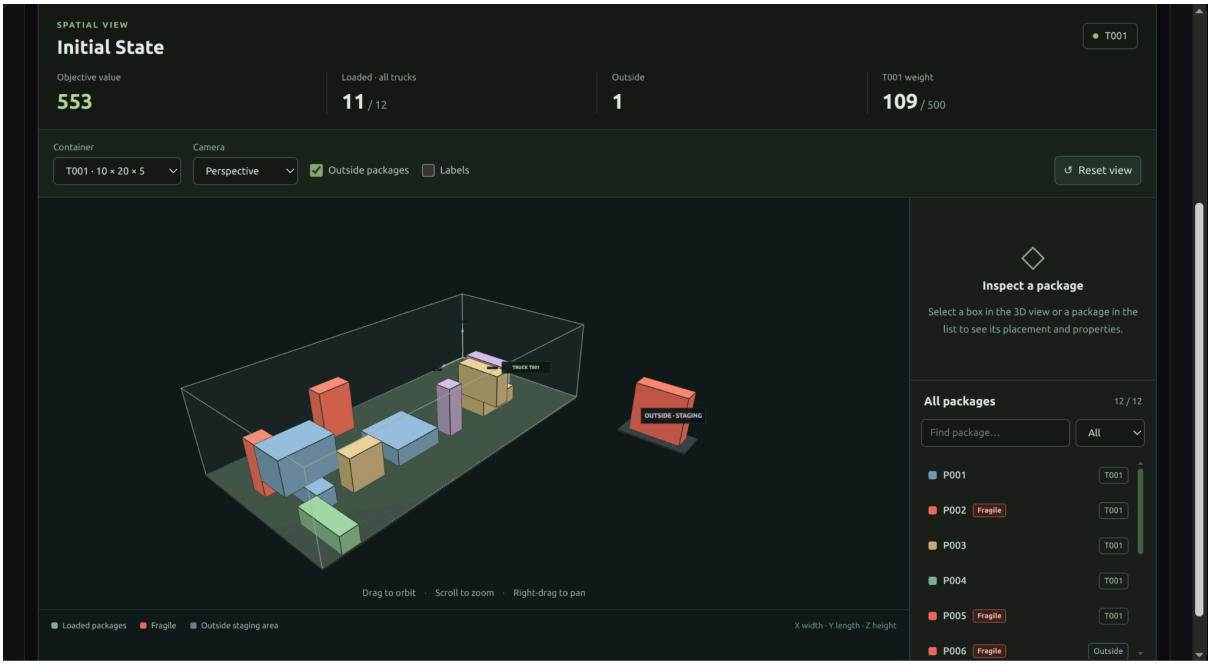

SPATIAL VIEW
Initial State
T001
Objective value
443
Loaded · all trucks
9 / 12
Outside
3
T001 weight
69 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
T001
P003
Fragile
T001
P004
Outside
P005
Fragile
T001
P006
Fragile
T001

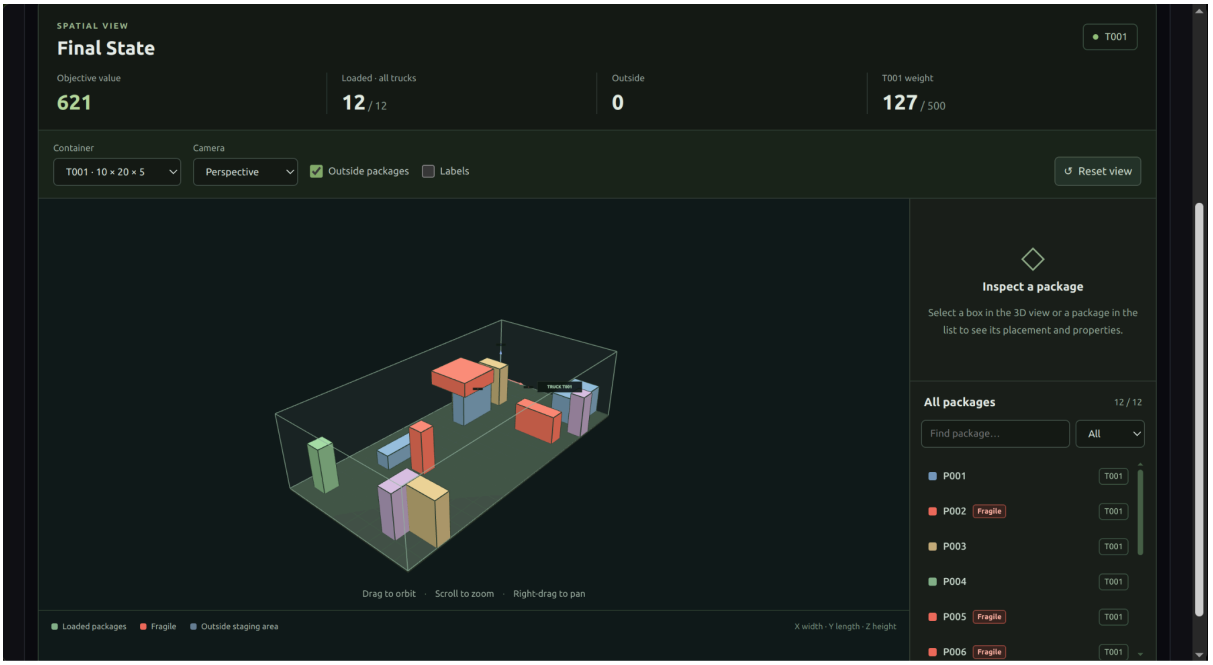

SPATIAL VIEW
Final State
T001
Objective value
561
Loaded · all trucks
11 / 12
Outside
1
T001 weight
81 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
P003
Fragile
P004
P005
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

Test Case 8 :

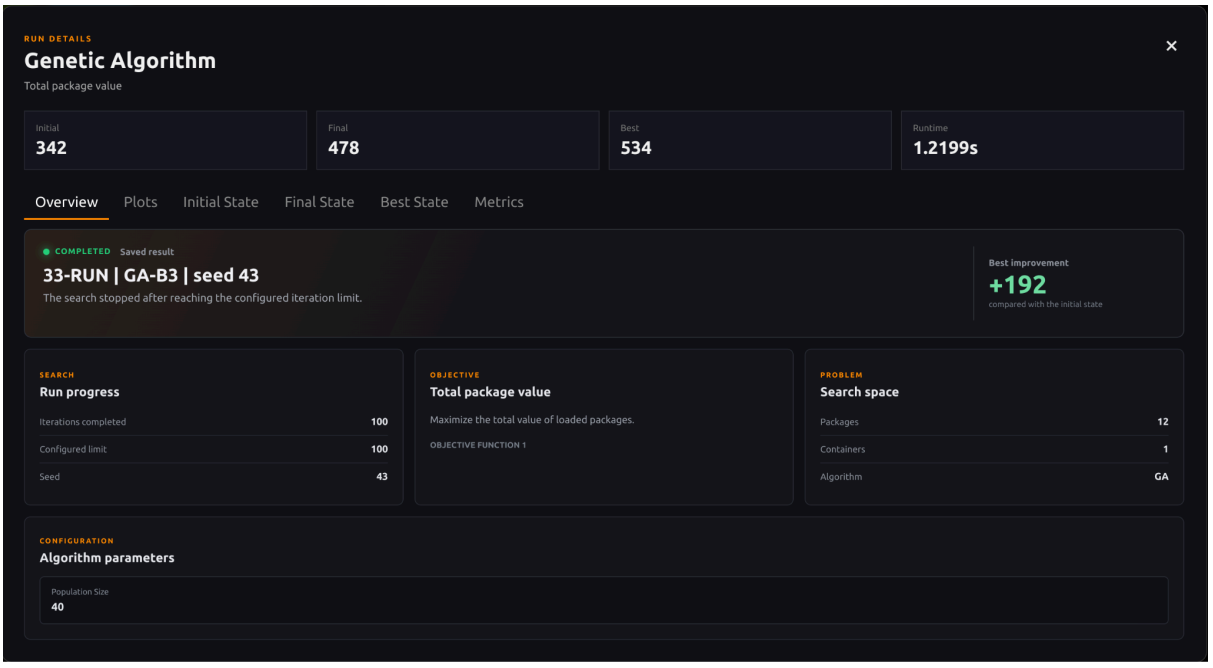

RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
621
Best
621
Runtime
1.2968s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-B3 | seed 42
The search stopped after reaching the configured iteration limit.
Best improvement
+68
compared with the initial state
SEARCH
Run progress
Iterations completed
100
Configured limit
100
Seed
42
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
GA
CONFIGURATION
Algorithm parameters
Population Size
40

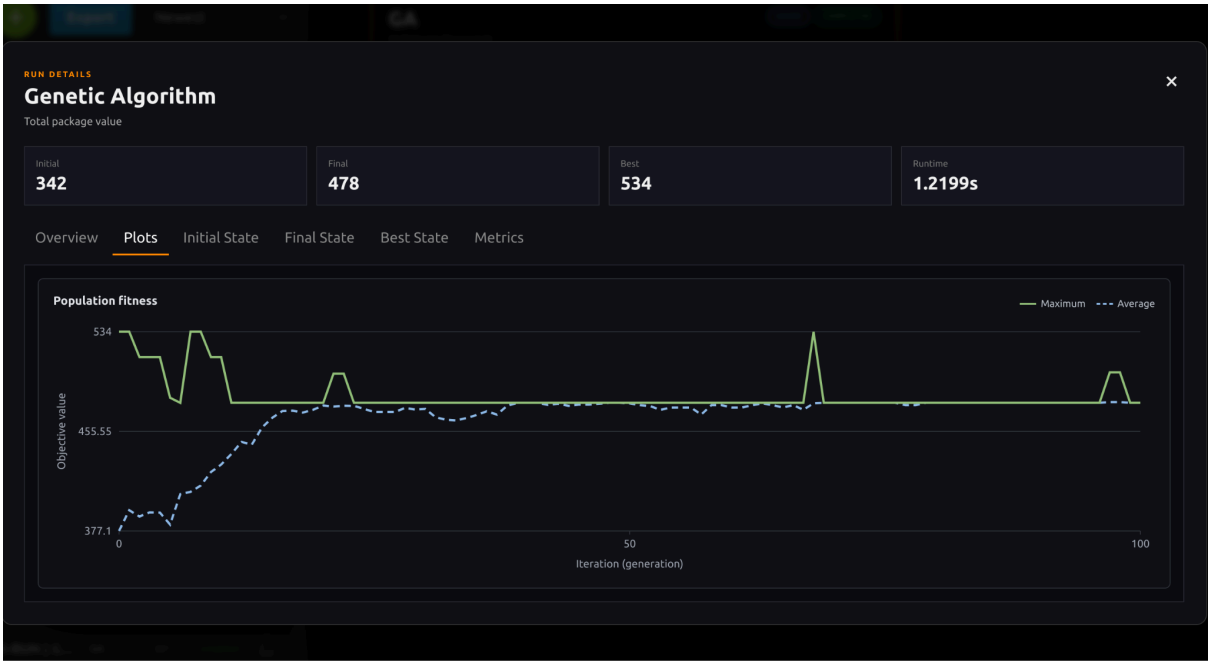
RUN DETAILS
Genetic Algorithm
Total package value
Initial
553
Final
621
Best
621
Runtime
1.2968s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
621
540.9
460.8
Objective value
0
50
100
Iteration (generation)

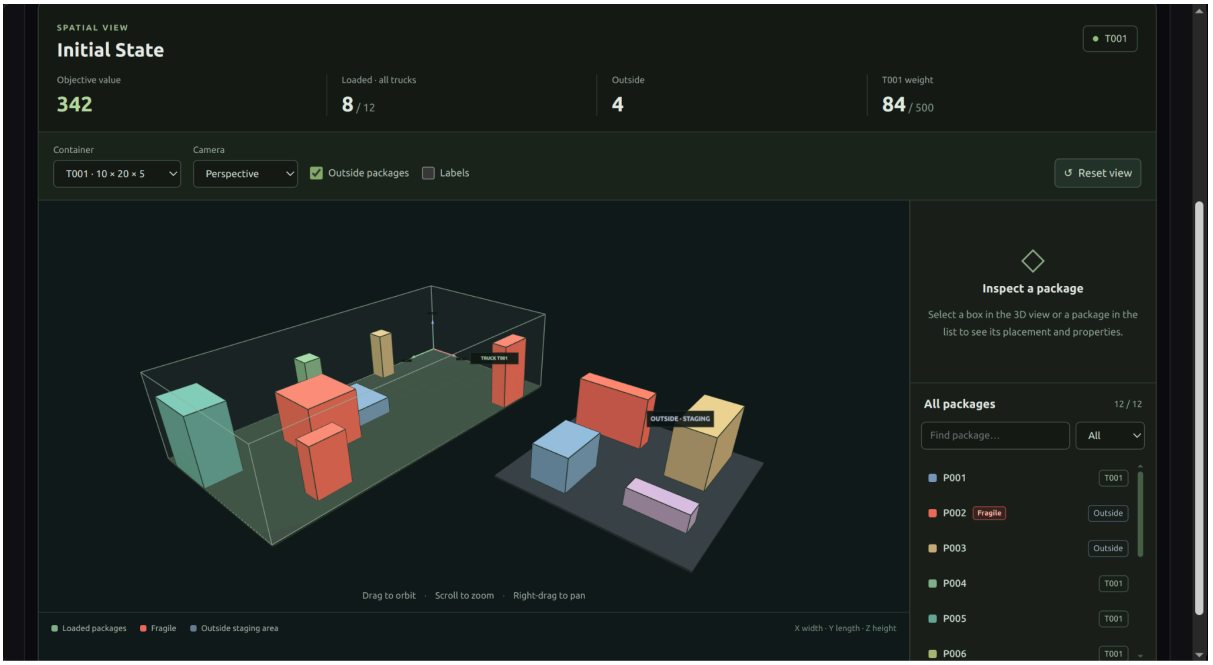

SPATIAL VIEW
Initial State
T001
Objective value
553
Loaded · all trucks
11 / 12
Outside
1
T001 weight
109 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
T001
P003
T001
P004
T001
P005
Fragile
T001
P006
Fragile
Outside

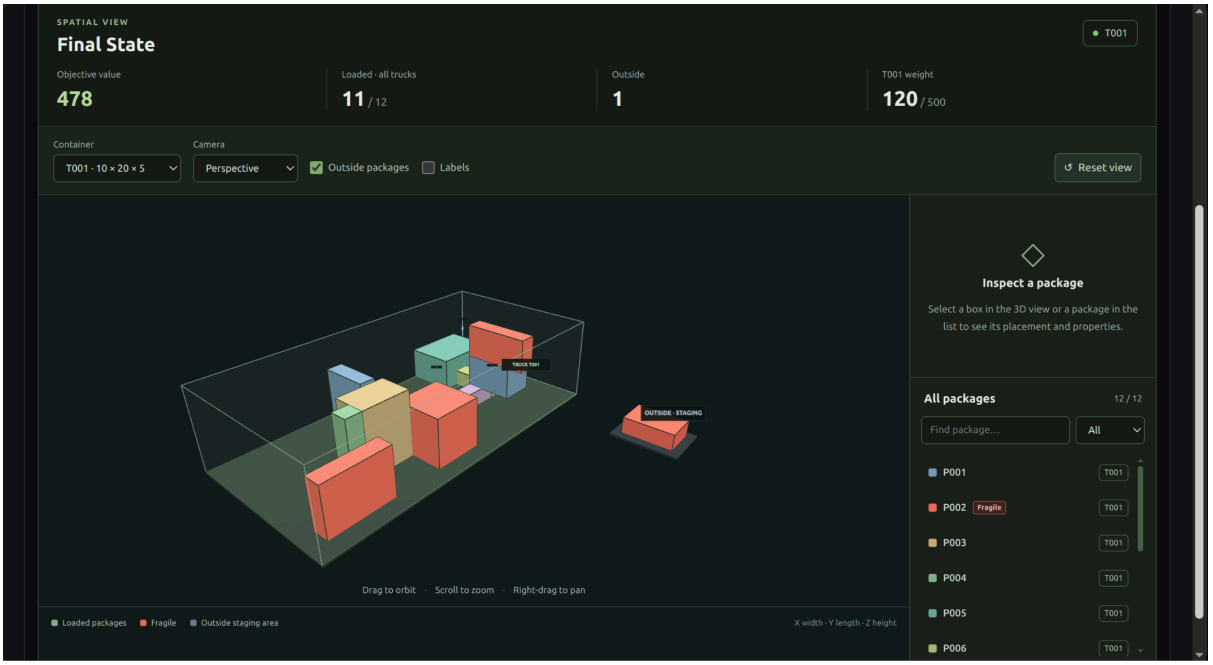

SPATIAL VIEW
Final State
T001
Objective value
621
Loaded · all trucks
12 / 12
Outside
0
T001 weight
127 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
P002
Fragile
P003
P004
P005
Fragile
P006
Fragile
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height

Test Case 9 :


RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
478
Best
534
Runtime
1.2199s
Overview
Plots
Initial State
Final State
Best State
Metrics
COMPLETED
Saved result
33-RUN | GA-B3 | seed 43
The search stopped after reaching the configured iteration limit.
Best improvement
+192
compared with the initial state
SEARCH
Run progress
Iterations completed
100
Configured limit
100
Seed
43
OBJECTIVE
Total package value
Maximize the total value of loaded packages.
OBJECTIVE FUNCTION 1
PROBLEM
Search space
Packages
12
Containers
1
Algorithm
GA
CONFIGURATION
Algorithm parameters
Population Size
40


RUN DETAILS
Genetic Algorithm
Total package value
Initial
342
Final
478
Best
534
Runtime
1.2199s
Overview
Plots
Initial State
Final State
Best State
Metrics
Population fitness
Maximum
Average
534
455.55
377.1
Objective value
0
50
100
Iteration (generation)


SPATIAL VIEW
Initial State
T001
Objective value
342
Loaded · all trucks
8 / 12
Outside
4
T001 weight
84 / 500
Container
T001 · 10 × 20 × 5
Camera
Perspective
Outside packages
Labels
Reset view
TRUCK T001
OUTSIDE · STAGING
Drag to orbit · Scroll to zoom · Right-drag to pan
Loaded packages
Fragile
Outside staging area
X width · Y length · Z height
Inspect a package
Select a box in the 3D view or a package in the list to see its placement and properties.
All packages
12 / 12
Find package…
All
P001
T001
P002
Fragile
Outside
P003
Outside
P004
T001
P005
T001
P006
T001



3.4. Analisis

3.4.1. Analisis Hill Climbing dan Simulated Annealing

Berikut adalah nilai objektif akhir yang dihasilkan oleh setiap algoritma :

Algoritma	Seed 41	Seed 42	Seed 43	Rata-rata
Steepest-Ascent	589	621	534	581.33
Stochastic	589	621	534	581.33
Sideways Move	589	621	534	581.33
Random Restart	589	621	534	581.33
Simulated Annealing	589	621	534	581.33

Dari hasil tersebut, terlihat bahwa setiap algoritma berhasil untuk mendapatkan nilai objektif maksimum pada seluruh dataset yang digunakan. Hal tersebut berarti setiap algoritma berhasil menemukan optimum global. Akan tetapi, hasil test case tidak terlihat variasi nilai objektif pada setiap algoritma. Oleh karena itu, perbandingan untuk setiap algoritmanya akan dilakukan dengan menggunakan waktu eksekusinya.

Berikut adalah waktu eksekusi setiap algoritma :

Algoritma	Seed 41 (s)	Seed 42 (s)	Seed 43 (s)	Durasi Rata-rata (s)
Stochastic HC	0.723	0.779	0.691	0.731

Simulated Annealing	0.925	1.019	0.925	0.956
Steepest-Ascent HC	8.573	5.842	8.44	7.618
Sideways Move HC	21.441	19.857	19.548	20.282
Random Restart HC	31.341	19.765	24.29	25.132

Dari tabel hasil tersebut, dapat dilihat bahwa Stochastic HC dan Simulated Annealing memiliki waktu eksekusi yang relatif lebih cepat dibandingkan algoritma lainnya. Hal itu bisa terjadi karena Stochastic HC dan Simulated Annealing membangkitkan neighbor secara acak sehingga biaya komputasi per iterasi relatif lebih rendah, dibandingkan algoritma lainnya yang membangkitkan seluruh neighbor lalu mengecek pada setiap iterasi

Perbandingan algoritma tersebut juga dapat dilihat dari titik pertama kali mereka menemukan global optimum. Berikut adalah total iterasi yang menghasilkan global optimum pertama untuk setiap algoritma :

Algoritma	Seed 41	Seed 42	Seed 43
Steepest-Ascent	3	1	4
Stochastic	122	158	252
Sideways Move	3	1	4
Random Restart	3	1	4
Simulated Annealing	211	276	416

Dari tabel tersebut, dapat dilihat bahwa hasilnya berkebalikan dengan waktu eksekusi. Algoritma yang lebih cepat dijalankan belum tentu lebih cepat menemukan global optimum. Stochastic HC dan Simulated Annealing punya waktu eksekusi yang singkat, tetapi membutuhkan lebih banyak iterasi sampai pertama kali menemukan global optimum. Sebaliknya, Steepest-Ascent HC, Sideways Move HC, dan Random Restart HC punya waktu eksekusi yang lebih lambat, tetapi mencapai global optimum hanya dalam beberapa iterasi.

Dengan demikian, Steepest-Ascent merupakan algoritma yang paling efisien berdasarkan jumlah langkah yang dibutuhkan untuk mencapai global optimum, sementara Stochastic adalah algoritma yang paling efisien dari segi waktu eksekusi.

3.4.2. Analisis Genetic Algorithm

3.4.2.1. Populasi sebagai kontrol

Berikut adalah hasil eksperimen untuk test case populasi sebagai kontrol :

Tabel nilai objektif akhir :

Iterasi	Seed 41	Seed 42	Seed 43	Rata-rata final
50	534	553	484	523.67
100	534	580	484	532.67
200	534	553	484	523.67

Tabel nilai objektif terbaik:

Iterasi	Seed 41	Seed 42	Seed 43	Rata-rata terbaik
50	589	621	534	581,33
100	589	621	534	581,33
200	589	621	534	581,33

Tabel durasi eksekusi :

Iterasi	Seed 41 (s)	Seed 42 (s)	Seed 43 (s)	Durasi rata-rata (s)
50	0.229	0.297	0.254	0.26
100	0.436	0.561	0.465	0.487
200	0.821	1.08	0.904	0.935

Dari tabel nilai objektif akhir tersebut, dapat dilihat bahwa penambahan jumlah iterasi tidak selalu menghasilkan nilai objektif akhir yang tetap atau meningkat. Dapat dilihat bahwa 100 iterasi memberikan rata-rata final 532.67, sementara 200 iterasi memberikan rata-rata final 523.67, yang dimana merupakan penurunan nilai objektif akhir.

Dari tabel nilai objektif terbaik, ketiga iterasi menghasilkan rata-rata yang sama, yaitu 581,33. Artinya, pada percobaan ini, menambah jumlah iterasi hingga 200 tidak menghasilkan nilai terbaik yang lebih tinggi dibandingkan 50 atau 100 iterasi. Nilai objektif akhir dan nilai objektif terbaik menggambarkan hal yang berbeda. Nilai terbaik menunjukkan solusi paling baik yang sempat ditemukan selama pencarian, sedangkan nilai akhir menunjukkan hasil pada akhir proses.

Dari tabel durasi eksekusi, dapat dilihat polanya konsisten. Semakin banyak iterasi, semakin lama waktu eksekusinya. Ini masuk akal karena semakin banyak iterasi, semakin banyak dijalankan proses pencariannya.

Secara keseluruhan, pada konfigurasi yang diuji, penambahan iterasi akan memperpanjang waktu eksekusi, tetapi tidak tentu meningkatkan nilai objektif terbaik. Hasil ini terbatas pada tiga seed yang dites, sehingga belum tentu menggambarkan pola yang sama untuk semua percobaan.

3.4.2.2. Iterasi sebagai kontrol

Berikut adalah hasil eksperimen untuk test case iterasi sebagai kontrol :

Tabel nilai objektif akhir :

Populasi	Seed 41	Seed 42	Seed 43	Rata-rata final
10	491	621	363	491,67
30	589	621	501	570,33
40	561	621	478	553,33

Tabel nilai objektif terbaik:

Populasi	Seed 41	Seed 42	Seed 43	Rata-rata terbaik
10	518	621	498	545,67
30	589	621	534	581,33
40	589	621	534	581,33

Tabel durasi eksekusi :

Populasi	Seed 41 (s)	Seed 42 (s)	Seed 43 (s)	Durasi rata-rata (s)
10	0.143	0.3	0.168	0.204
30	0.879	0.938	0.783	0.867
40	0.999	1.297	1.22	1.172

Dari tabel nilai objektif, terlihat bahwa peningkatan populasi dari 10 ke 30 memberikan peningkatan nilai objektif untuk tiap seed, tetapi peningkatan populasi dari 30 ke 40 memberikan penurunan nilai objektif.

Pada tabel nilai objektif terbaik, rata-rata terbaik yang tercatat juga paling tinggi pada populasi 30. Hasil Seed 41 dan Seed 43 menunjukkan nilai terbaik yang sama pada populasi 30 dan 40, sedangkan Seed 42 tetap memperoleh nilai 621.

Dari tabel durasi eksekusi, terlihat bahwa semakin besar jumlah populasi, semakin lama waktu yang dibutuhkan. Durasi rata-rata meningkat dari 0,204 detik pada populasi 10, menjadi 0,867 detik pada populasi 30, lalu 1,172 detik pada populasi 40. Hal ini terjadi karena semakin banyak state dalam populasi,semakin banyak pula yang perlu diproses pada setiap iterasi.

3.4.2.3. Perbandingan dengan Algoritma lainnya

Berdasarkan tabel nilai objektif akhir dan nilai objektif terbaik, Genetic Algorithm menunjukkan bahwa hasil pada akhir proses bisa berbeda dari solusi terbaik yang ditemukan selama pencarian.

Berdasarkan tabel yang sudah dipaparkan sebelumnya, Genetic Algorithm memiliki nilai objektif akhir yang lebih rendah daripada algoritma local search lainnya. Hampir seluruh test case Genetic Algorithm gagal (4 dari 18 test case berhasil) untuk mendapatkan optimum global sebagai hasil akhir, sementara setiap algoritma local search lainnya berhasil menemukan optimum global.

Akan tetapi, jika dibandingkan dengan nilai objektif terbaik, terlihat bahwa 16 dari 18 test case berhasil sampai ke global optimum. Hal tersebut menunjukkan bahwa permasalahan utama pada Genetic Algorithm diimplementasi ini adalah kemampuan untuk mempertahankan individu terbaik pada proses evolusi.

Meskipun demikian, Genetic Algorithm memiliki keunggulan dalam waktu eksekusinya. Test case yang membuahkan nilai objektif tertinggi memiliki waktu eksekusi yang lebih cepat daripada algoritma seperti Steepest Ascent, Sideways Move, dan Random Restart. Tergantung jumlah populasi dan iterasinya, Genetic Algorithm memiliki waktu eksekusi yang menyerupai Stochastic HC dan Simulated Annealing.

3.4.3. Analisis Kedekatan terhadap Global Optimum

Perhitungan kedekatan terhadap global optimum dilakukan dengan rumus :

Rasio Optimum (%) = (Nilai Objektif Akhir / Nilai Global Optimum) * 100%

Untuk kasus Genetic Algorithm akan dicek menggunakan nilai objective terbaik, karena nilai objective terakhir belum tentu merupakan nilai objective terbaik.

Berikut adalah tabel perhitungan kedekatan setiap algoritma terhadap global optimum :

Algoritma	Seed 41	Seed 42	Seed 43	Rata-rata final
Steepest-Ascent HC	100%	100%	100%	100%
Stochastic HC	100%	100%	100%	100%
Sideways Move HC	100%	100%	100%	100%
Random Restart HC	100%	100%	100%	100%
Simulated Annealing	100%	100%	100%	100%
GA-A1 (20 pop., 50 iter.)	100%	100%	100%	100%

GA-A2 (20 pop., 100 iter.)	100%	100%	100%	100%
GA-A3 (20 pop., 200 iter.)	100%	100%	100%	100%
GA-B1 (10 pop., 100 iter.)	87,95%	100%	93,26%	93,73%
GA-B2 (30 pop., 100 iter.)	100%	100%	100%	100%
GA-B3 (40 pop., 100 iter.)	100%	100%	100%	100%

Dari tabel tersebut, terlihat bahwa seluruh algoritma local search dan Simulated Annealing mencapai global optimum pada ketiga seed. Sebagian besar konfigurasi Genetic Algorithm juga mencapai 100%, kecuali di GA-B1.

Dengan demikian, GA-B1 menjadi satu-satunya konfigurasi pada tabel ini yang belum mencapai global optimum untuk semua seed. Hasilnya menunjukkan bahwa Genetic Algorithm juga dapat mencapai global optimum, tetapi keberhasilannya dapat dipengaruhi oleh pengaturan populasi dan seed yang digunakan.

Untuk Genetic Algorithm, hal ini dihitung menggunakan nilai objektif terbaik yang ditemukan selama proses pencarian, bukan nilai objektif akhir. Berikut merupakan tabel perhitungan jika Genetic Algorithm menggunakan nilai objektif akhir:

Algoritma	Seed 41	Seed 42	Seed 43	Rata-rata final
Steepest-Ascent HC	100%	100%	100%	100%
Stochastic HC	100%	100%	100%	100%
Sideways Move HC	100%	100%	100%	100%
Random Restart HC	100%	100%	100%	100%
Simulated Annealing	100%	100%	100%	100%
GA-A1 (20 pop., 50 iter.)	90.66%	89.05%	90.64%	90.12%

GA-A2 (20 pop., 100 iter.)	90.66%	93.40%	90.64%	91.57%
GA-A3 (20 pop., 200 iter.)	90.66%	89.05%	90.64%	90.12%
GA-B1 (10 pop., 100 iter.)	83.36%	100%	67.98%	83.78%
GA-B2 (30 pop., 100 iter.)	100%	100%	93.82%	97.94%
GA-B3 (40 pop., 100 iter.)	95.25%	100%	89.51%	94.92%

Dapat dilihat hanya 4 dari 18 testcase total untuk Genetic Algorithm yang mencapai global optimum.

3.4.4. Analisis Konsistensi Hasil Akhir

Berdasarkan tabel yang sudah dipaparkan pada poin-poin sebelumnya, terlihat bahwa Steepest-Ascent, Stochastic, Sideways Move, Random Restart, dan Simulated Annealing menunjukkan konsistensi hasil akhir yang baik. Semua algoritma tersebut berhasil untuk mendapatkan nilai optimum global pada setiap test case.

Sementara itu, Genetic Algorithm memiliki hasil yang bervariasi. Perbedaan populasi dan iterasi dapat memberikan hasil yang berbeda untuk test case yang sama. Pada test case dengan populasi sebagai kontrol, variasi hasil akhir relatif rendah, sementara pada test case dengan iterasi sebagai kontrol, variasi hasil akhir relatif tinggi. Hal tersebut menunjukkan bahwa Genetic Algorithm memiliki sensitivitas yang lebih tinggi terhadap perubahan populasi dibandingkan dengan perubahan iterasi.

4. Kesimpulan dan Saran

4.1. Kesimpulan

Berdasarkan pengerjaan tugas besar ini, dapat disimpulkan bahwa algoritma local search dapat diterapkan untuk menyelesaikan persoalan optimasi penempatan paket ke dalam truk. Algoritma Hill Climbing beserta keempat variannya, Simulated Annealing, dan Genetic Algorithm telah berhasil diimplementasikan untuk mencari konfigurasi penempatan paket yang memaksimalkan nilai fungsi objektif dengan tetap memenuhi batasan dimensi, kapasitas berat, penopang paket, serta aturan barang pecah belah.

Program yang dikembangkan juga telah berhasil mengintegrasikan proses optimasi dengan aplikasi berbasis web yang memungkinkan pengguna mengatur parameter eksperimen, menjalankan algoritma, membandingkan hasil pencarian, serta melihat konfigurasi paket melalui visualisasi tiga dimensi. Selain itu, pengembangan fungsi objektif alternatif dan fitur multi-truk memperluas kemampuan program dalam menyelesaikan berbagai variasi persoalan optimasi logistik.

Secara keseluruhan, tugas besar ini memberikan pemahaman mengenai penerapan algoritma local search pada persoalan optimasi yang memiliki banyak kemungkinan solusi dan batasan. Implementasi yang dilakukan juga menunjukkan bahwa setiap algoritma memiliki mekanisme pencarian yang berbeda sehingga pemilihan algoritma dan parameternya perlu disesuaikan dengan karakteristik persoalan yang dihadapi.

4.2. Saran

Untuk pengembangan selanjutnya, program dapat ditingkatkan dengan melakukan optimasi pada mekanisme pembangkitan neighbor dan evaluasi state agar proses pencarian dapat berjalan lebih efisien, terutama ketika jumlah paket dan truk semakin besar.

Selain itu, Genetic Algorithm dapat dikembangkan dengan menambahkan mekanisme elitism untuk mempertahankan individu terbaik pada setiap generasi. Pengujian terhadap dataset yang lebih kompleks serta variasi parameter yang lebih luas juga dapat dilakukan untuk mengevaluasi kemampuan setiap algoritma pada kondisi persoalan yang lebih beragam.

5. Pembagian Tugas

Nama	Tugas
Audric (13524010)	● Algoritma Hill Climb ● Bonus Hill Climb ● Visualisasi ● Eksperimen dan Analisis
Mahatma (13524015)	● Algoritma Genetic Algorithm ● Bonus Objective Function
Ishak (13524022)	● Algoritma Simulated Annealing ● Bonus Multi-Truck ● Visualisasi ● Arsitektur Program

6. Lampiran

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM : 1. Audric Yusuf Maynard Simatupang/13524010
2. Mahatma Brahmana/13524015
3. Ishak Palentino Napitupulu/13524022
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas :Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Lingkup Pengerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	No AI
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	No AI
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	AI-Assisted Idea Generation and structuring
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon,	Tidak	No AI

DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan dinyatakan di dalam surat ini.

Audric Yusuf Maynard Simatupang

Ishak Palentino Napitupulu

Mahatma Brahmana